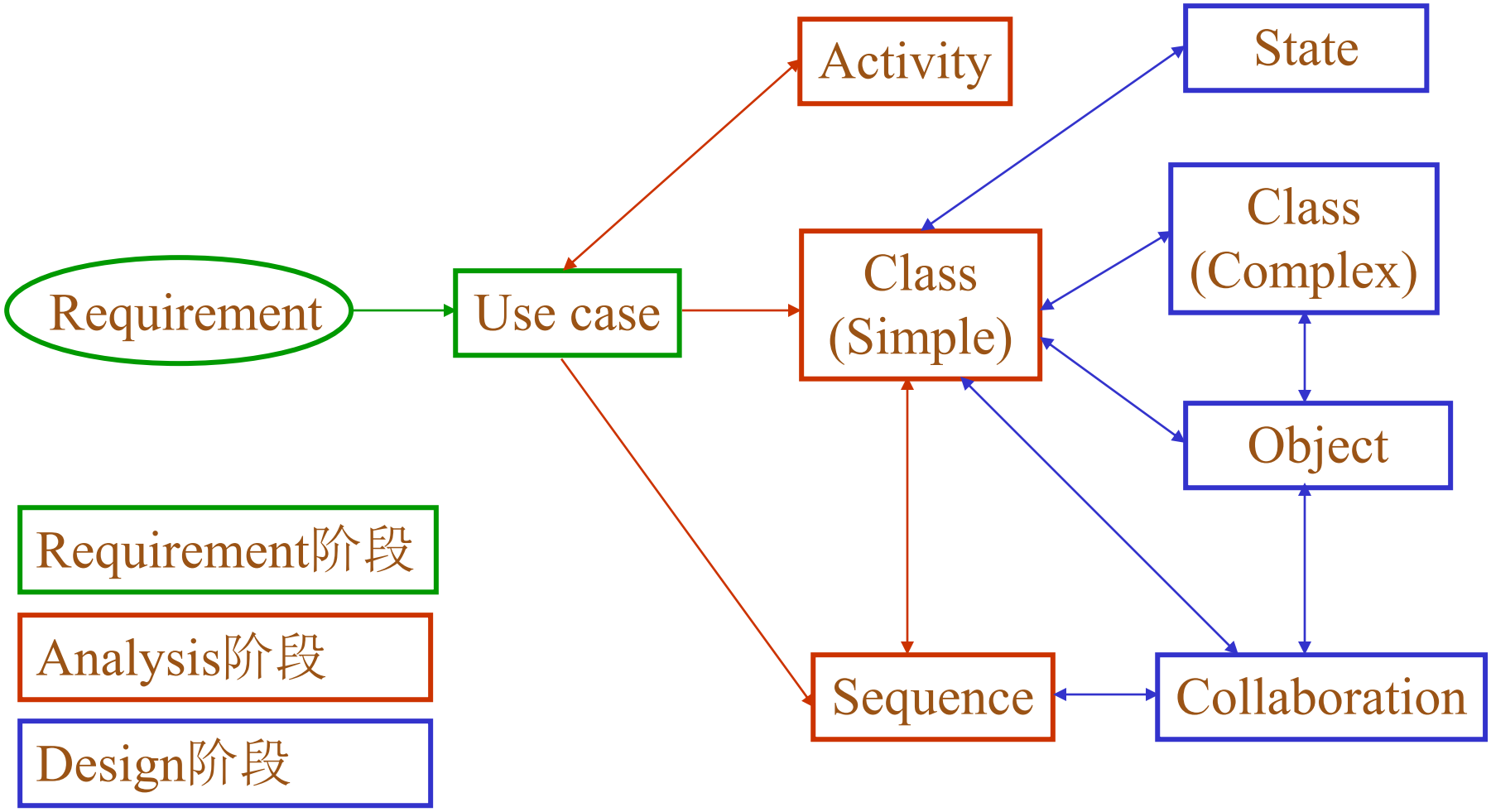
第9章 顺序图和协作图

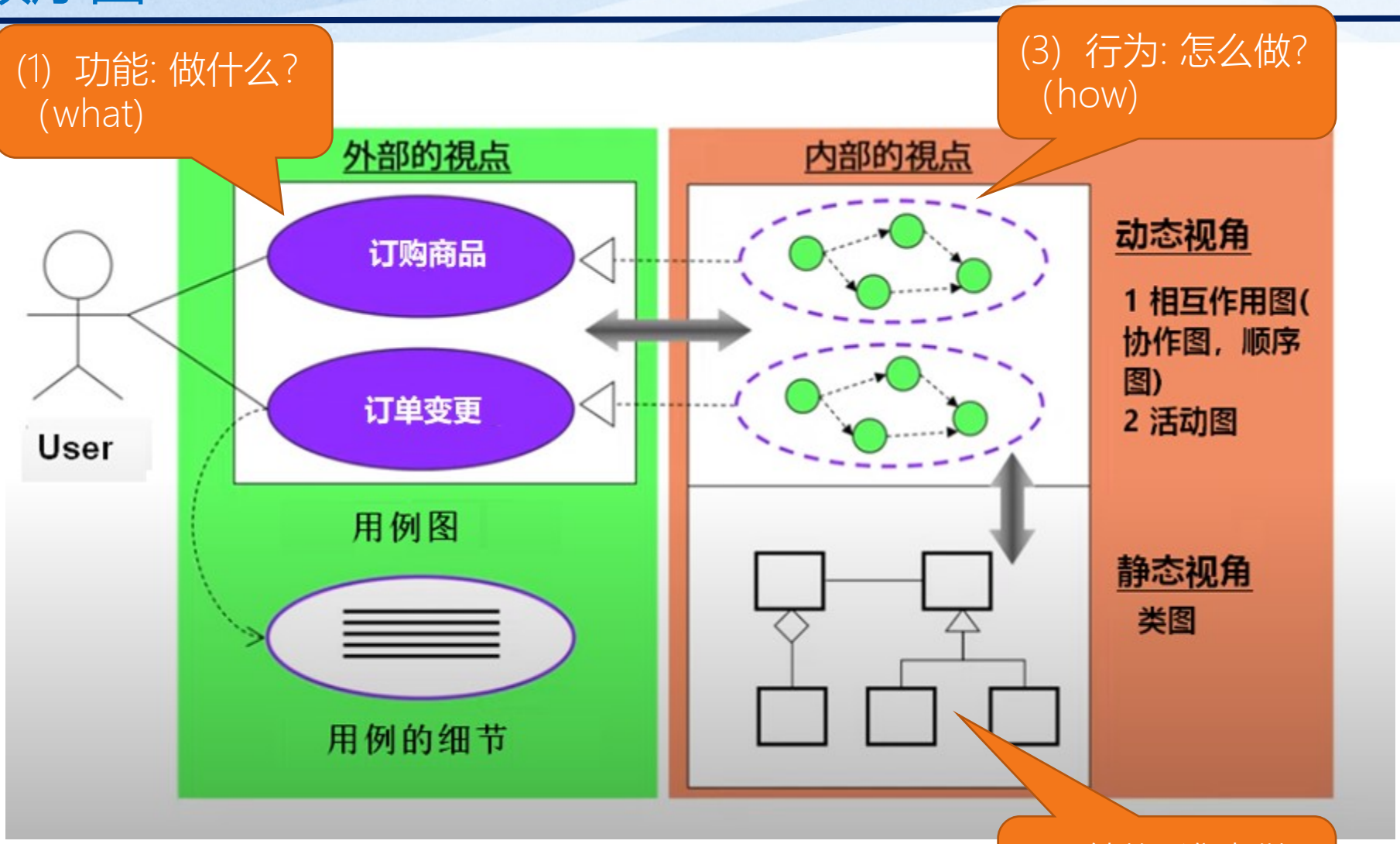
2024/4/21

- 顺序图和协作图都属于交互建模，顺序图描述对象之间按照时间顺序进行通信，协作图描述对象之间的相互作用，它们是面向对象建模的一种重要的模型
- 学习完本章节，要求达到以下状态：
  - 能够说明交互图的基本概念
  - 能够说明顺序图的表示方法和使用方法
  - 能够读懂顺序图并理解其中的含义

UML 1.x中各种图的关系



顺序图



- 顺序图显示为了完成用例的功能，类之间消息交互的状况。

交互图(interaction diagram)

- 在对软件密集型(software-intensive)系统建模时，如何对它的**动态方面**建模？怎样才能可视化一个运行的系统？
- **交互图(interaction diagram)**是一种详细表示对象之间以及对象与系统外部的参与者(actor)之间动态联系的图形文档。
- UML 1.x版本中，交互图有两种形式，即**顺序图(sequence diagram)** 和**协作图(collaboration diagram)** 。

交互图说明：

- **交互图**是用来描述对象之间的动态协作关系以及协作过程中的行为次序，它常常用来描述一个用例的行为，显示该用例中所涉及的对象和这些对象之间的消息传递情况。
- 一般一个use case需要一个或多个顺序图或协作图，除非特别简单的use case.
- **顺序图**和**协作图**从不同的角度表达了系统中的交互和系统的行为，它们之间可以相互转化。

  *有些复杂的顺序图可能无法通过等效的通信图来完全表示：包含 interaction use, combined fragments,或者发生了 message overtaking 。
- **顺序图**着重描述对象按照时间顺序的消息交换，**协作图**着重描述系统成分如何协同工作。

交互图的用途

- 帮助分析员对照检查每个use case 中描述的用户需求，是否已经落实到一些能够完成这些功能的类中去实现。提醒分析员去补充遗漏的类或服务。
- 和**类图**相互补充
  - 类图对对象之间的消息（交互情况）表达不够详细；
  - 交互图表示少数几个对象之间的交互。

交互图

- 表示“人”或者“物”(生命线lifeline)之间的消息交互。。
- 根据用例图中的脚本来做成交互图。
- (简单的)顺序图和协作图之间可以互相转换，根据需要选择使用其中一种。

- 对象间的相互作用体现了对象的行为。这种相互作用可以描述成两种互补的方式，一种以对象为中心进行考察，另一种以互相作用的一组对象为中心进行考察。
- 顺序图以对象为中心描述一组对象的整体行为。协作图以对象间协作关系为中心描述一组对象的整体行为。

9.1.1 协作（Collaboration）

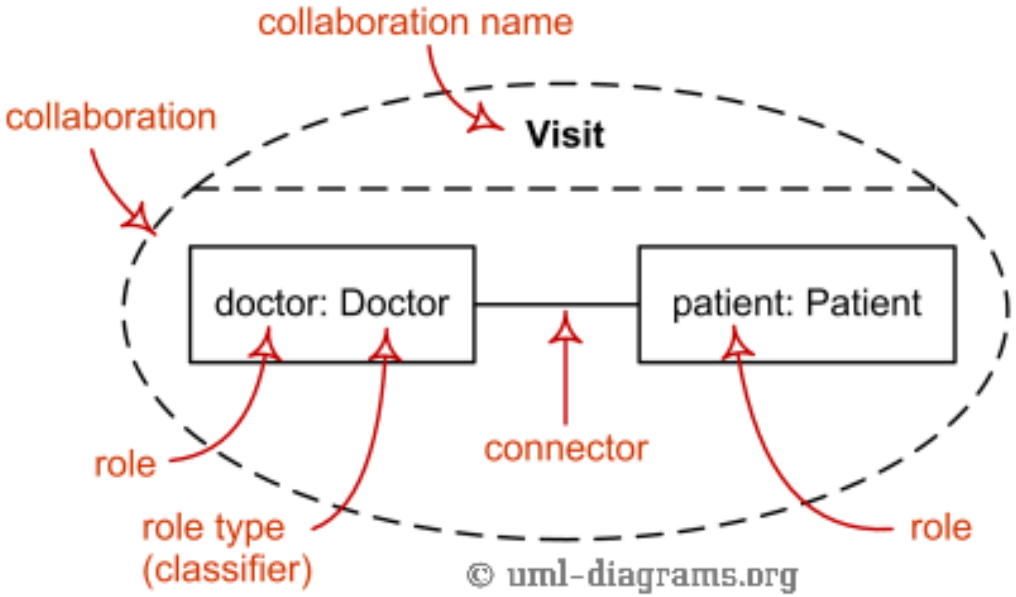
- 协作描述了在一定的场景下一组对象以及用以实现某些行为的这些对象间的相互作用。它描述了为实现某种目的而相互合作的对象的“社会”。协作中的类角色表示参与协作执行的对象的描述，关联角色表示参与协作执行的关联的描述。类角色是在协作中被部分约束的类；关联角色是在协作中被部分约束的关联。协作中的类角色与关联角色之间的关系只在特定的场景下才有意义。通常，同样的关系不适用于协作外的潜在的类和关联
- 系统中的对象可以参与一个或多个协作。虽然协作的执行通过共享对象相连，但是对象所出现的协作不必直接相关

9.1.1 协作（Collaboration）

- 协作包括结构和行为两个方面。
  - 结构方面：包含一个角色集合和它们之间的关系，这些关系定义了行为方面的内容。
  - 行为方面是一个消息集合，这些消息在具有某一角色的各对象间进行传递交换。协作中的消息集合叫作交互。一个协作可以包含一个或多个交互，每个交互描述了一系列消息，协作中的对象为了达到目标而交换消息。

9.1.1 协作（Collaboration）

- 协作显示在composite structure diagram中。



Collaboration elements - roles, parts, connectors.
Collaboration Visit shows cooperation of doctor and patient roles.

9.1.2 交互

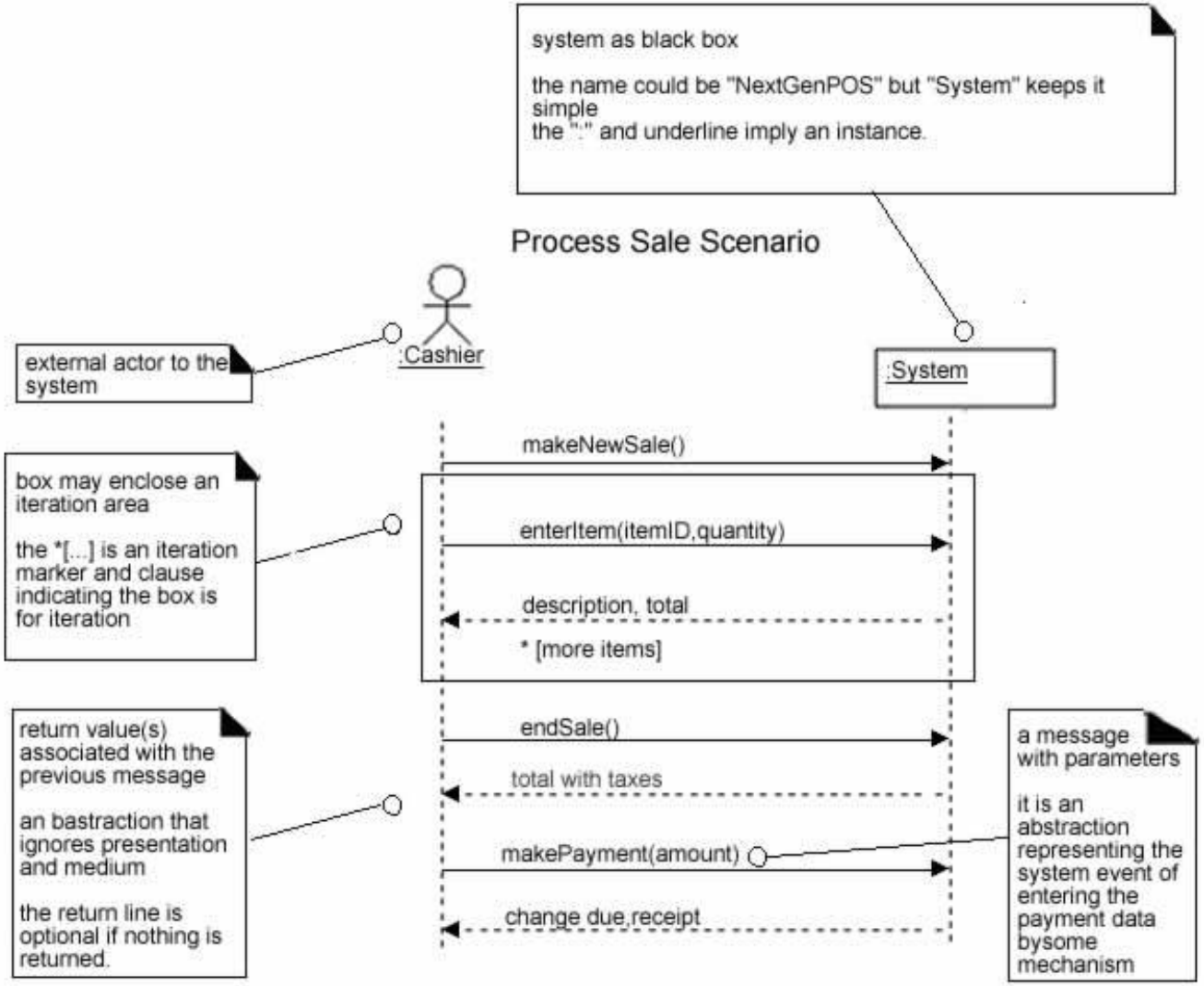
- 交互是一个消息集合，这些消息被类角色通过关联角色进行交换。当协作的对象在运行时，受类角色约束的对象通过受关联角色约束的链接交换消息实例。交互作用可对操作的执行、用例或其他行为实体建模。
- 消息是两个对象之间的单向通信，即从发送者到接收者的控制信息流。消息具有在对象间传值的参数。消息可以是信号，即一种明确的、命名的、对象间的异步通信；消息也可以是函数调用，即可以具有传入参数并具有返回控制机制的同步通信。
- 消息可以被组织成顺序的控制线程。分离的线程代表并发的几个消息集合。线程间的同步通过不同线程间消息的约束建模。同步结构能够对分叉控制、合并控制和分支控制建模。
- 消息序列可以用两种图来表示，即顺序图和协作图。

系统顺序图和顺序图

- **系统顺序图**（System Sequence Diagram）把待建系统看成一个黑盒子，研究参与者与系统边界的交互。描述的是在特定的用例场景下，外部参与者、内部系统、以及他们之间的事件消息传递
- **顺序图**（Sequence Diagram）又叫时序图，是用于描述对象间交互以及这些对象在时间上的排列顺序的一种图。是显示对象之间交互的图，这些对象是按时间顺序排列的。顺序图中显示的是参与交互的对象及其对象之间消息交互的顺序。

系统顺序图的例子

■显示参与者和系统之间的交互

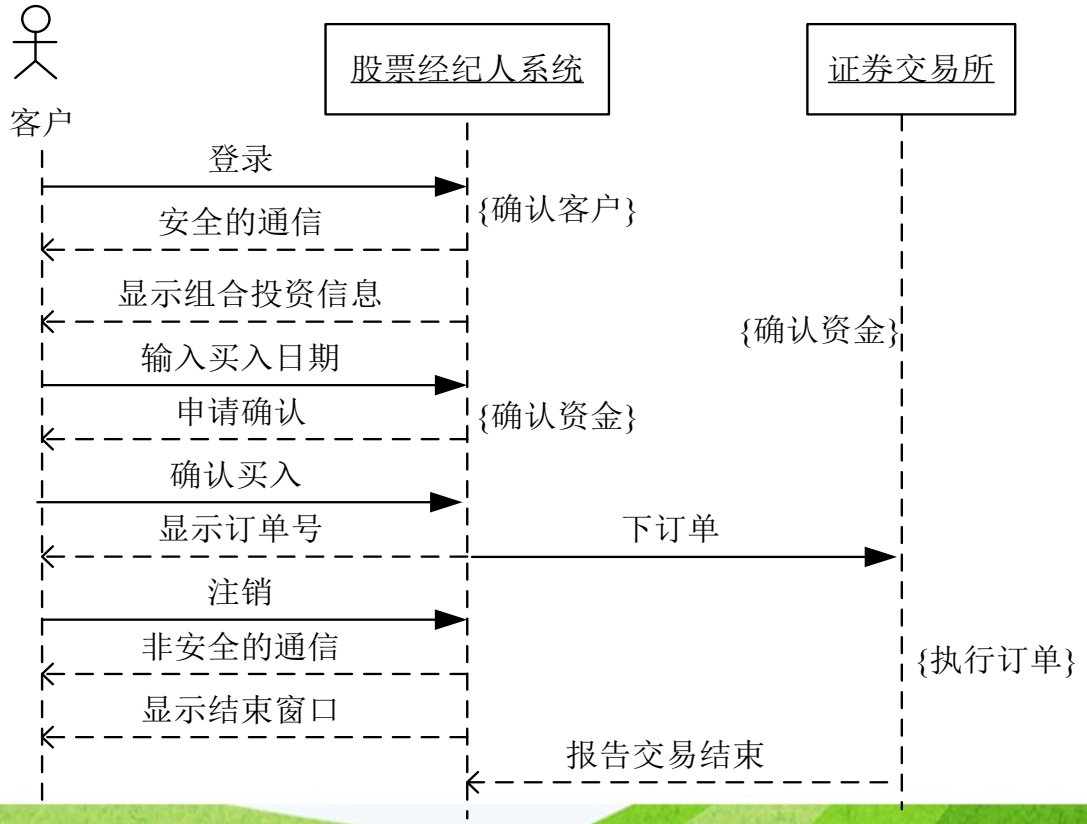


9.2 顺序图（Sequnce Diagram）

- 顺序图是显示按时间顺序排列的对象交互的图。特别是，它显示了参与交互的对象和交换的消息序列。顺序图显示了交互的参与者以及参与者之间的消息传递顺序。顺序图也显示了系统为了执行全部或者部分用例而与其参与者的交互。
- UML建模时，顺序图将交互关系表示为一个二维图。纵向是时间轴，时间沿竖线向下延伸。横向轴代表了各独立对象的类角色。类角色用生命线表示。当对象存在时，角色用一条虚线表示，当对象的过程处于激活状态时，生命线是一个双道线。

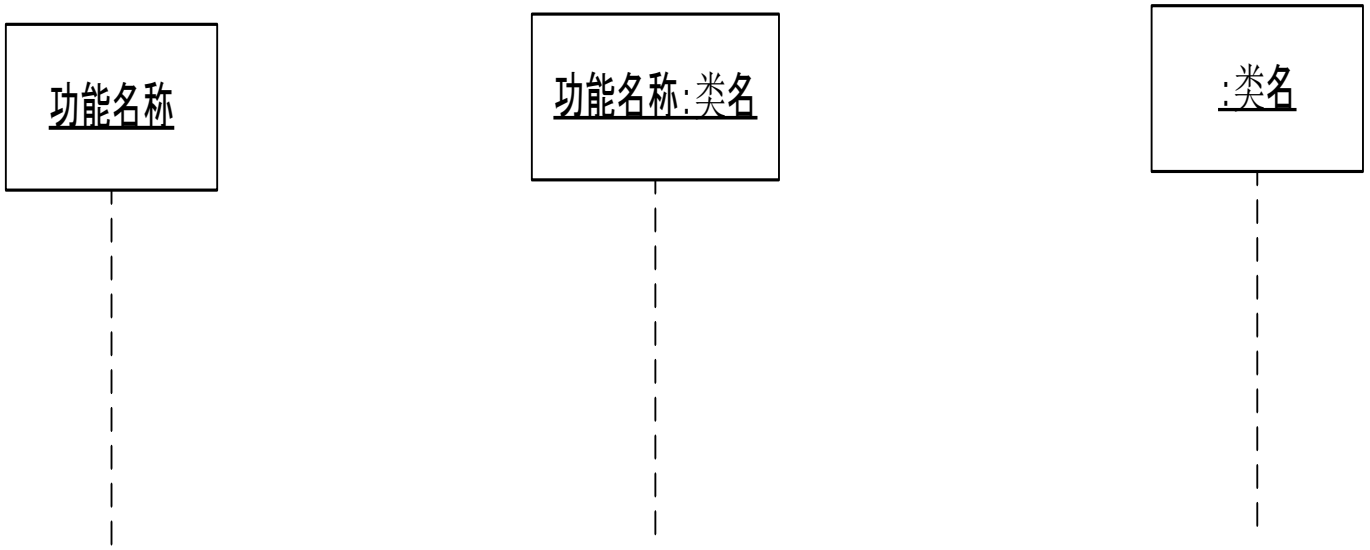
9.2顺序图-概要

- 按时间顺序来表示生命线之间的消息交互。
  - 可用来表示用例实现的时候类或者组件之间的交互情况。
- 顺序图的特征：
  - 消息从上到下，从左到右进行排列，处理顺序一目了然。
- 顺序图的组成元素：
  - 生命线
  - 消息



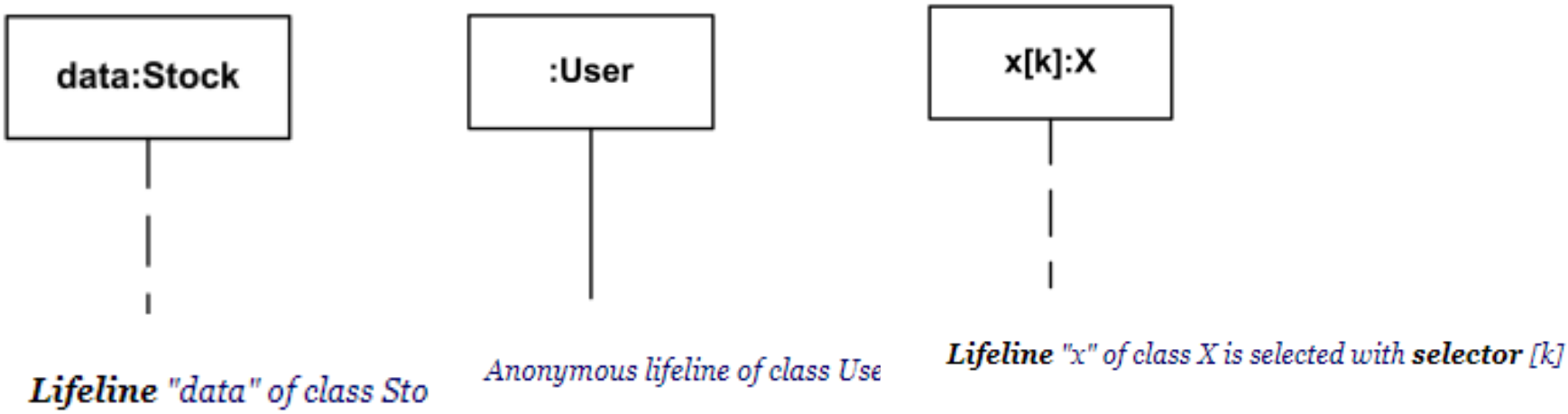
9.2.1生命线

- 表示参与交互活动的元素（对象，组件等实体）
- 生命线的表示方法
  - 矩形框下加一条虚线来表示，矩形框中记述生命线的名称。虚线表示生命线的存在。
  - 生命线的名称有三种表示方法：



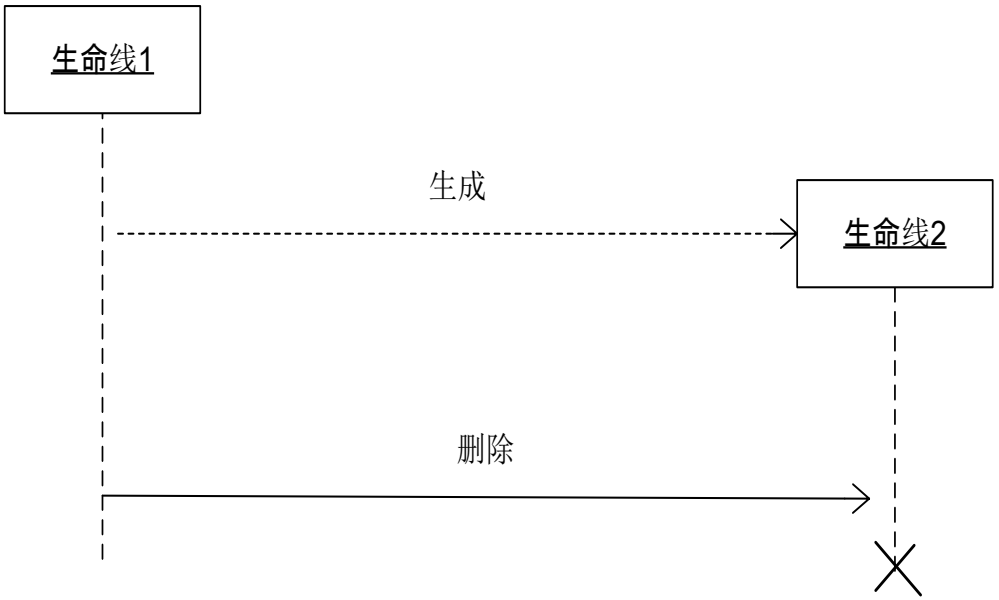
9.2.1生命线

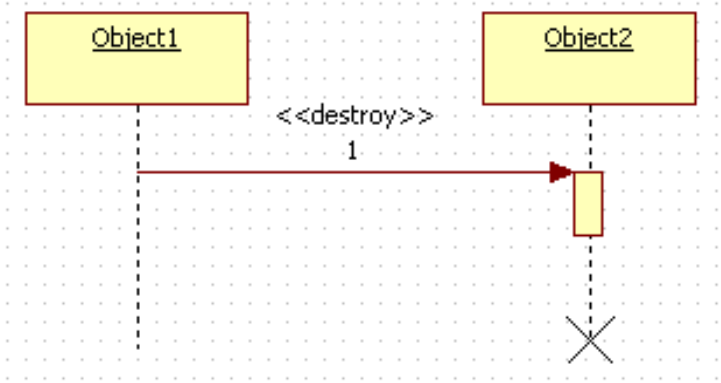
- UML规范只是规定生命线may be dashed。也有划为实线的。当然虚线更流行。
- 对象名的下划线是一种习惯，并非规范要求。



9.2.1生命线

- 生命线的生成和消失
  - 生命线名称矩形框下面的虚线表示生命线的生存期间。
  - 生成一个生命线的消息要用虚线箭头表示，新生成的生命线连接在箭头方。
  - 在生命线的末端用“X”符号表示生命线在此刻消失。

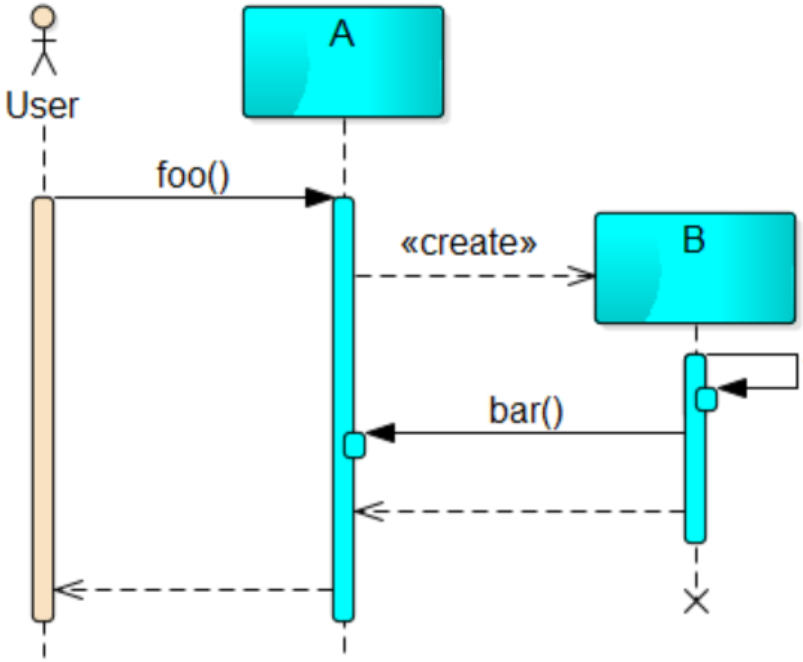


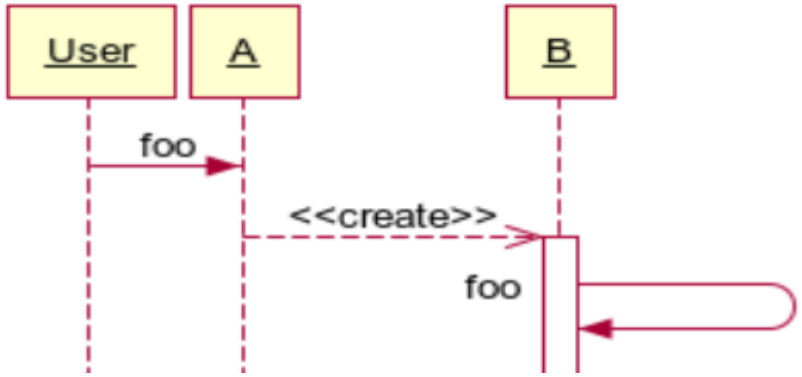


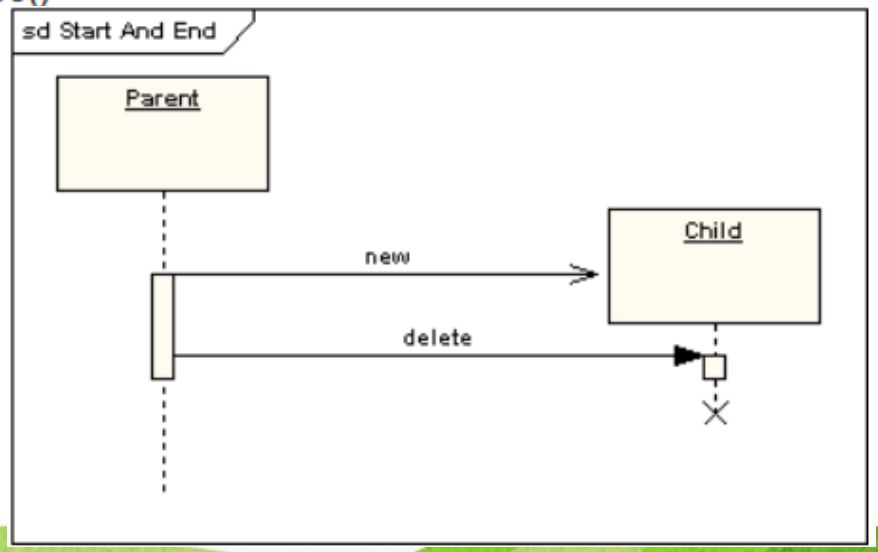


9.2.1生命线

- 创建一个对象指的是发送者发送一个实例化消息后实例化对象的结果。
- 创建对象操作的执行使用消息的箭头表示，箭头指向被创建对象的框。(也有的指向对象生命线，带<<create>>构造型)

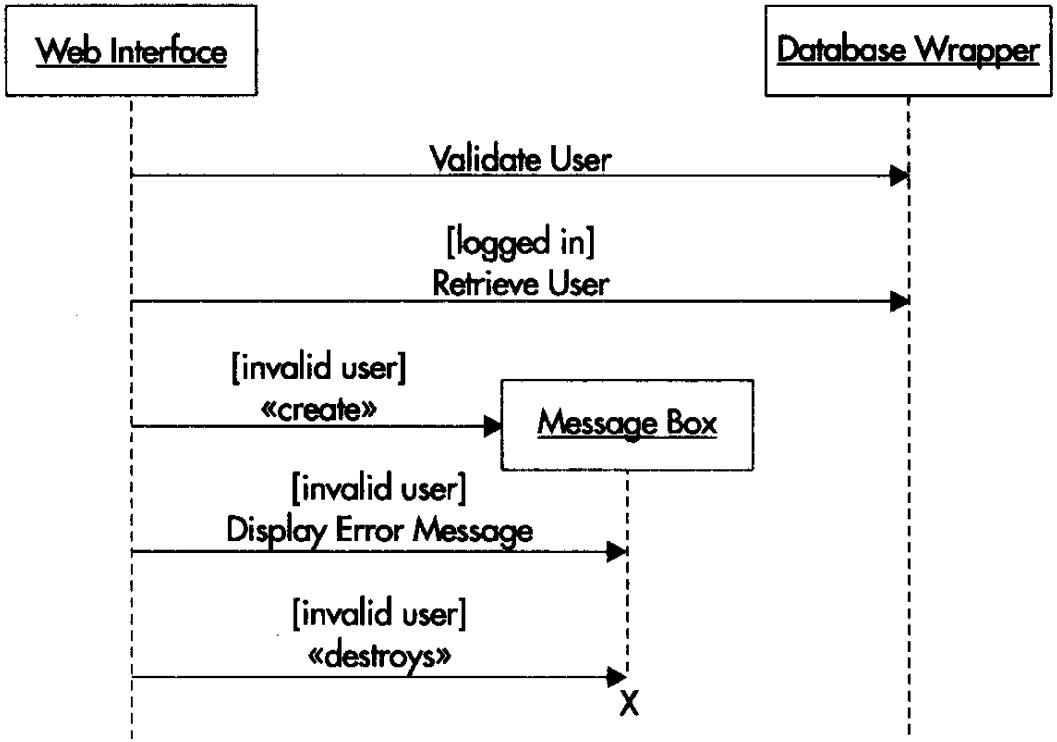


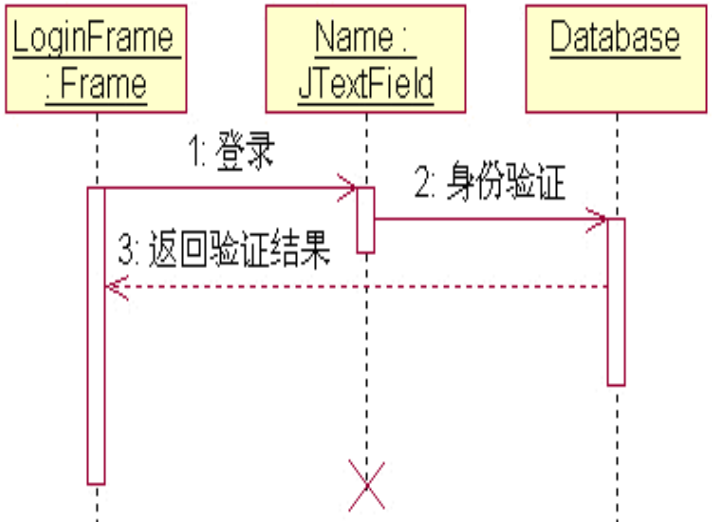




9.2.1生命线

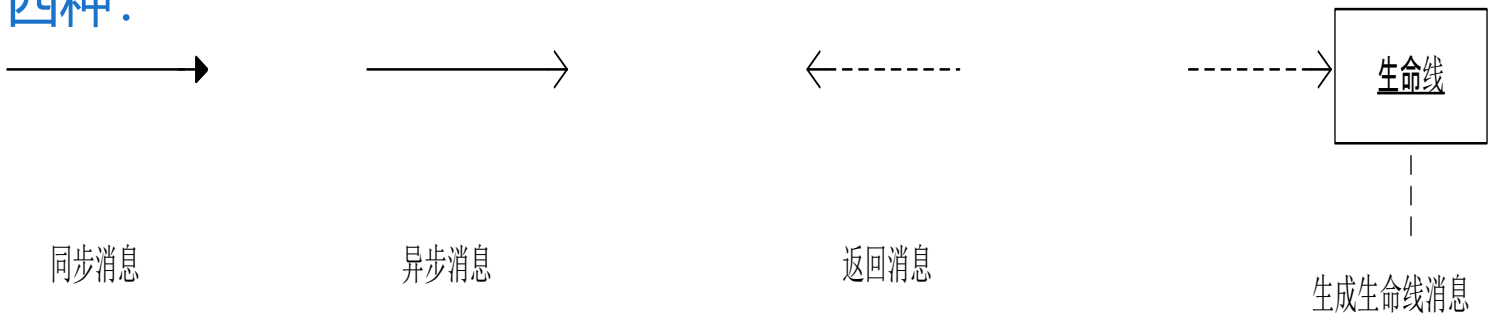
- 销毁对象指的是将对象销毁并回收其拥有的资源，它通常是一个明确的动作，也可以是其它动作、约束或垃圾回收机制的结果。
- 对象被销毁是使用在对象的生命线上画大×表示，在销毁新创建的对象，或者序列图中的任何其他对象时，都可以使用。它的位置是在导致对象被销毁的信息上，或者在对象自我终结的地方。





9.2.2消息

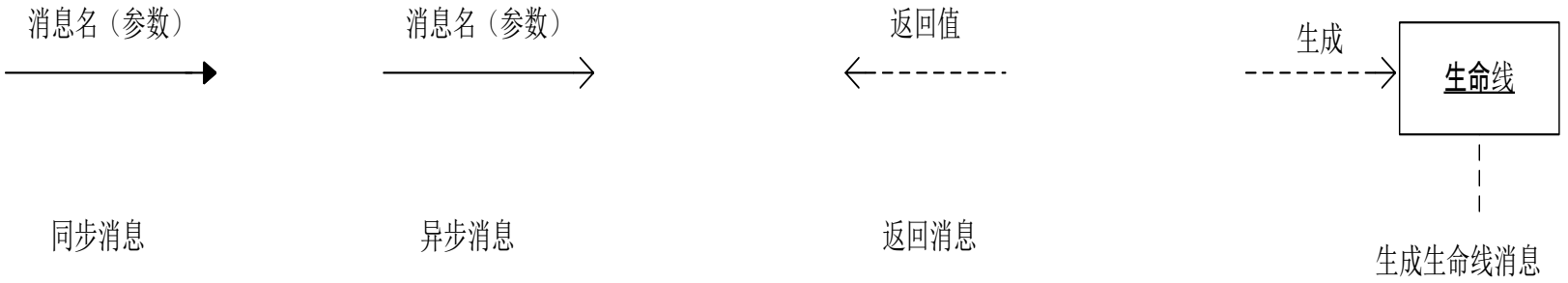
- 表示生命线之间的通信。
- 在生命线之间通过带箭头的线来连接，箭头所指的生命线为接收消息方，接受消息方的生命线接受到该消息后激活相应的动进行响应。
- 消息有四种：



- 消息上方可以加上消息名称，参数或者返回值，格式为：

发送消息：　消息名（参数1，参数2..。。）

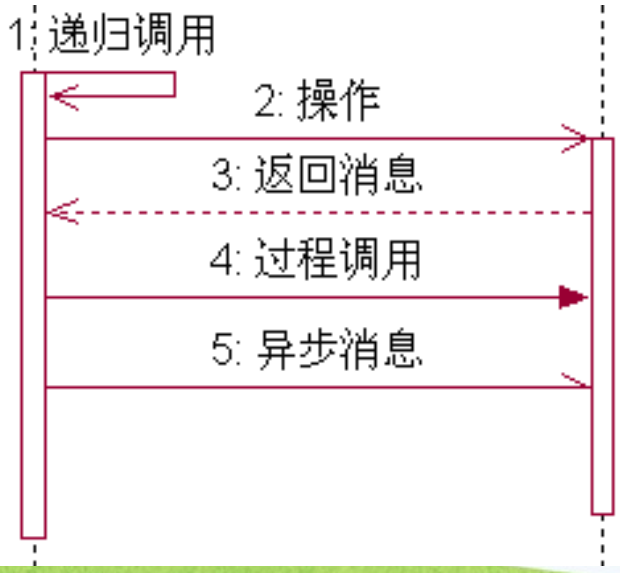
返回消息：　变量=消息名（参数1，参数2..。。）：返回值



9.2.2消息

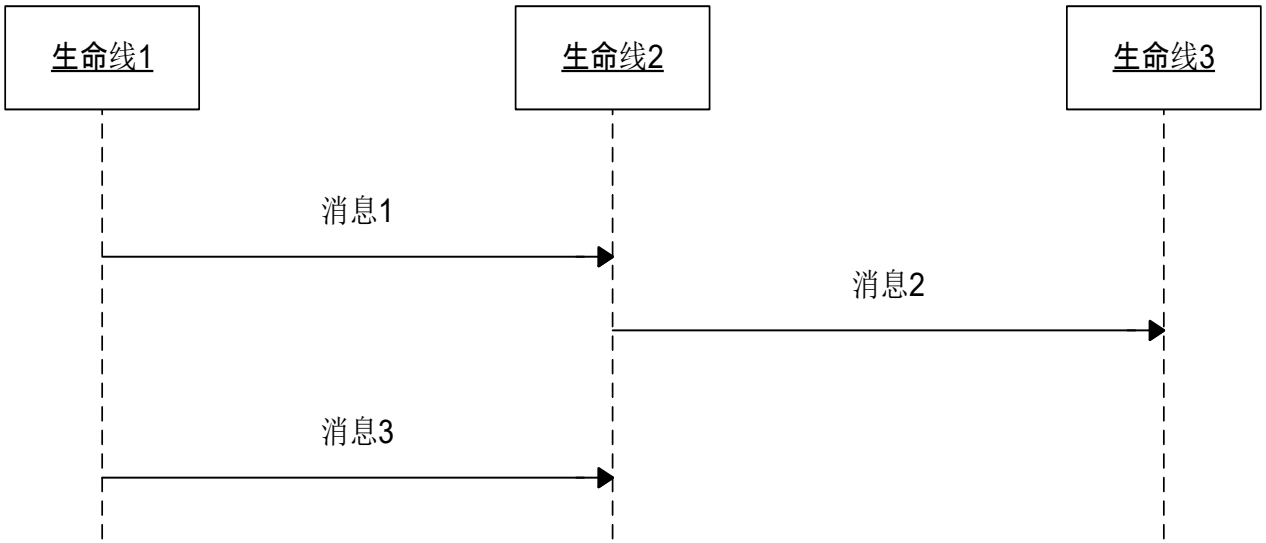
- Rose的消息符号略有不同（万一碰到了认识即可）。

消息种类	UML 2	Rational Rose
同步消息(Rose的过程调用或者简单消息)		
异步消息		
返回消息		



9.2.2消息-同步消息

- 表示消息执行的同步顺序（层次化关系）。
  - 某个生命线发出的上一条同步消息处理完全结束之后，才能发送和处理下一条消息。
  - 用实线实心箭头表示。
  - 可以明确控制消息发送和处理的先后顺序。

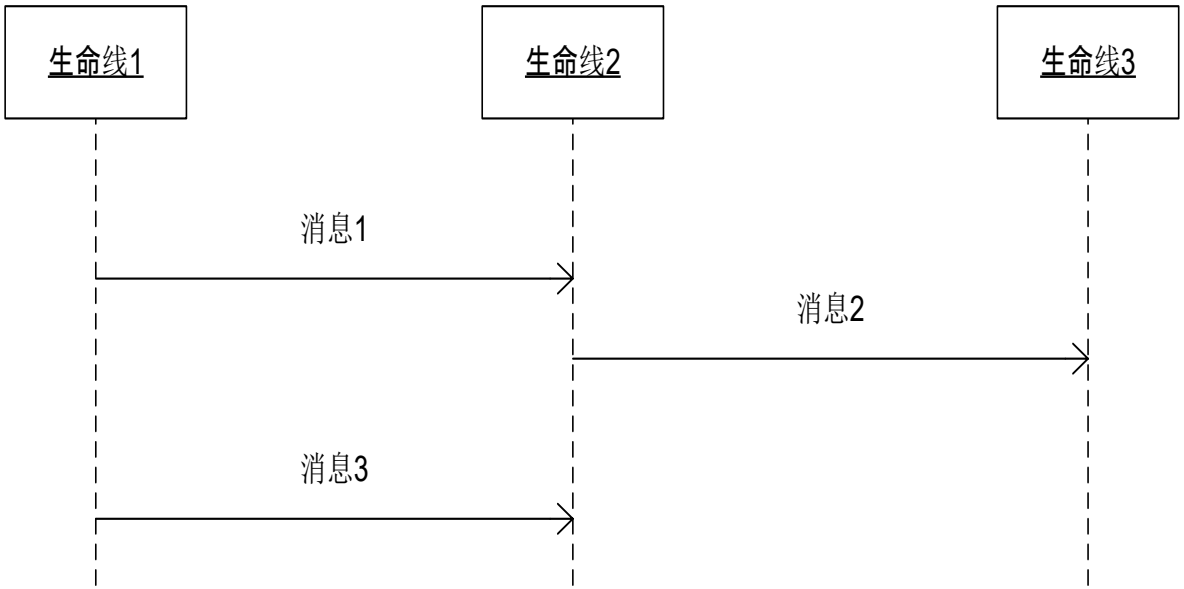


消息2的处理结束之后，消息1的处理才完全结束。

等消息1的处理完全结束后，生命线1才能发送消息3.

9.2.2消息-异步消息

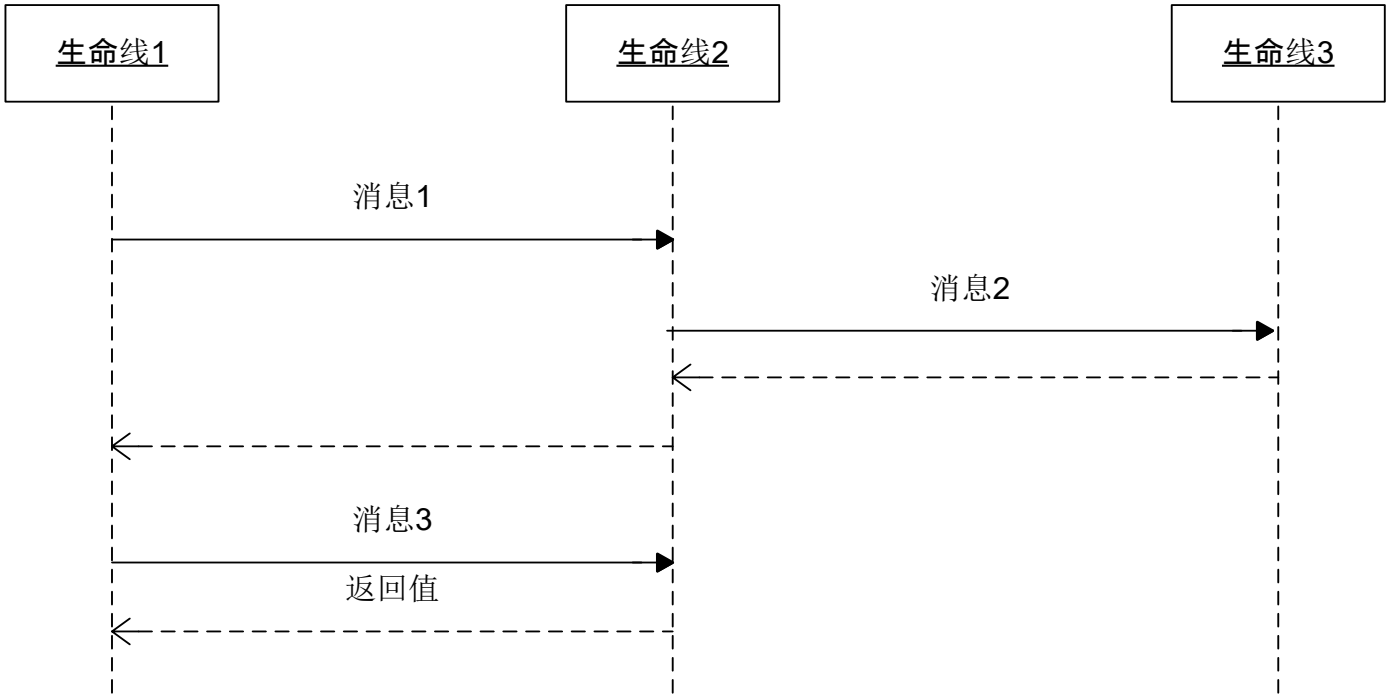
- 表示没有同步关系的消息。
  - 某个生命线不用等待发出的上一条异步消息处理结束，即可发送和处理下一条消息。
  - 用实现空心箭头表示。
  - 可以用来实现多条消息的并行发送。



只要消息1成功发出后，不用等待消息1的处理完全结束，生命线1就能发送消息3.

9.2.2消息-返回消息

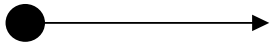
- 表示对于同步消息的应答。
  - 不强调返回关系和返回值的情况下，可省略返回消息的表示。



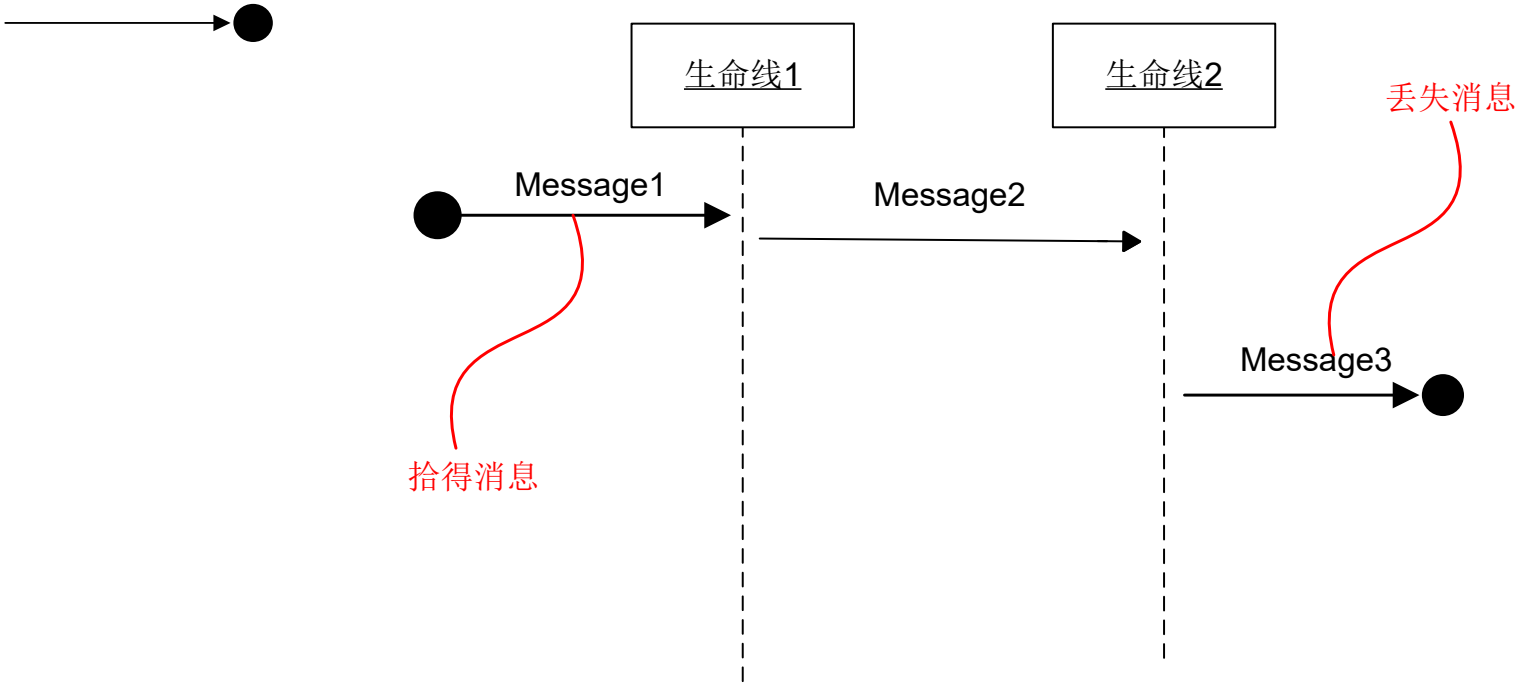
返回消息可以携带返回值，也可省略返回值。

参考：拾得消息和丢失消息

- 顺序图中的消息不一定都有明确的发送源和接收方。
  - 对于发送方不明确的消息成为拾得消息（found message）。如下表示：

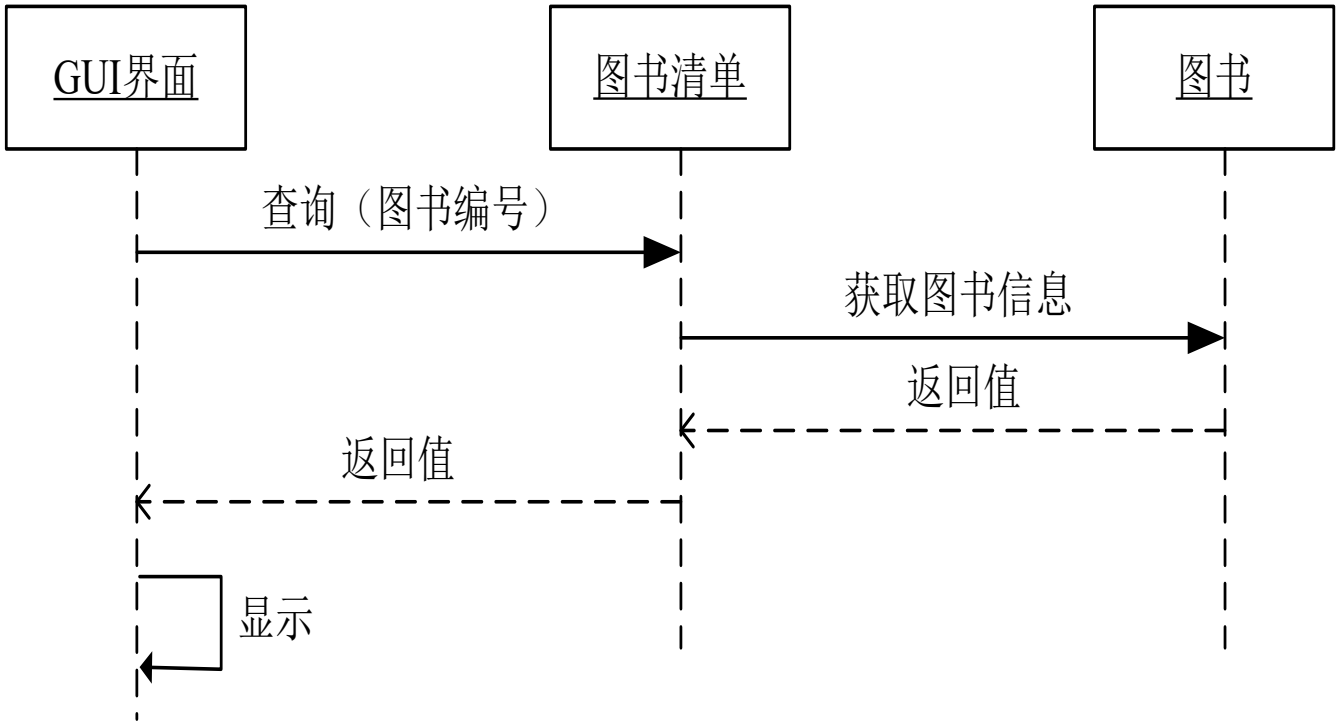


  - 对于接收方不明确的消息成为丢失消息(lost message)。如下表示：

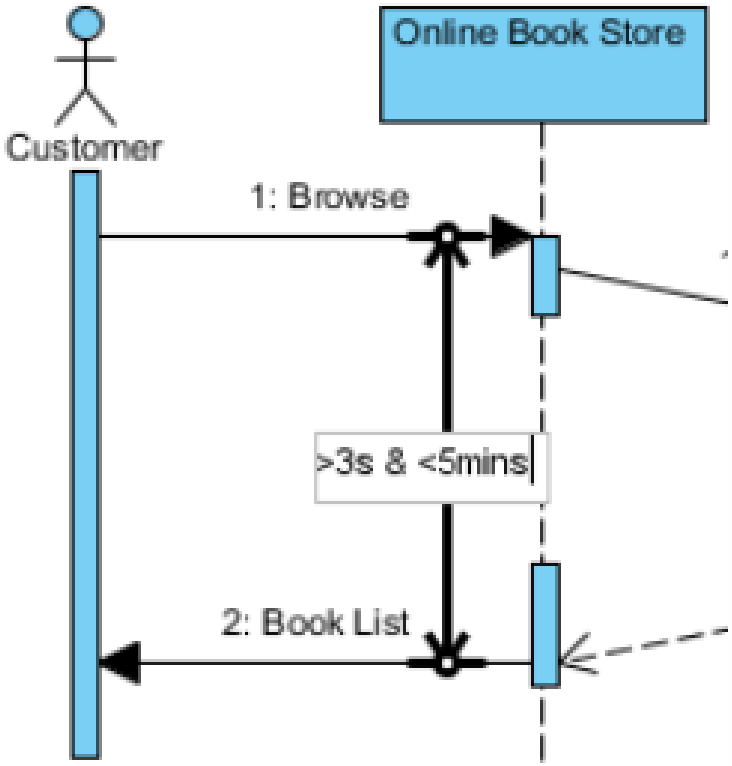


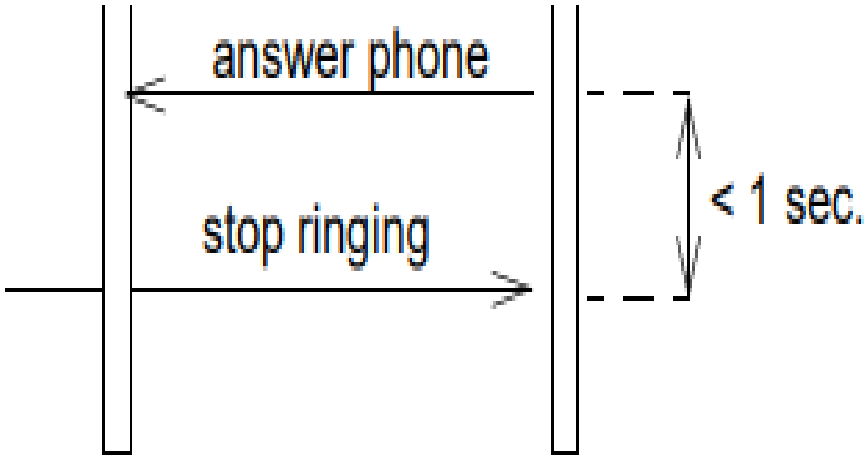
9.2.2消息-递归调用

- 表示生命线对自身发送消息调用自身操作的情况。
  - 从生命线向自身通过折线进行消息发送的表示。



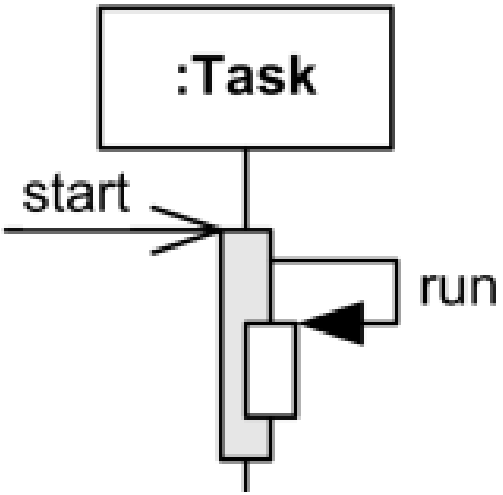
◆可以标注消息持续时间的约束。



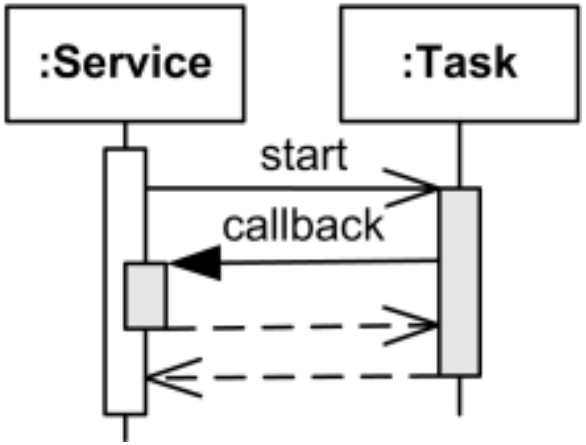


◆叠加执行：在一次交互过程中调用同一个对象的同一个或者不同方法, 在激活上叠加小矩形。

- 自关联消息（Self-Message）:表示方法的自身调用以及一个对象内的一个方法调用另外一个方法。

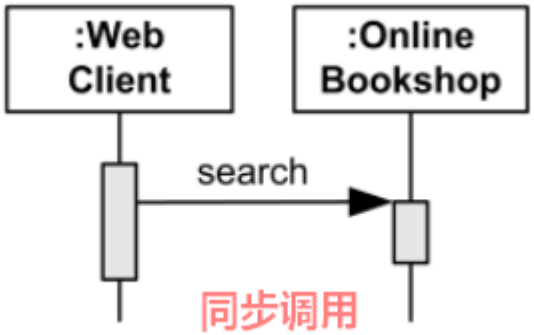


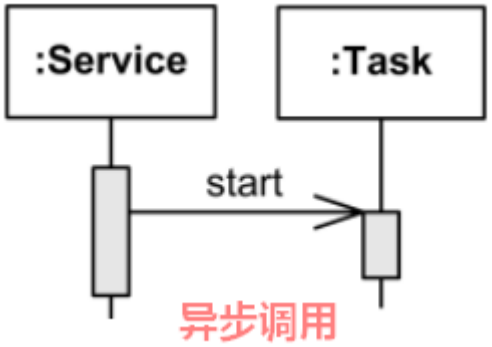
Overlapping execution specifications on the same lifeline - message to self.



Overlapping execution specifications on the same lifeline - callback message.

消息的表示： 创建消息和返回消息有点像











4. 消息的表示

◆消息的语法格式:

◆**[前置消息列表][警戒条件][消息顺序号][返回值:=]消息名称([参数列表])**

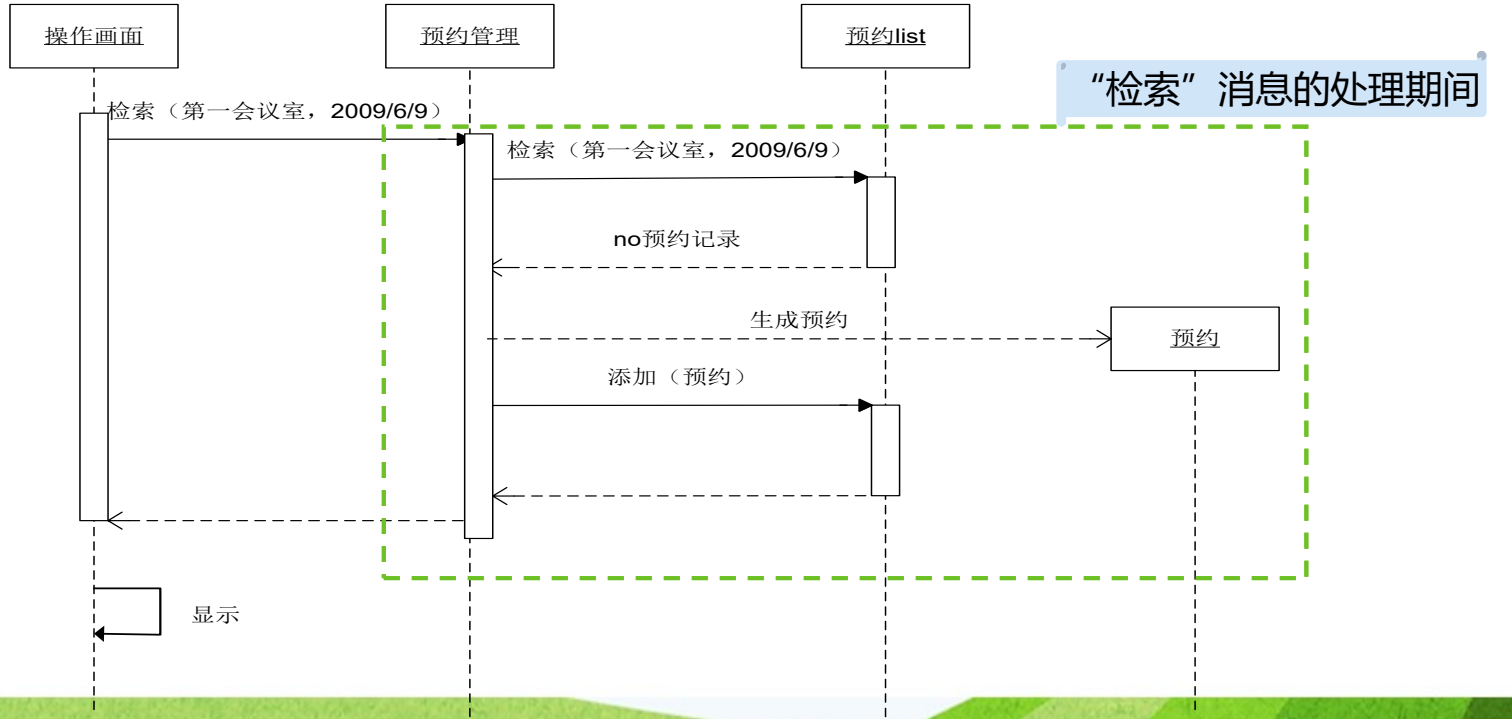
◆例子

2:display(x,y)	简单消息
1.3.1 p:= find(specs)	嵌套消息, 有返回值
[x<0] 4: invert(x, color)	条件消息
3.1 *: update()	循环消息

https://blog.csdn.net/weixin_43387245

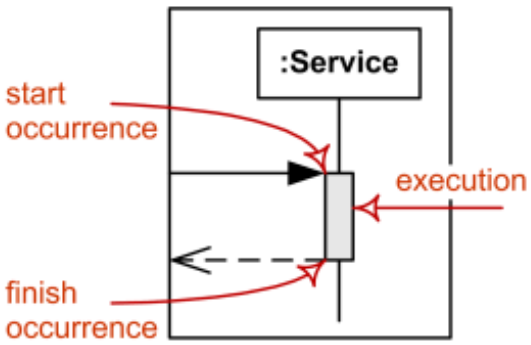
9.2.2消息-激活(Activation)

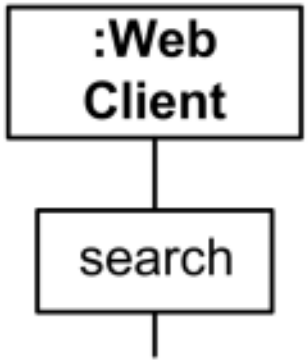
- 激活框包含以下两方面的信息：
  - 生命线的某个处理被执行的期间。
  - 生命线的某个处理的执行和调用方之间的控制关系。
- 激活框的表示方法：
  - 在生命线上用细长的矩形框来表示。
  - 矩形框最上端表示处理的开始时点，矩形框的最下端表示处理的结束时点。
- 在顺序图中，激活框为可选项，可省略。
  - 需要明确表示消息和被调用的处理之间的控制关系，以及消息处理的先后顺序时，可使用激活框。



9.2.2消息-激活(Activation)

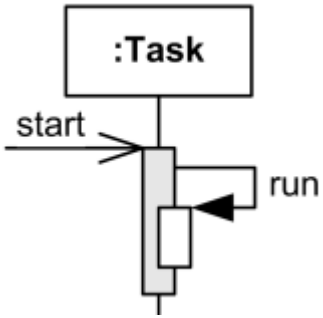
- 激活在UML规范中正式的名称是“执行规范(execution specification)” 。Rose中称为focus of control。 代表参与者生命周期中的一段交互片段，这期间它正在：
  - 在生命线中执行行为或动作的一个单元，
  - 向另一参与者发送信号，
  - 等待来自另一参与者的回复消息。
- 激活在生命线上表示为一个细长的灰色或白色矩形。
- 注意：激活包括参与者等待回复的期间。UML规范把激活进一步划分为start occurrence和finish occurence。
- 也可以用一个较宽矩形，内部写上活动名字。

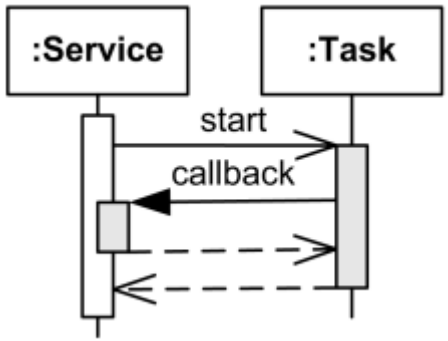




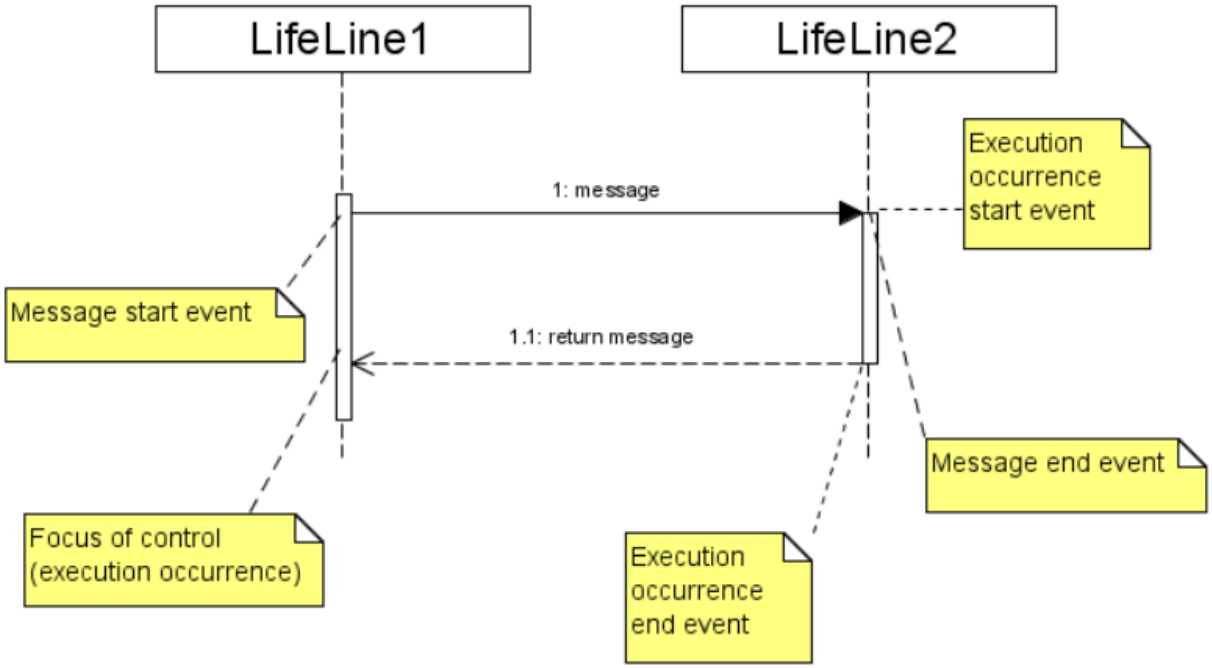
9.2.2消息-激活(Activation)

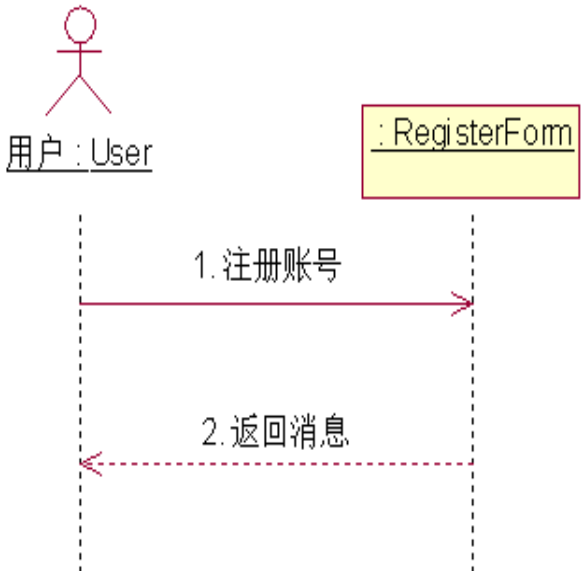
◆重叠的激活用重叠的矩形表示。





◆激活不是必须的。可以设置激活是否显示（右图激活没有显示）





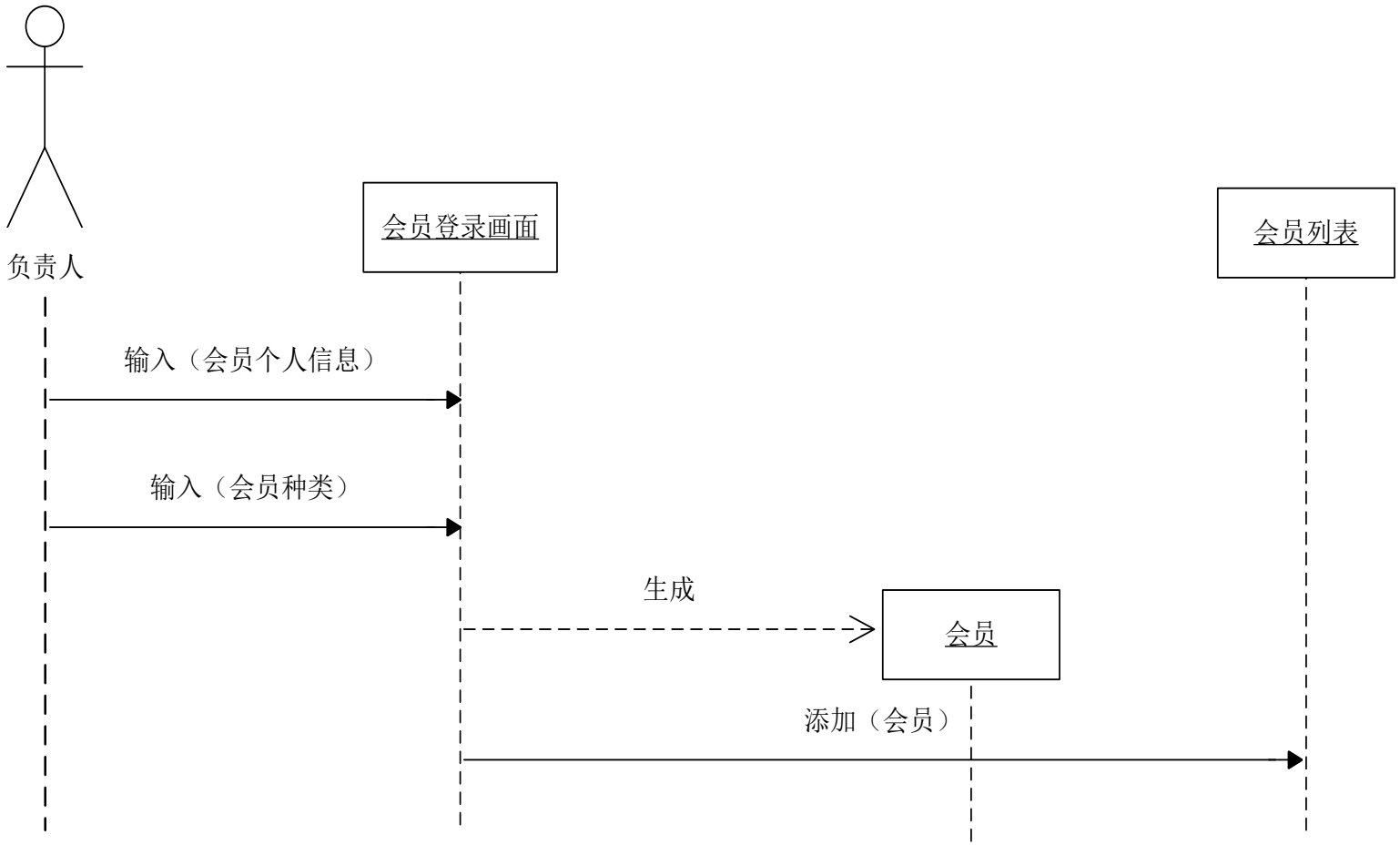
例题：画顺序图

- 想要描述的脚本如下：

某电子商务系统，负责人在有人申请入会时，首先在会员登录画面上输入入会申请人的个人信息，然后再输入会员申请的会员种类。

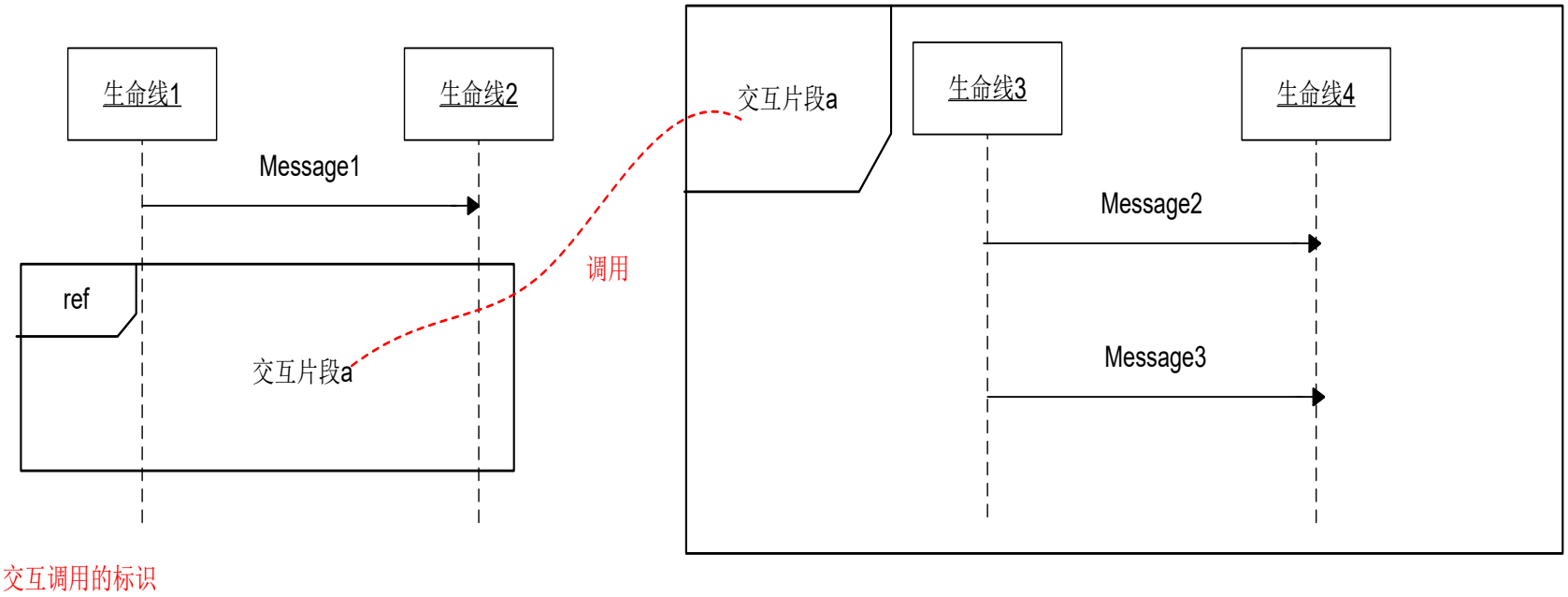
系统根据输入的信息新生成一会员，添加到会员列表中。

题解：



9.2.2消息-交互调用

- 在一个交互的执行过程中去参照调用另外一个在它处定义好了的交互序列称为交互调用(Interaction use)。
  - 在框架的head部分用关键字“ref”进行标识，框架中央指明要调用的交互序列名称。
  - 方便合理组织顺序图中的元素，简化图中的交互调用关系，增强复杂交互关系的可读性和可维护性。



交互调用的标识

9.2.2消息-组合片段(combined fragment)

西北工业大学 NORTHWESTERN POLYTECHNICAL UNIVERSITY

- 对复杂的交互图进行有意义的片段划分。
  - 方便表现具有循环，或者条件分支等交互序列。
- 常用的组合片段有：

片段	含义
loop	示循环执行某个交互序列。
alt (alternatives)	根据条件选择不同交互处理序列，各个分支的条件必须具有互斥关系。
opt (option)	表示只有在给定的条件满足的时候才能执行的交互序列，不能带有条件分支。
ctitical (critical region)	表示不能接受从其它处理来的中断的交互序列。
break	表示只有在给定条件满足的时候才能中断的交互序列。
par(parallel)	表示有并行处理的交互序列。

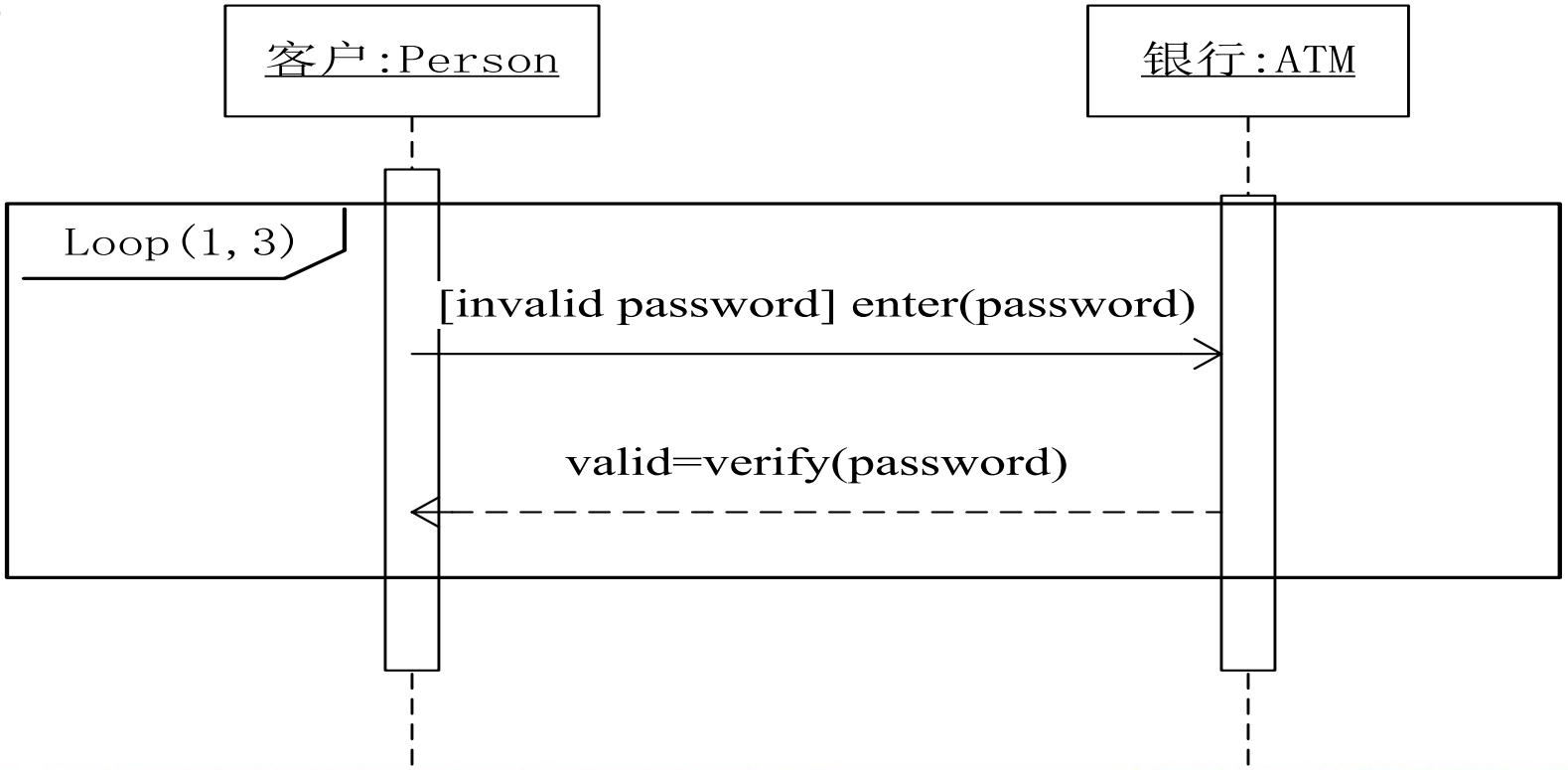
loop组合片段

Loop表示循环。语句格式如下：

- Loop(1，n)：Loop 组合片段具有“Min”和“Max”属性，它们指示片段可以重复的最小和最大次数。
- Loop(n)：表示执行n次。
- Loop(条件表达式）表示条件满足时循环
- Loop: 没有限制循环次数。具体循环次数可由片段的监护条件决定。

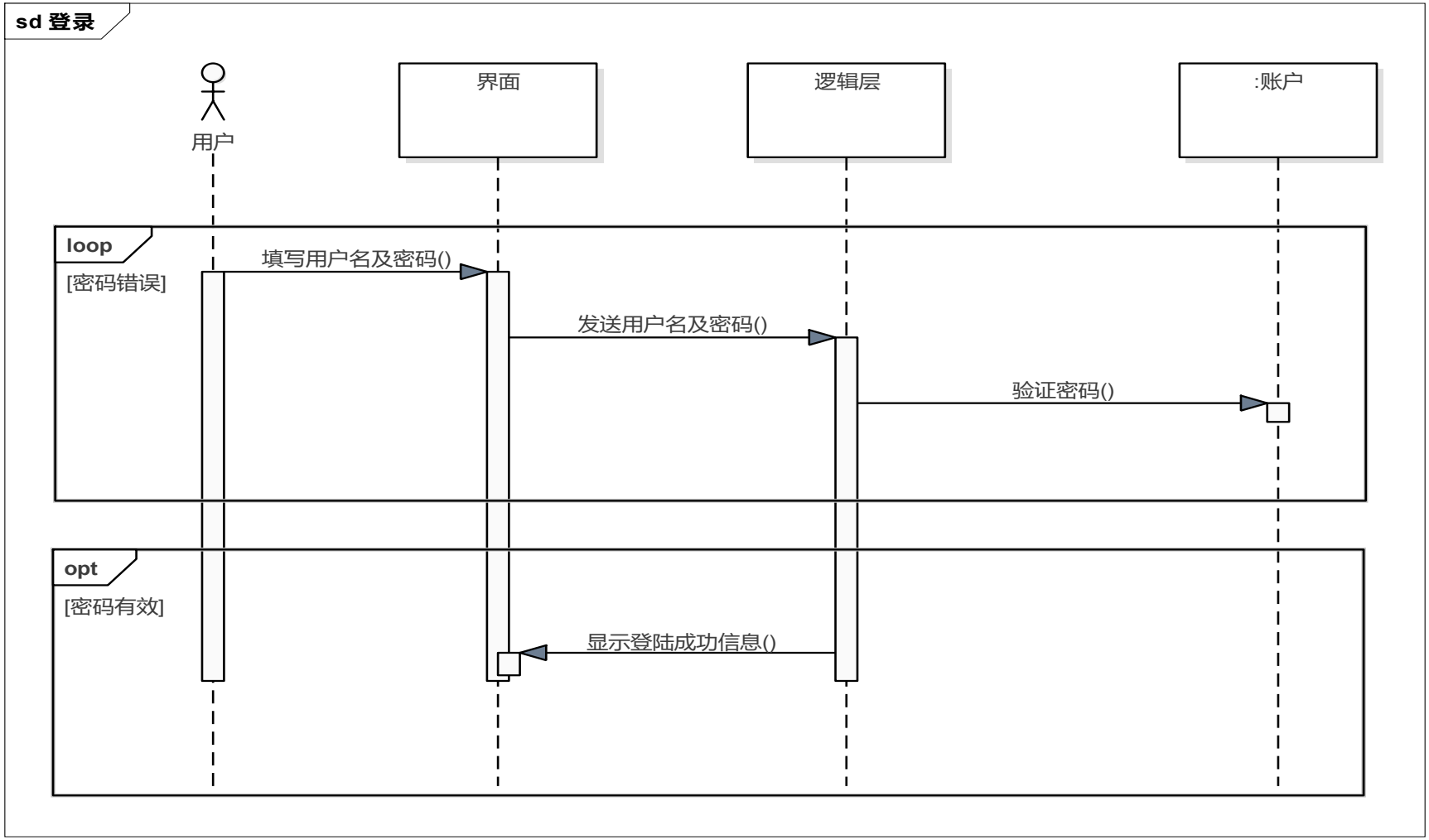
loop组合片段

- 当执行流进入组合区后，对监护条件[invalid password]进行测试，若密码无效，则执行语句enter(password)，然后，进入下一个循环。
- 但循环的次数不能超过3次，即你输入密码不能超过三次。loop(1,3)表示矩形框中的循环次数不能超过3次。



loop组合片段

■循环输入密码直到密码正确。密码正确后显示登录成功。

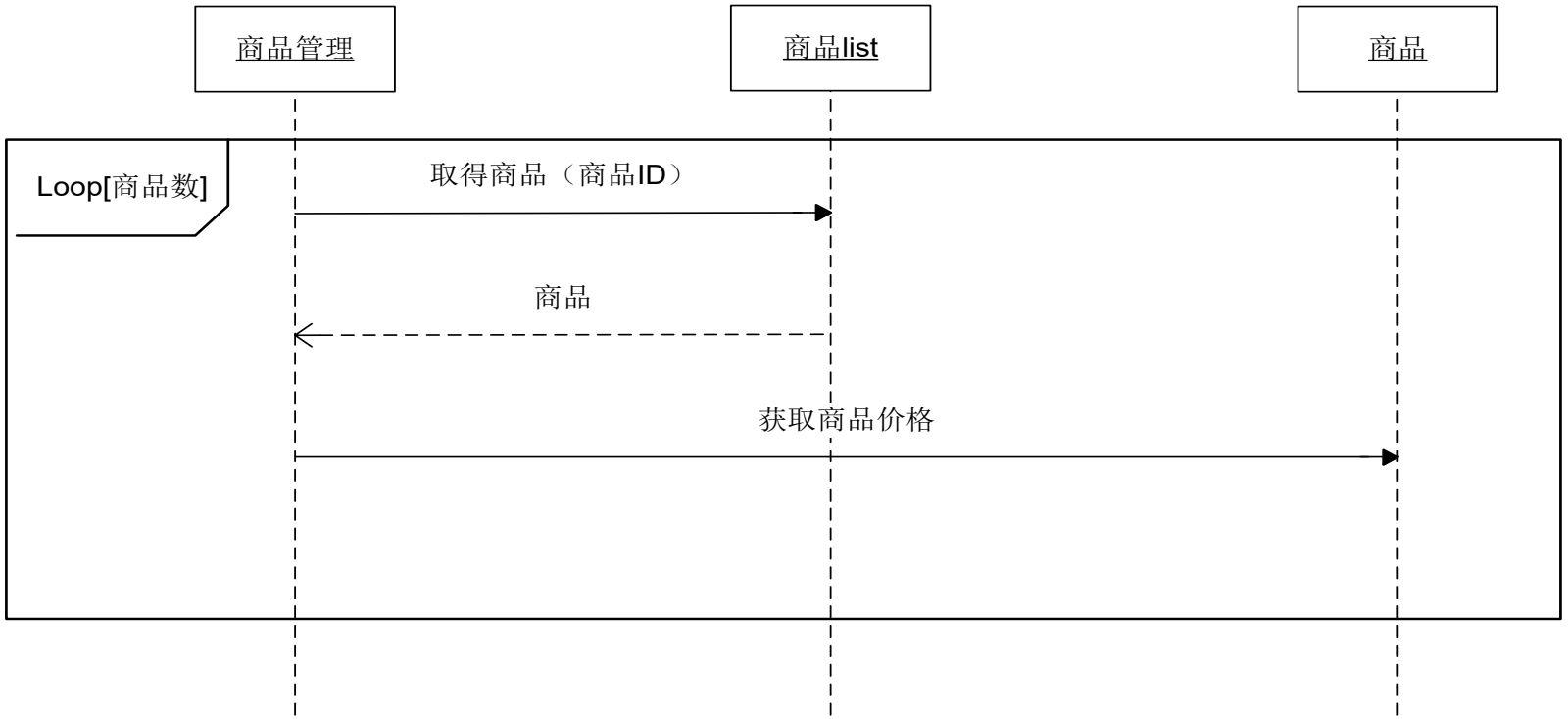


loop组合片段的例子

- 在组合片段框的head部用如下格式标识：

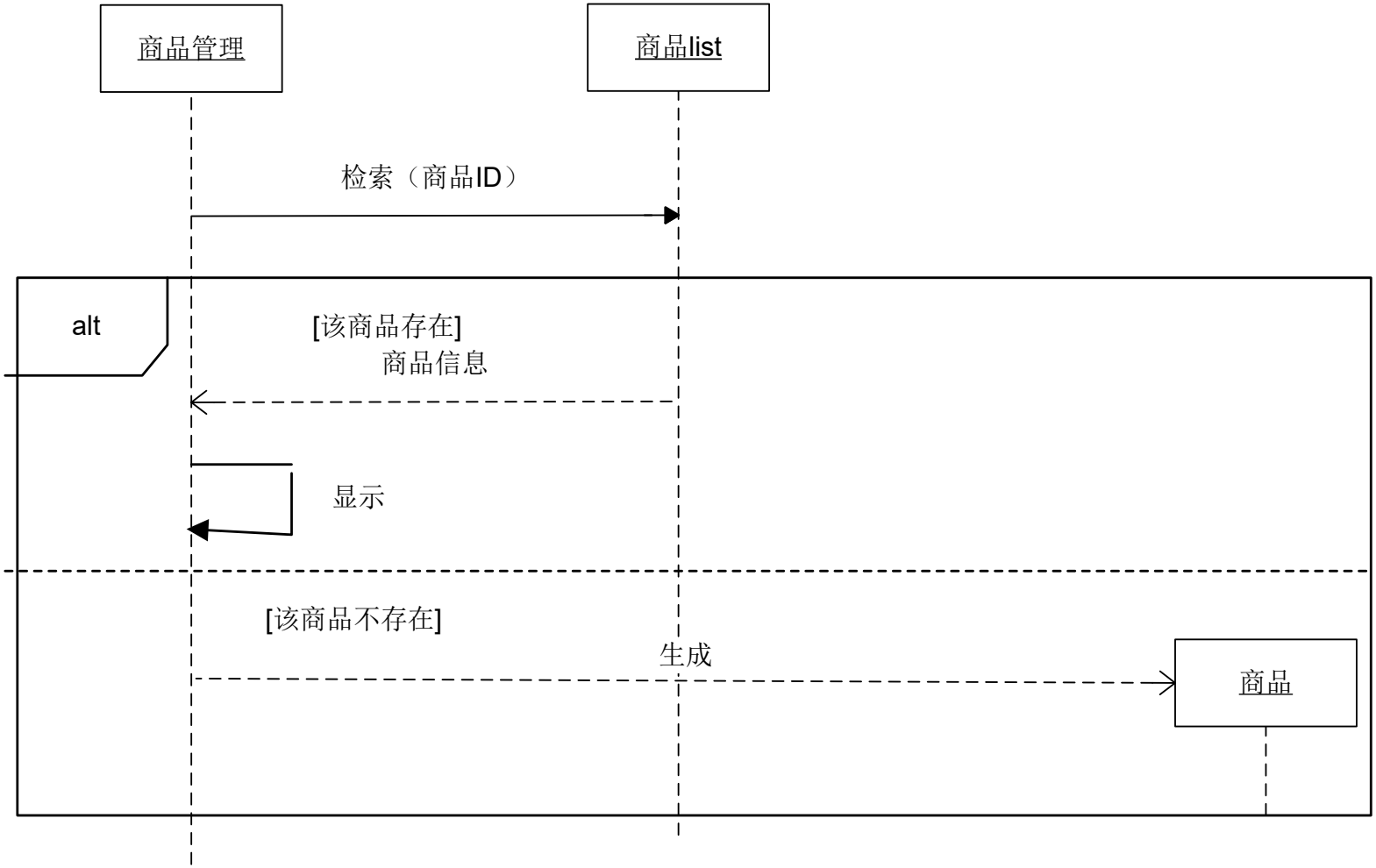
  loop【循环次数】

  - 循环次数表达式分为两类
    - 循环次数表达式　LOOP【m】，LOOP【m,n】　　m,n均为常数
    - 循环条件表达式　LOOP【条件表达式】



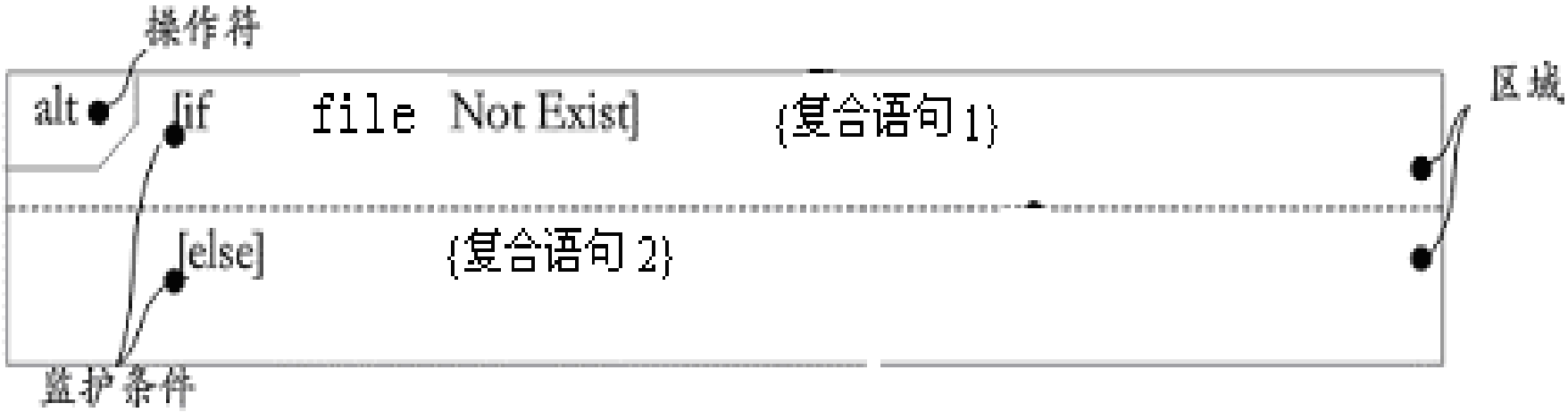
alt组合片段的例子

- 在组合片段框的head部用关键字“alt”来进行标识：
  - 组合片段框中央用虚线进行划分，【】中表明处理分支条件。



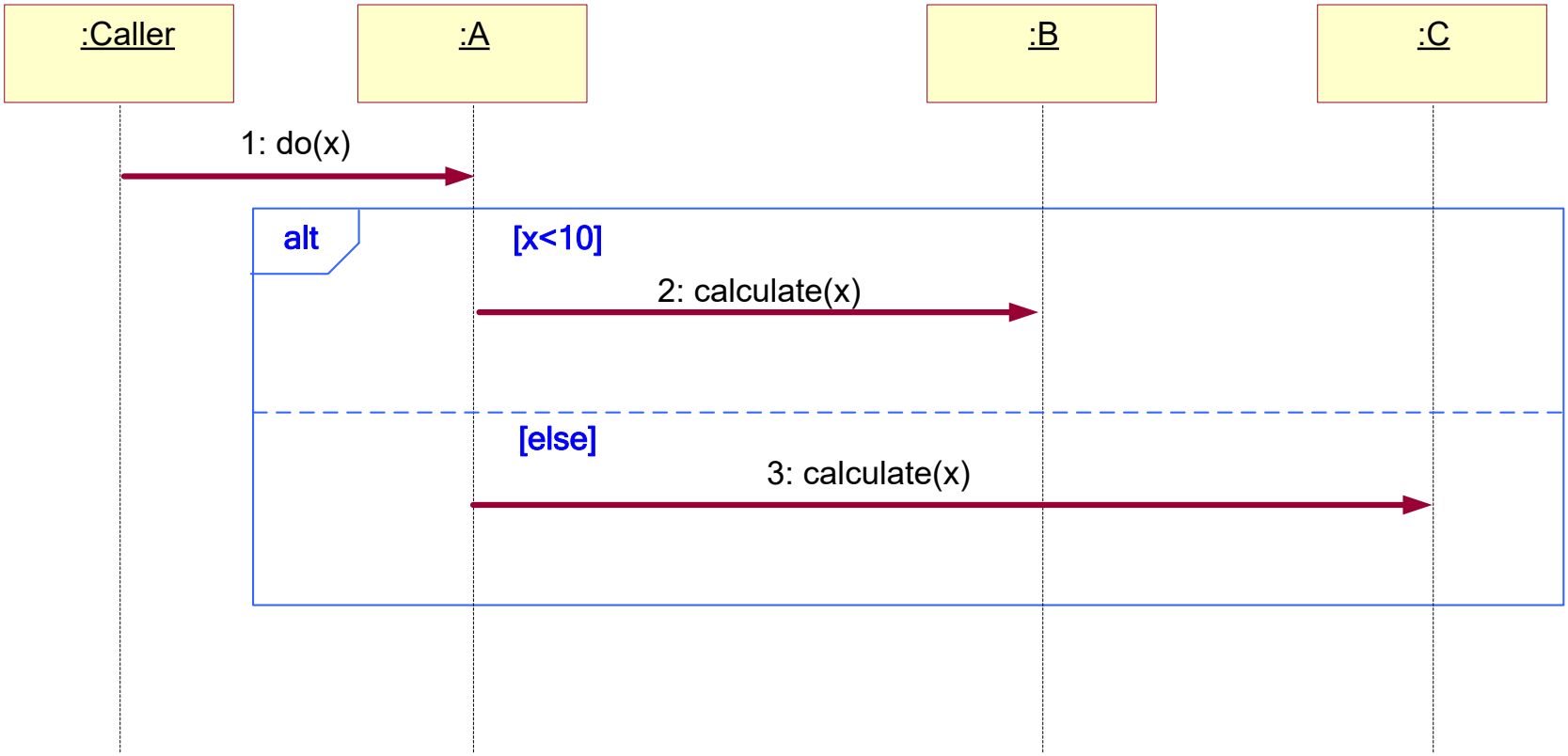
alt组合片段的例子

- 下图组合区包含2个区域，组合区的操作符是alt。第一个区域的监护条件是[if file Not Exist]，第二个区域的监护条件是[else]。
- 整个组合区语义是：如果file(文件)不存在，则执行复合语句1；如果file存在，则执行复合语句2。
-



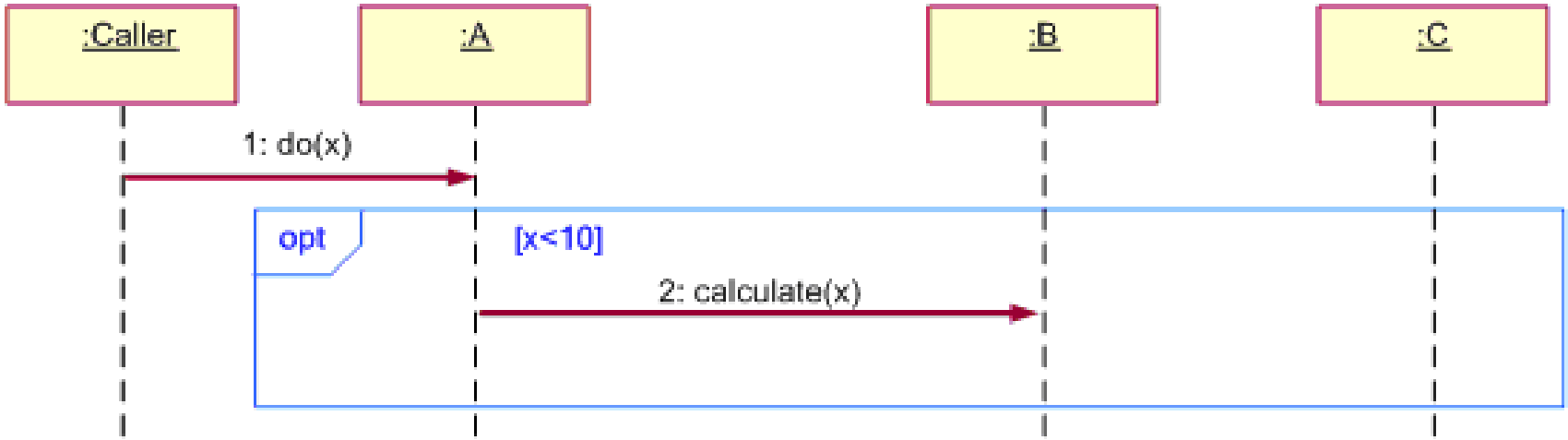
alt组合片段的例子

◆alt表示多选一，它包含2个区域。上面的区域的监护条件是：[x<10],执行语句是calculate(x);下面的区域的监护条件是[else]，执行语句是calculate(x)。



opt组合片段

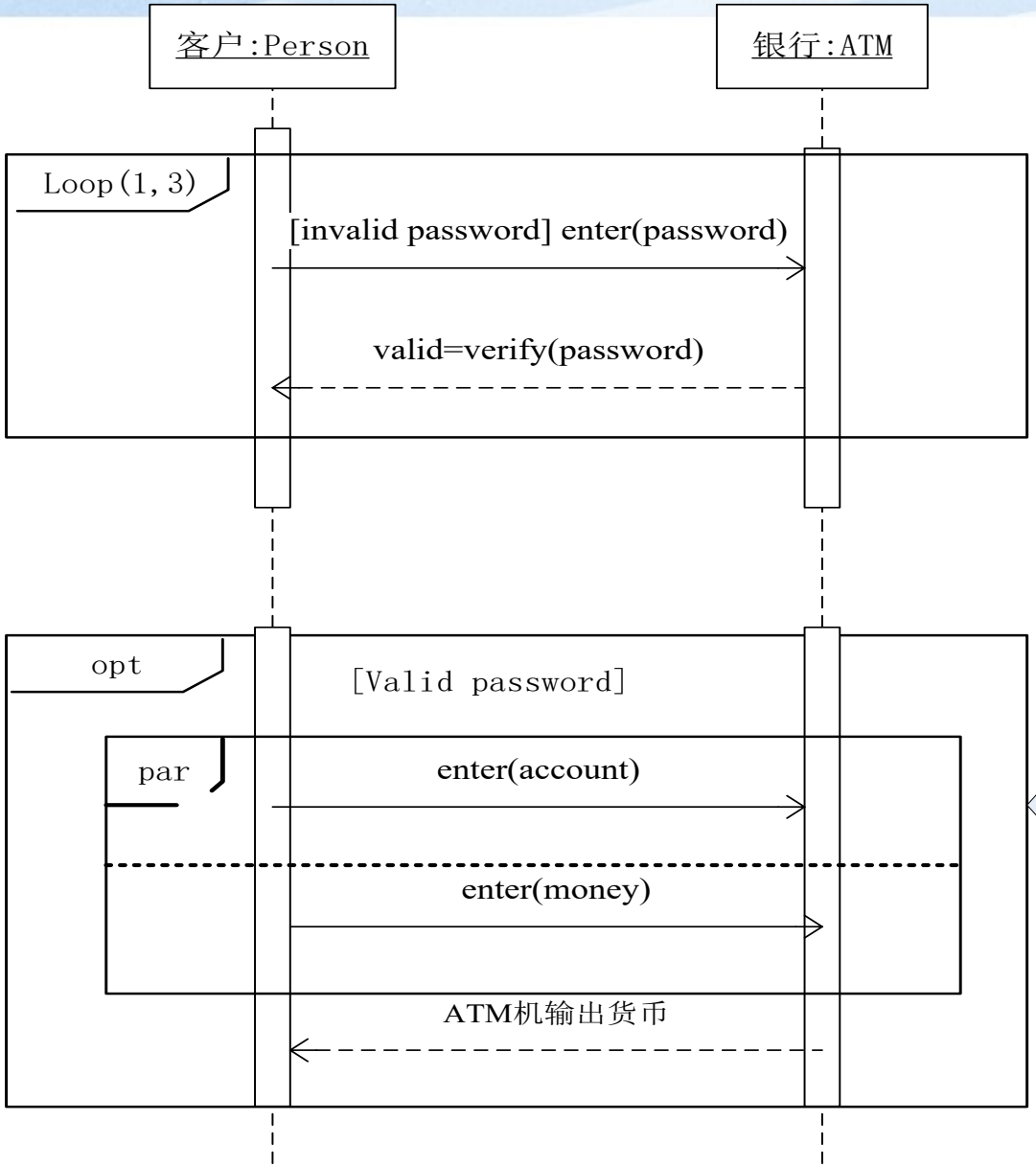
◆opt表示单选一。该组合区表示的逻辑是: “如果x<10,就要求B类对象执行calculate(x)操作。



par组合片段

- par表示并发控制。par组合区中的多个区域中的操作是并行执行的。
- 下图表示某个客户的取款操作。
- 该顺序图包括2个组合区，第一个组合区的操作符是loop（1，3），该组合区验证用户密码的有效性，当密码有效时，执行流程进入opt组合区执行。
- Opt组合区表示的逻辑是：当密码有效时执行par组合区，par组合区包含2个区域，这2个区域中的操作并发执行。
- 当执行完par组合区后，ATM机输出货币。
-

par组合片段

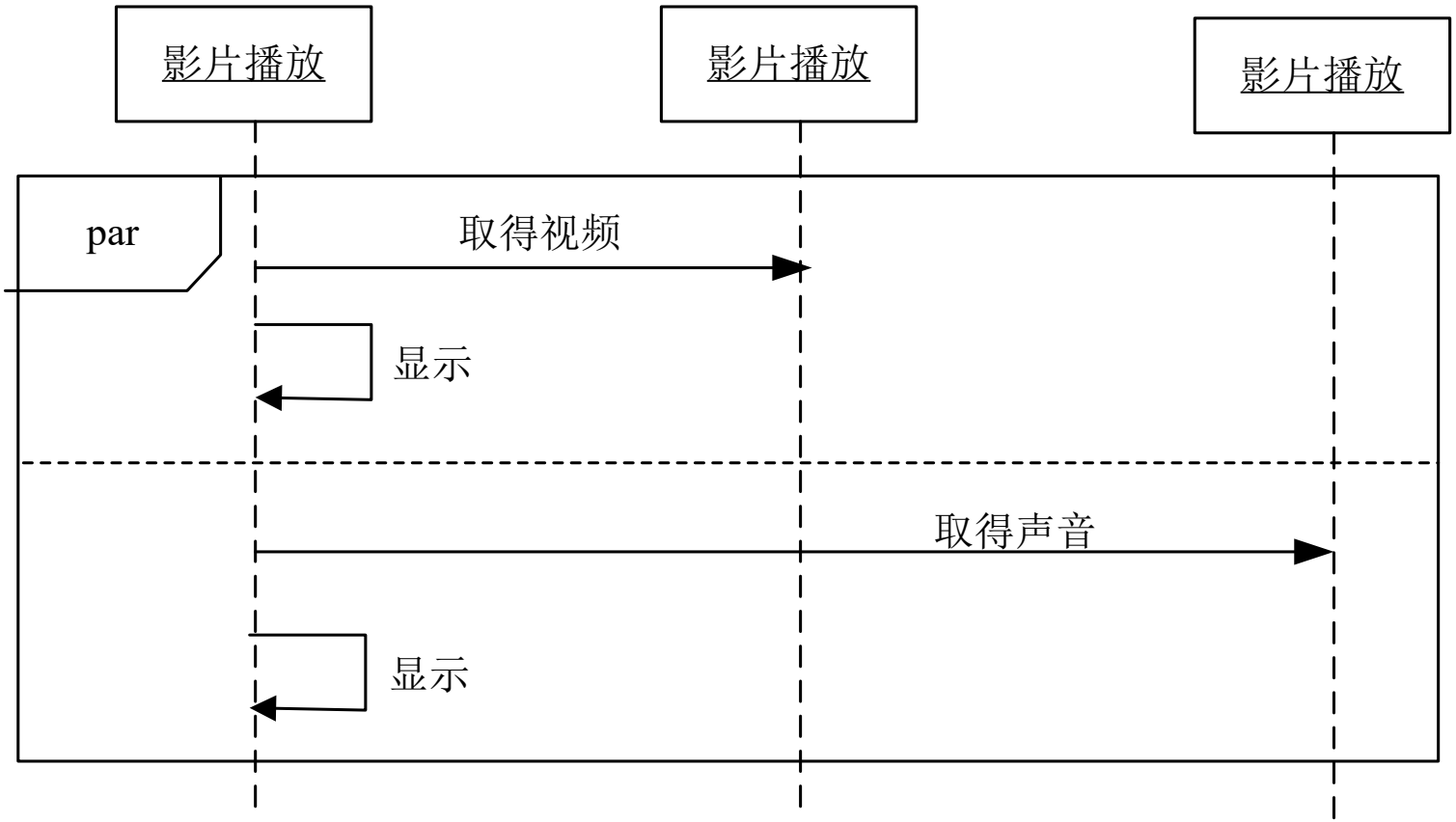


该区的操作符是loop，其逻辑语义是：如果用户输入的密码无效，最多可以输入密码3次。

opt单选操作符表示，当密码有效时执行par操作符所在的组合区，该组合区包含两个区域，一个区域要求用户输入账号（account）,另一个区域要求用户输入取款数目（money）。因为，这两个区域的操作是并行的，所以，谁先执行都可以。par组合区执行完后，ATM机就输出货币。

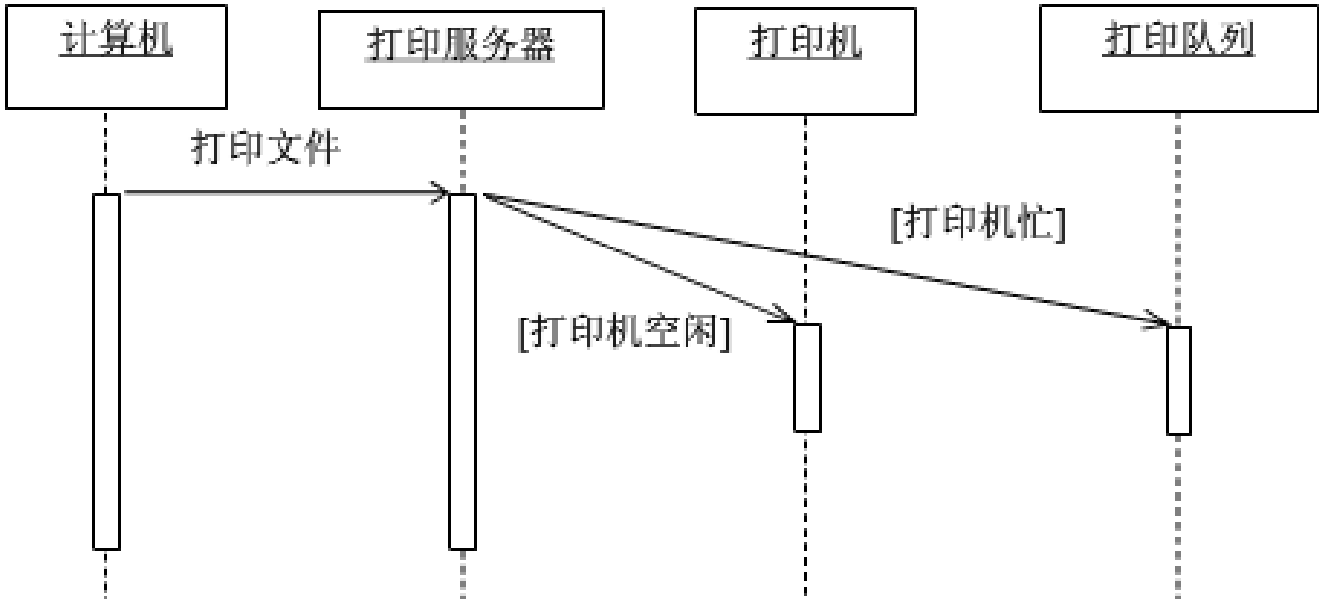
par组合片段的例子

- 在组合片段框的head部用关键字“par”来进行标识:



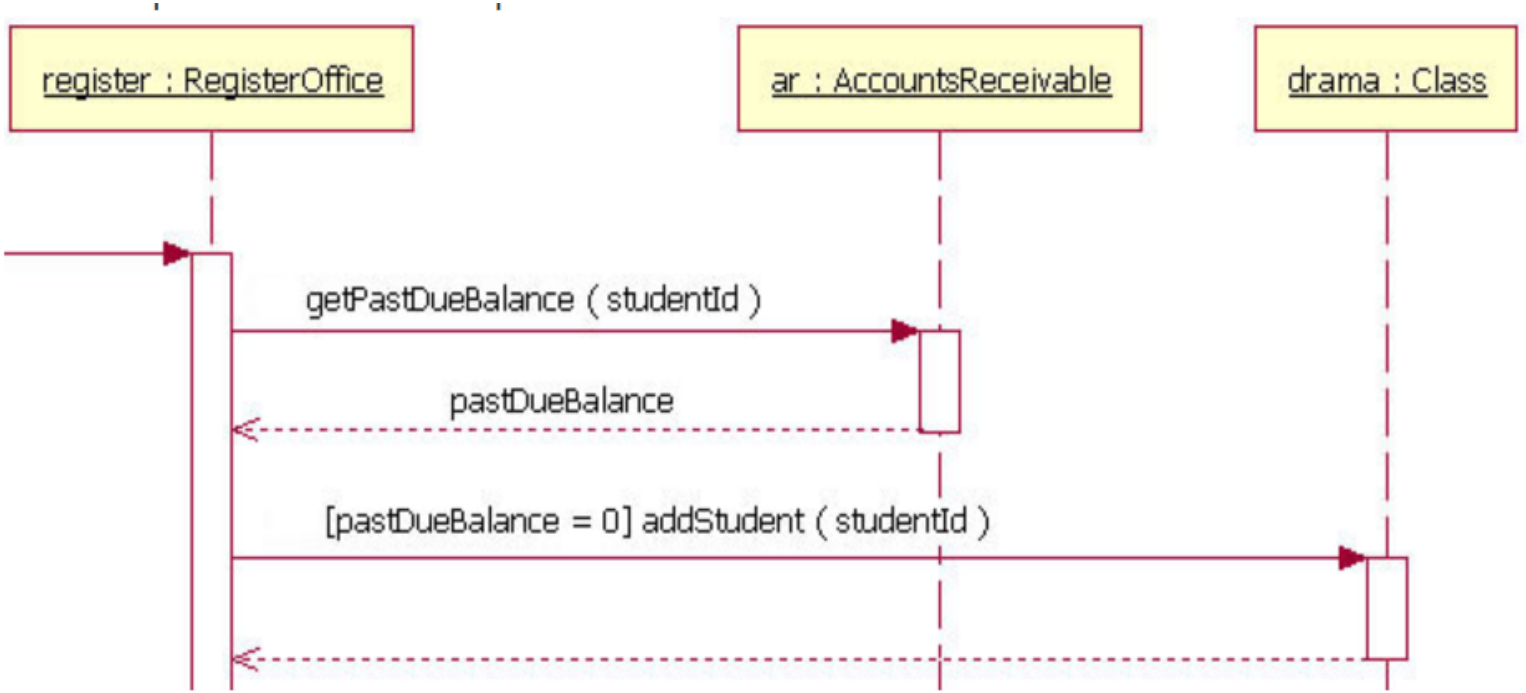
分支与从属流(UML 1.4)

- 在UML中，存在两种方式可以来修改序列图中消息的控制流，分别是：分支和从属流。
  - 分支是指的是从同一点发出多个消息并指向不同的对象，根据条件是否互斥，可以有条件和并行两种结构。



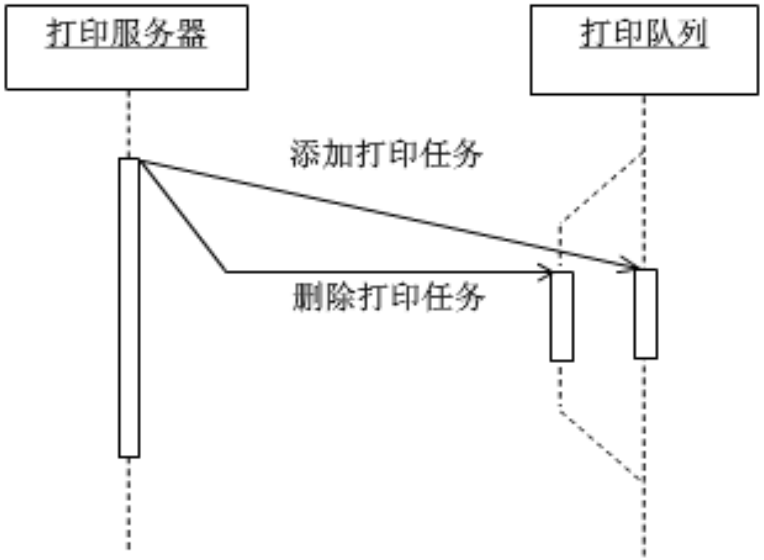
分支与从属流(UML 1.4)

- 可以利用监护条件表示消息执行的条件。比如下图的"[pastDueBalance=0]"。



2. 分支与从属流(UML 1.4)

- 从属流指的是从同一点发出多个消息指向同一个对象的不同生命线。比如下图允许根据不同的条件改变执行不同的操作，创建对象的另一个生命线分支。

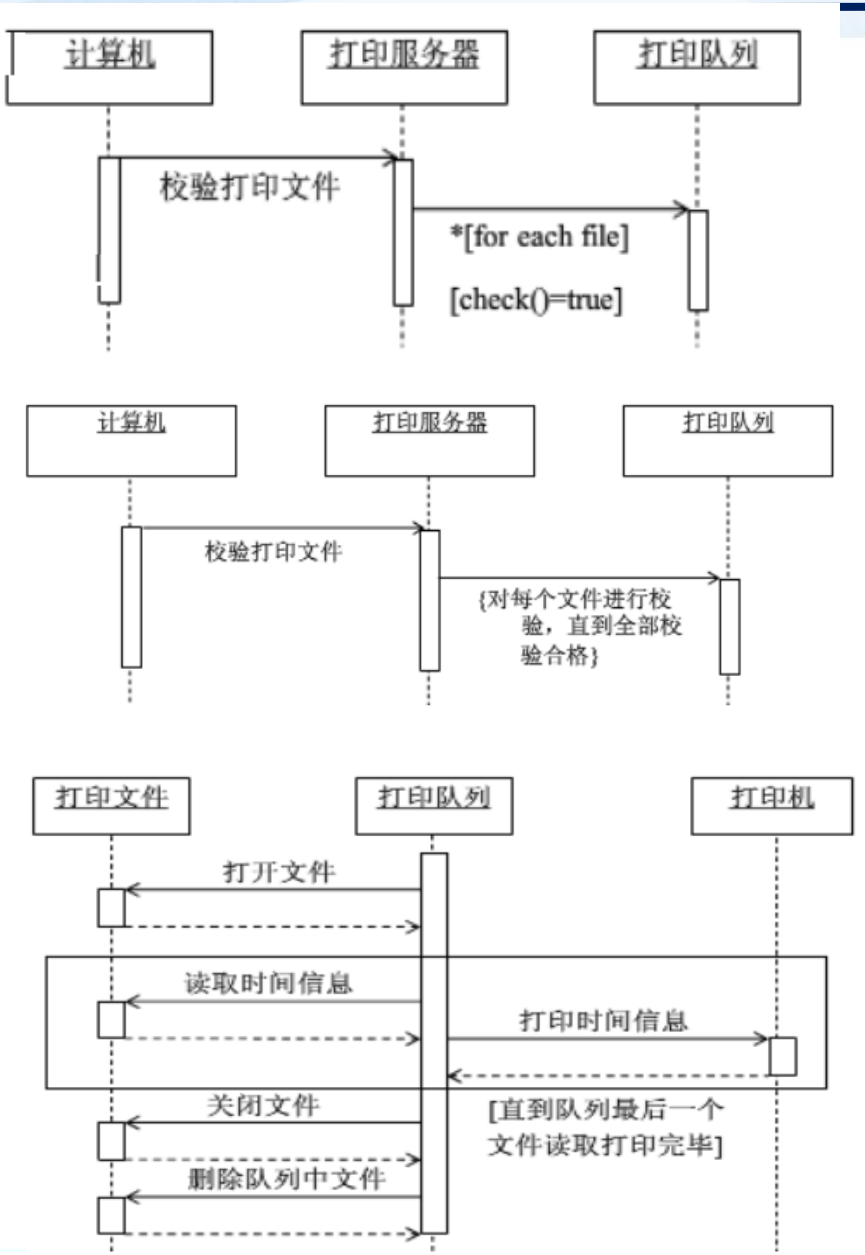


循环(UML 1.4)

■第一种：在需要循环重复执行的消息前添加符号*，并在其后的中括号中写明具体条件。

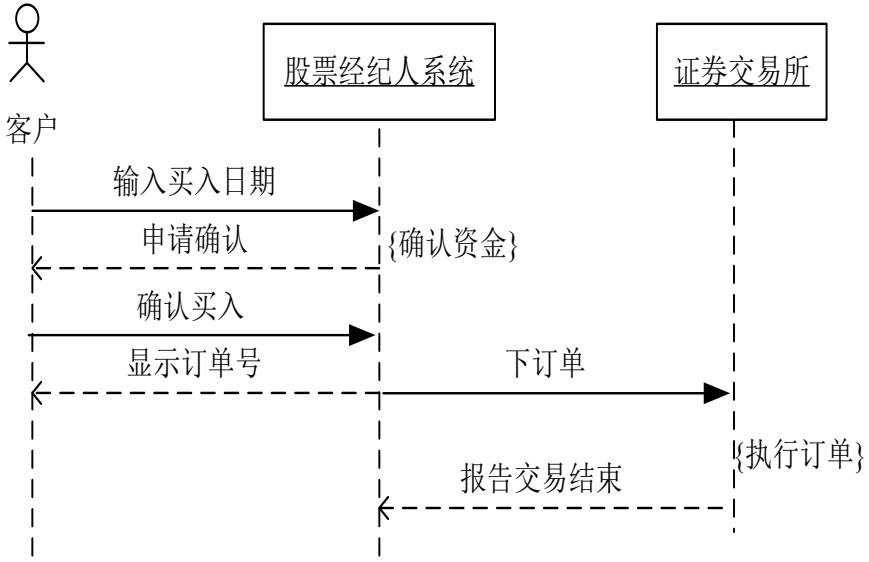
■第二种：在时间轴上注释表明消息的重复执行，使用大括号标明说明文字（约束)。

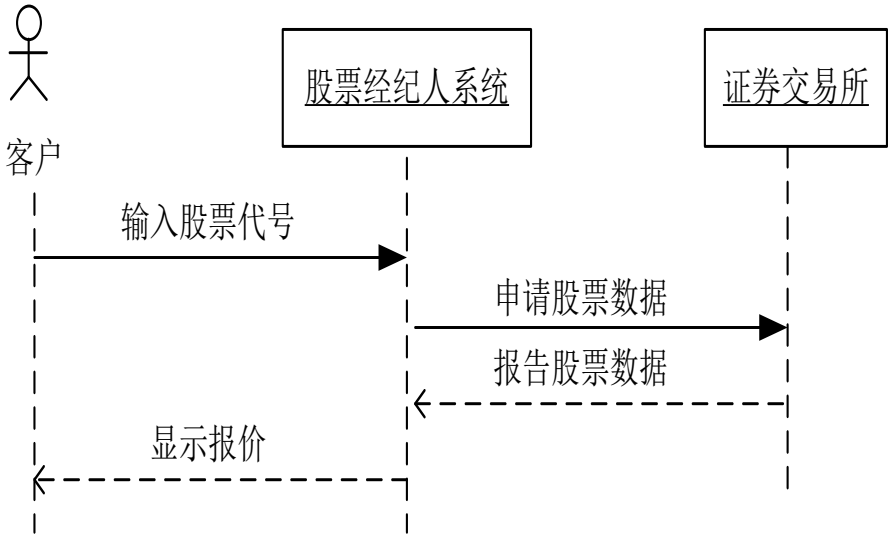
■第三种：使用矩形框将需要重复执行的消息框起来，并在中括号内说明重复执行的条件。



◆顺序模型建立准则

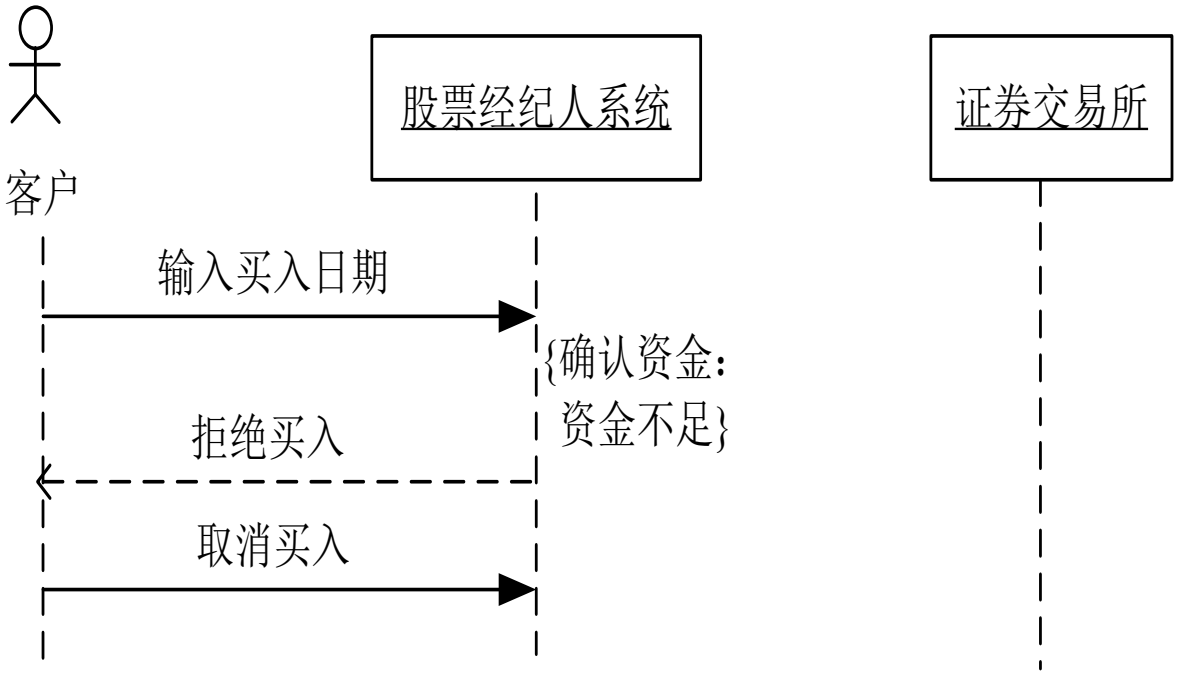
每个用例需要一张或者多张顺序图来描述其行为。每张顺序图显示用例的一个特定行为序列。顺序图可以显示大规模的交互，例如，股票经纪人系统的整个会话过程。但是，整个会话过程也可以按照不同方式组合，形成一些单独的任务。例如，针对股票经纪人系统的组合中，把买入股票的订单和对某种股票的报价请求单独罗列出来，如下2个图所示





◆顺序模型中的异常

有时候，需要为用例内部的每种异常条件绘制一张顺序图。



9.2.3 顺序模型准则

◆原则

- 至少为每个用例编写一个场景。场景中的步骤应该是逻辑命令，而不是单次的按钮点击。
- 把场景抽象为用例图。顺序图清晰地显示了每个参与者的贡献，重要的是在组织对象行为开始时，要分辨每个参与者的共献。
- 划分复杂的交互。把大型的交互划分为各个组成的任务，并为每个任务绘制一张顺序图。
- 为每种错误条件绘制一张顺序图。显示系统对于错误条件的响应。

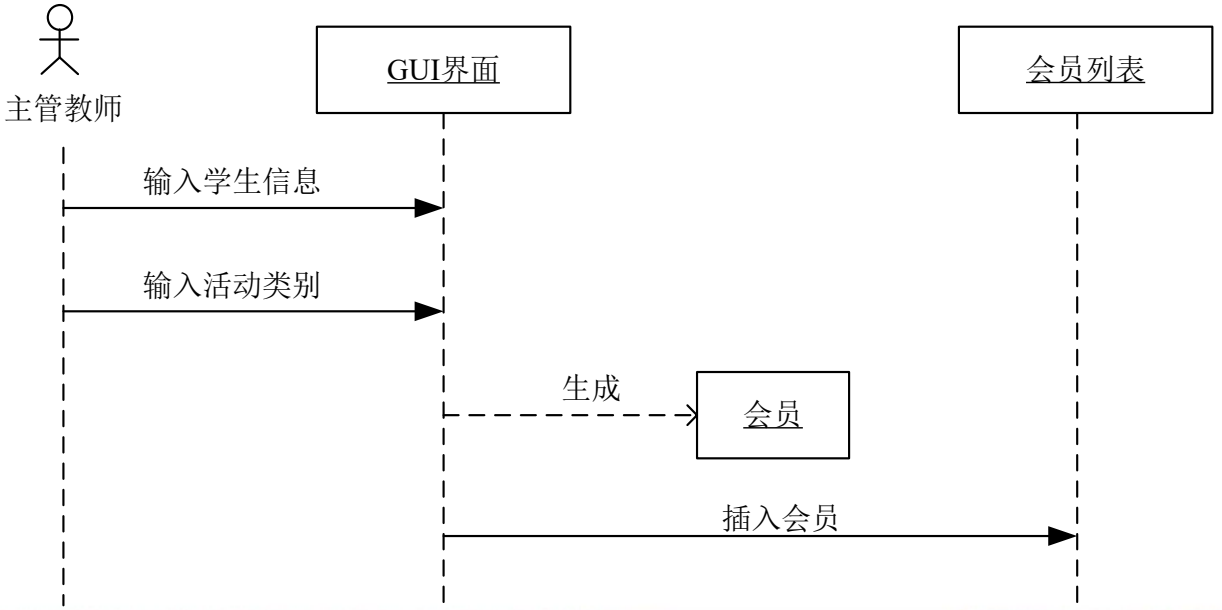
9.2.3 顺序模型建立步骤

1. 确定交互过程的上下文(context)；
2. 识别参与交互过程的对象；
3. 为每个对象设置生命线，即确定哪些对象存在于整个交互过程中，哪些对象在交互过程中被创建和撤销；
4. 从引发这个交互过程的初始消息开始，在生命线之间从顶到下依次画出随后的各个消息；
5. 如果需要表示消息的嵌套，或/和表示消息发生时的时间点，则采用FOC(focus of control)；
6. 如果需要说明时间约束，则在消息旁边加上约束说明；
7. 如果需要，可以为每个消息附上前置条件和后置条件。

9.2.3 建立顺序模型

◆根据以下用例描述的脚本，建立用例的顺序图。

- 某大学的课外爱好者协会系统，主管教师在有学生申请加入协会时，首先在会员登录GUI界面上输入学生的个人信息，然后再输入加入哪一类活动。
- 系统根据输入的信息新生成一会员，添加到会员列表中。
- 解答：根据用例描述的脚本，这个用例的参与者是主管老师，他是交互的发起者，他通过键盘等输入系统向GUI界面输入信息，即与GUI界面进行交互。GUI界面接收到消息后，创建一个会员对象，并将新创建的对象加入会员列表中。因此，对象包括：主管教师、GUI界面、会员以及会员列表。根据它们的交互，得到如下的顺序图。



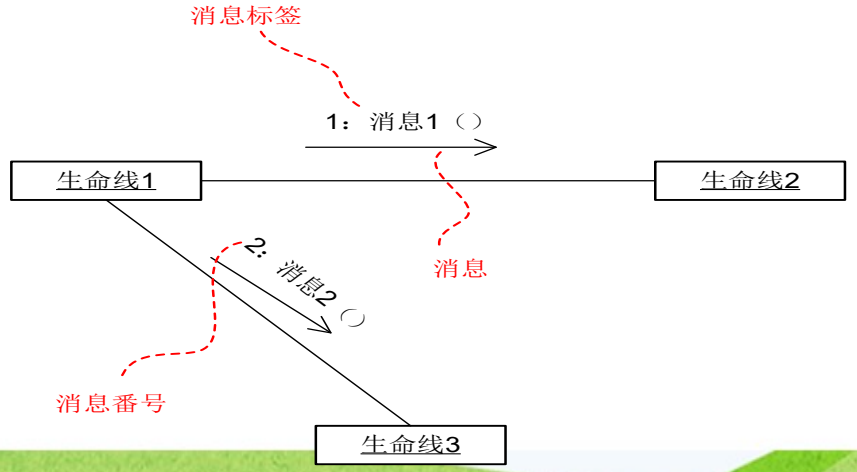
9.3 协作图

◆UML建立的协作图模型与类图结构相似，协作图中的对象角色与类图中的类相似，对象角色和对象角色之间的链与类图中的关联相似，但是，协作图只对相互间具有交互作用的对象角色和对象角色间的关联建模，而忽略了其他对象角色和链接。

◆协作图包含一组对象和链(link)，用于描述系统的行为是如何由系统的成分协作实现的。

9.3.1协作图的概要

- 协作图同顺序图一样，都属于交互图中的一种。
- 顺序图是按照时间顺序来配置消息描述生命线之间的消息交互，协作图是着眼于生命线之间的链接来描述生命线之间的消息交互。
- 顺序图和协作图可以互相转换。
- 协作由以下元素构成：
  - 生命线
    - 协作图中的生命线同顺序图中的生命线基本一致，唯一不同的是协作中生命线矩形框下面不带虚线。
  - 消息
    - 生命线之间用实线进行连接，实线上面标明消息方向和消息标签。

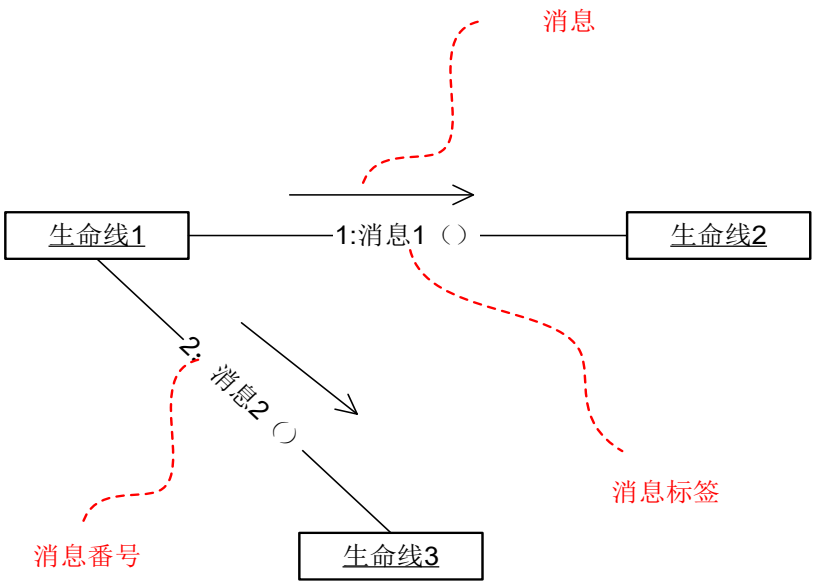


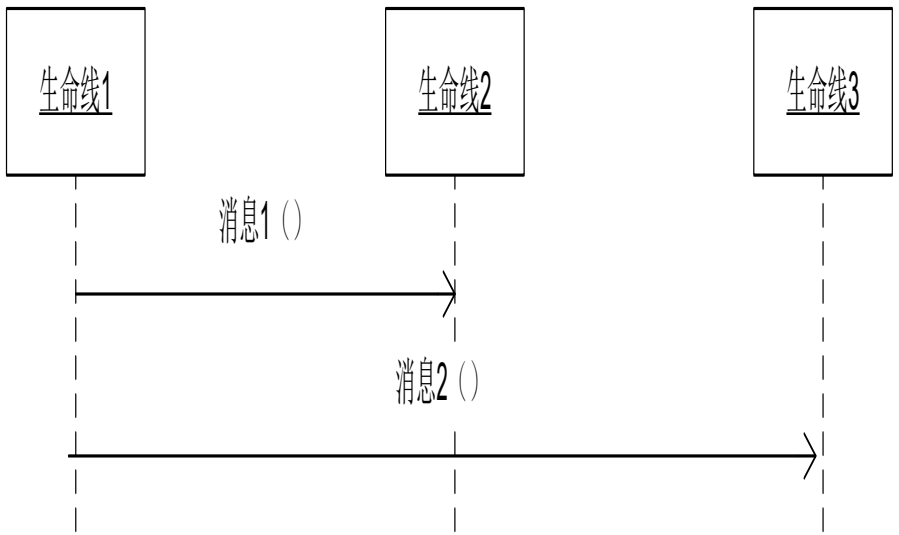
9.3.1协作图的概要（续）

- 消息种类：
  - 同步消息
  - 异步消息
  - 返回消息

- 顺序图中消息是按照时间顺序从上到下进行排列的，因此表征消息发送的先后顺序的消息编号可以省略。

  协作图中的消息没有按照时间先后顺序进行组织，需要注明消息编号。





9.3.2消息的标签格式

- 协作图中的消息标签格式：

前置项/ 消息编号 [监护条件] *[循环条件]： 返回值： = 消息名（参数1，参数2.。。）

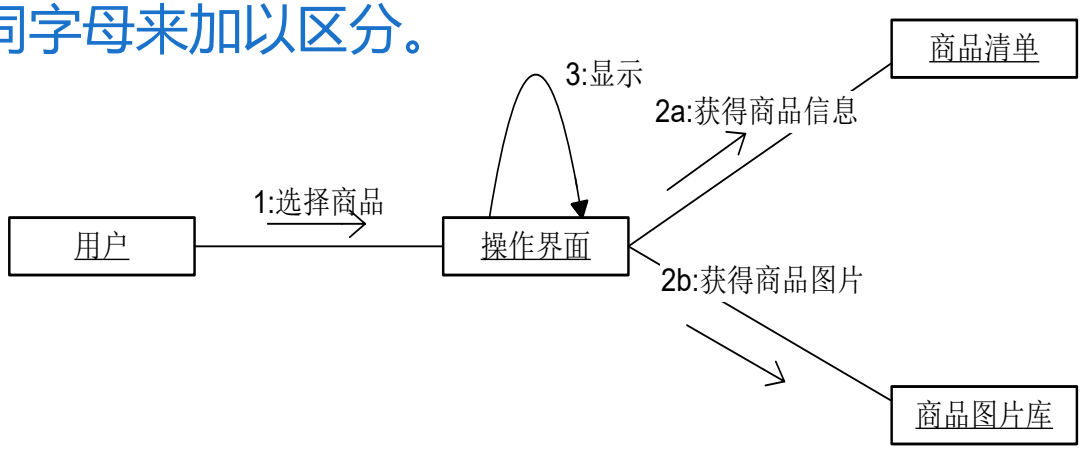
- 前置项
  - 用来指定发送该消息之前必须要发送完成的消息。
  - 被指定为前置项消息序号之后加“/”来表示。
  - 当有多个前置项时，之间用逗号进行分隔。
- 消息编号
  - 表明消息执行的先后顺序。
- 监护条件
  - 表示发送该消息必须具备的前提条件。
- 循环条件
  - 表示重复发送该消息的次数条件。

例： 1b,1c/ 2 [ID号码≠0] *[i=1..n]: 商品信息 : = 取得商品信息(商品ID)

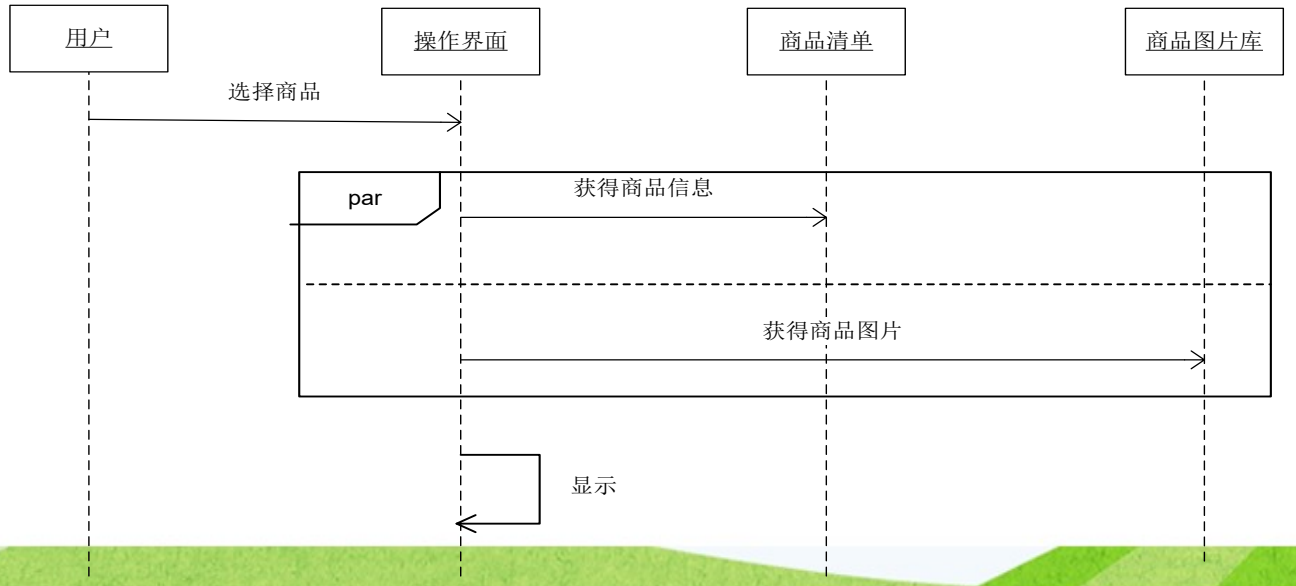
在消息1b和消息1c发送完毕前提下，当商品ID号码不为零时重复发送消息编号为2的消息，消息名称为“取得商品信息”，消息参数为“商品ID”，消息处理的返回值返回给变量“商品信息”。

9.3.3 并行处理的表示方法

- 多条消息的同时并行发送称为并行处理。
- 在顺序图中，用组合片段“par”来表示消息的并行发送。
- 在协作图中，为了表示并行发送的消息，相同数字的消息编号后面跟上不同字母来加以区分。

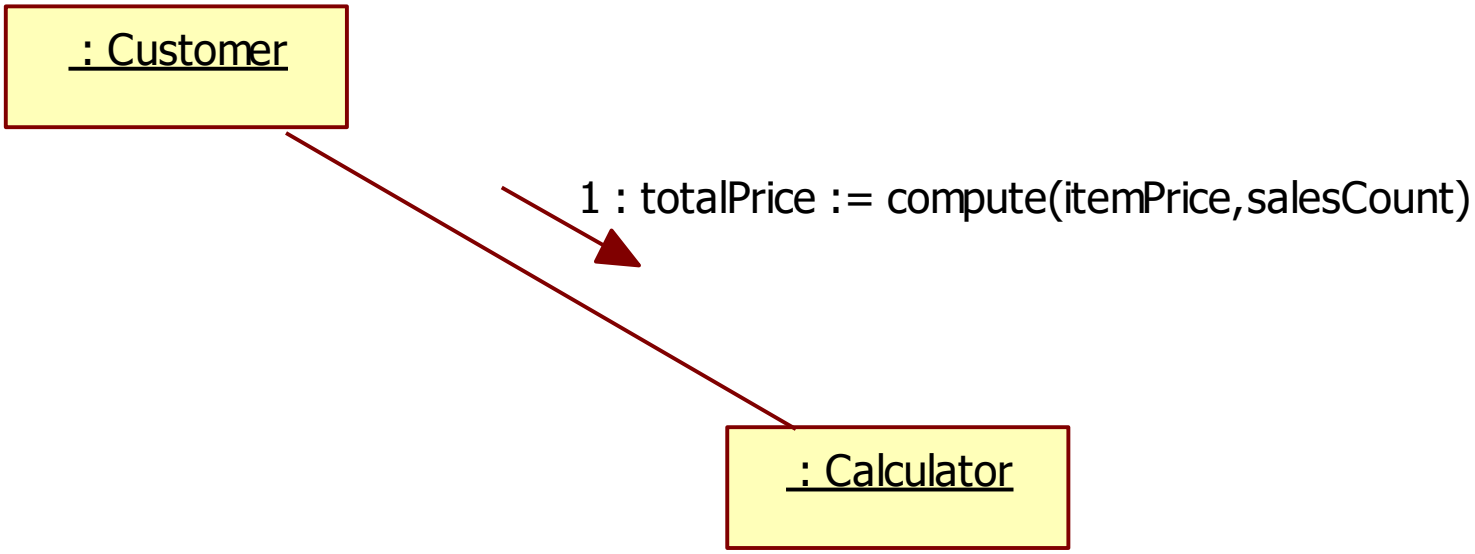


消息“获得商品信息”和“获得商品图片”是同时并行发送的消息。



消息返回结果

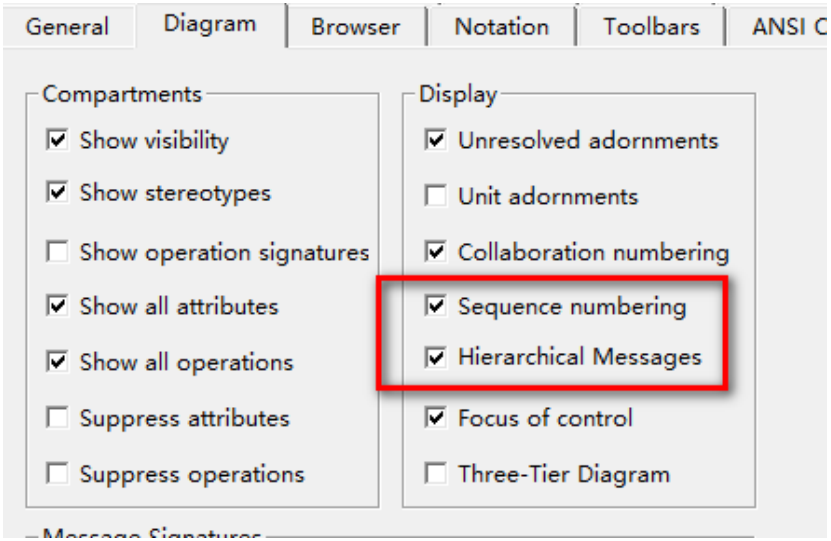
- 消息可能是要求某个对象进行计算并返回结果的值。例如一个顾客对象可能请求一个计算器（calculator）对象计算某项商品的总价，包括该项商品的价格和税款。
- UML提供了返回值的表示法。返回值的名字在最左，后跟赋值号“:=”，接着是操作名和操作的参数。对计算商品价格这个例子，可以表示成：totalPrice := compute(itemPrice, salesTax）。
- 表达式中赋值号的右边部分被称为消息签名（message signature)。

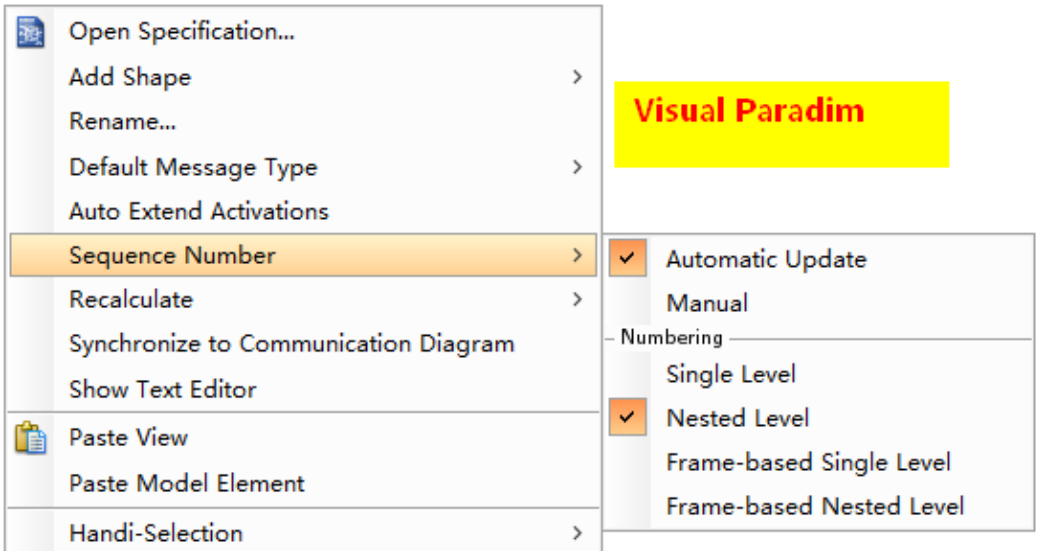


（1）消息编号

◆按消息产生的先后顺序给消息编号。有两种消息编号方案：一种是顺序编号；另一种是嵌套编号。

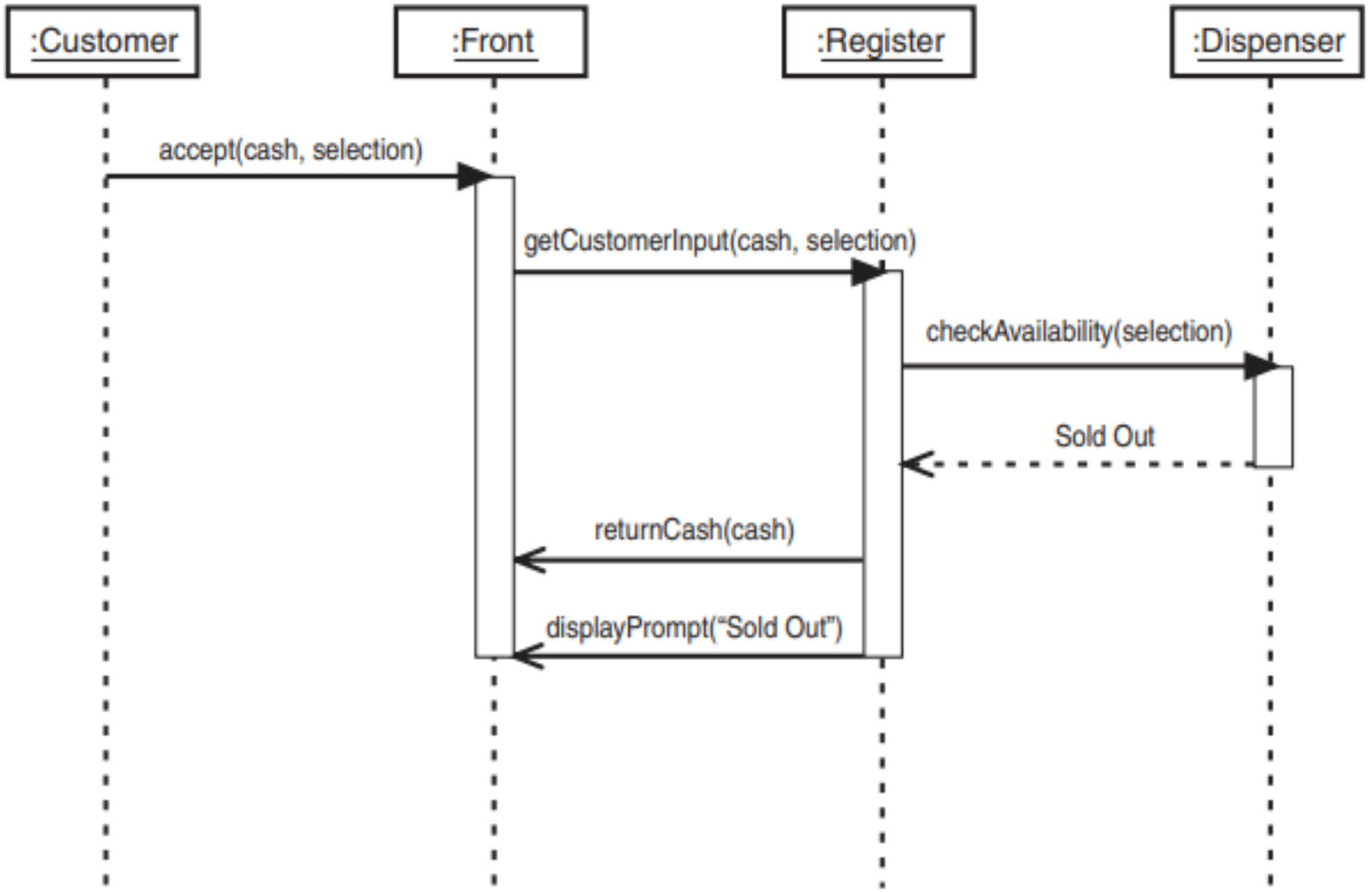
◆使整个消息的传递过程形成了一个完整的序列，因此通过在每个消息名的前面加上一个用冒号隔开的顺序号（按照消息的先后顺序，从1开始对消息编号）来表示消息的时间顺序。





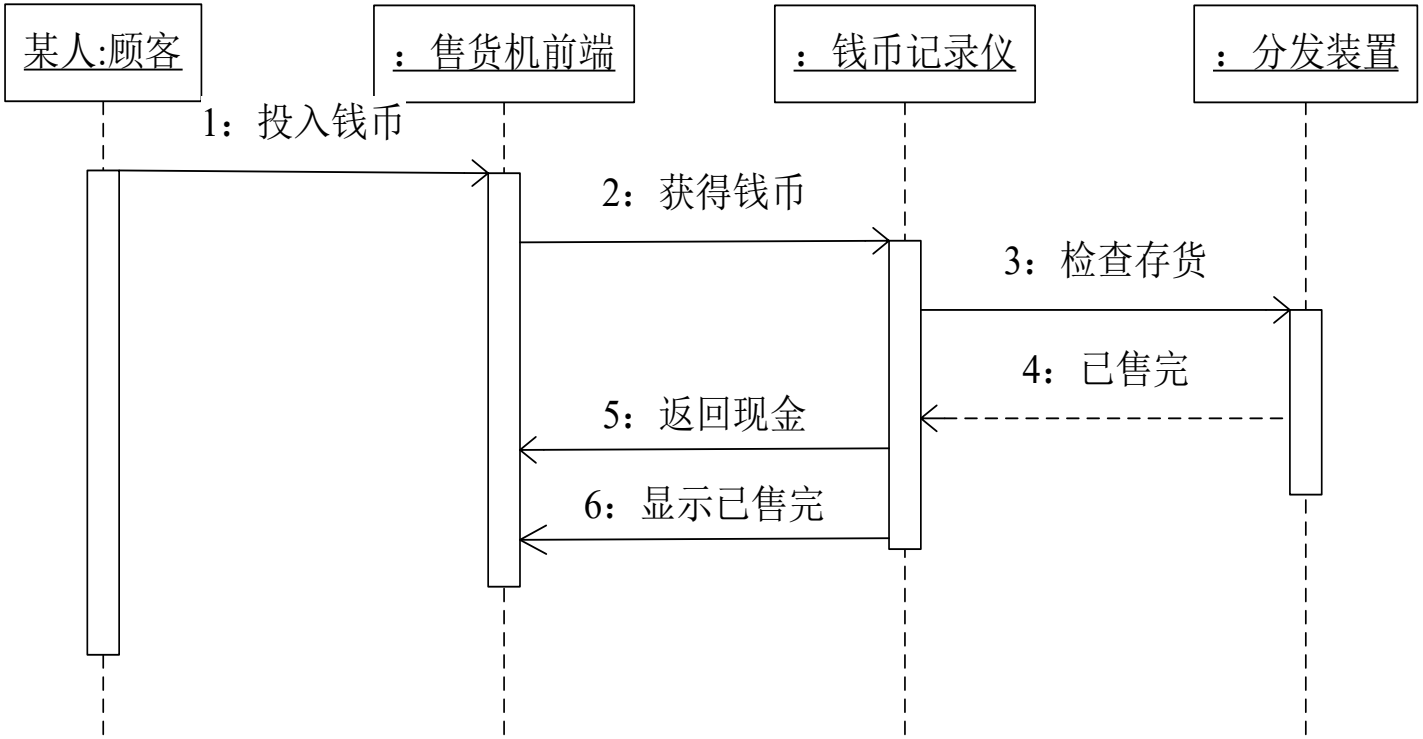
自动售货机“汽水已售完”：无序号

◆用visio画的图



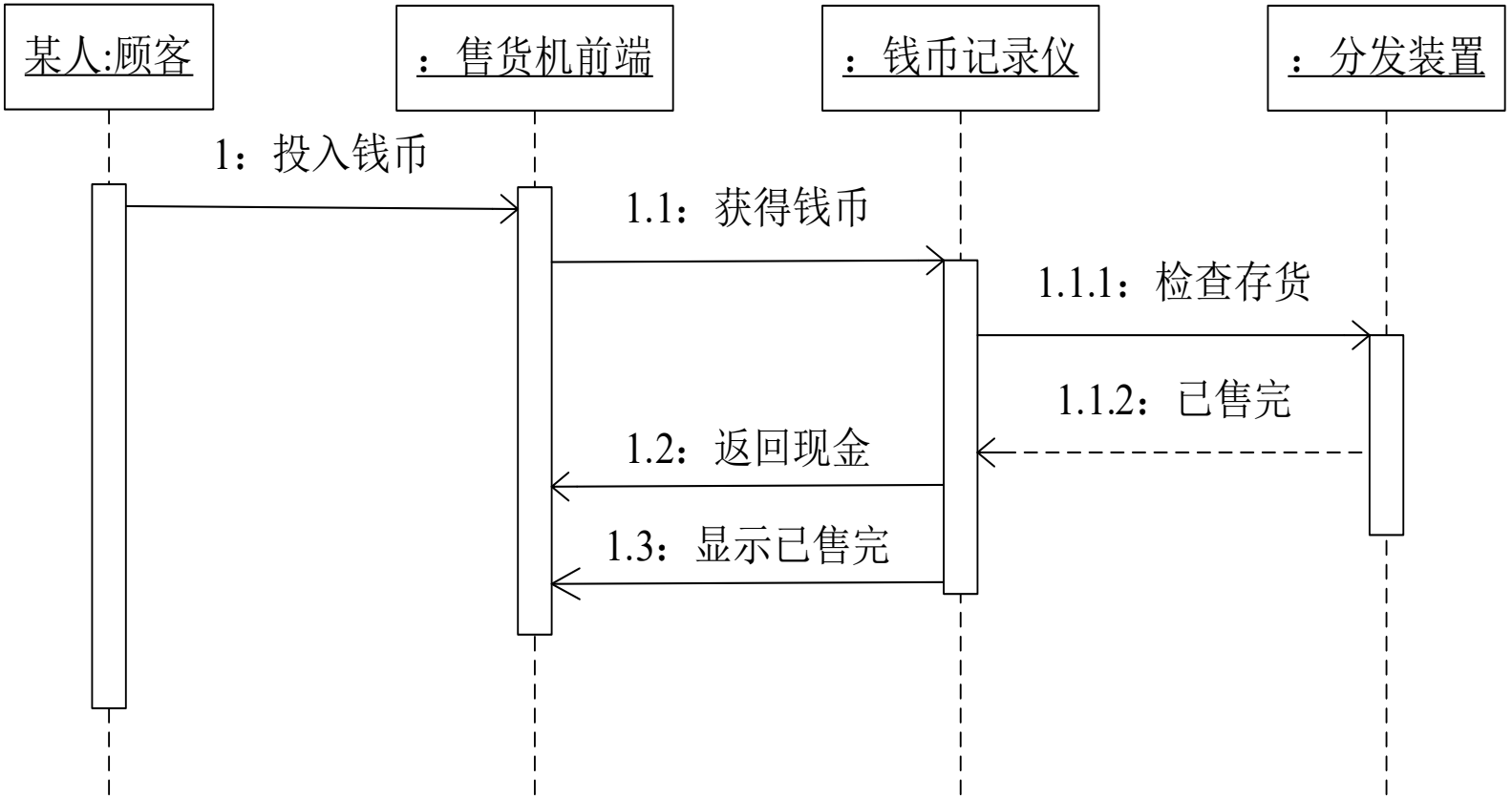
自动售货机“汽水已售完”：单级编号

◆下面是一个顺序图，该图演示了自动售货机“汽水已售完”的场景（加顺序编号）



(2) 嵌套编号

- ◆对自动收货机按照一种嵌套编号的方案修改:
- ◆把属于同一个对象发送和接收的消息放在同一层进行编号。
- ◆如"某人：顾客"的发送消息放在第一层编号，编号用1位数字表示，编号是1；
- ◆把"售货机前端"发送和接收的消息放在第二层编号，编号用2位数字表示，
- ◆给它们分配的编号是1.1、1.2、1.3；
- ◆匿名对象“钱币记录仪”的发送和接收消息 放在第三层编号，编号用3位数字数字表 示，给它们分配的编号是1.1.1、1.1.2。


某人:顾客
：售货机前端
：钱币记录仪
：分发装置
1：投入钱币
1.1：获得钱币
1.1.1：检查存货
1.1.2：已售完
1.2：返回现金
1.3：显示已售完

(2) 嵌套编号

- UML标准没有明确规定嵌套编号的格式和语法，但是提供了一些指导原则和示例。
- UML标准中定义了消息的顺序号（sequence number）为一个字符串，它表示消息在交互中的顺序和层次。
- UML标准中建议使用小数点分隔不同层次的顺序号，例如1.1，1.2，1.2.1等。但是层次也不宜太多。
- UML标准中也允许使用其他符号或字符来表示嵌套编号，例如1a，1b，1b1等。这里1a,1b表示并行处理的两个消息。(相同数字的消息编号后面跟上不同字母来加以区分的，表示并行处理。)
- UML标准中还允许使用星号（*）来表示迭代或循环，例如2*或2.3*等。
- 不同的工具嵌套编号方法有不同。总之能看出来消息发送的顺序。

(2) 嵌套编号（UML 2通信图)

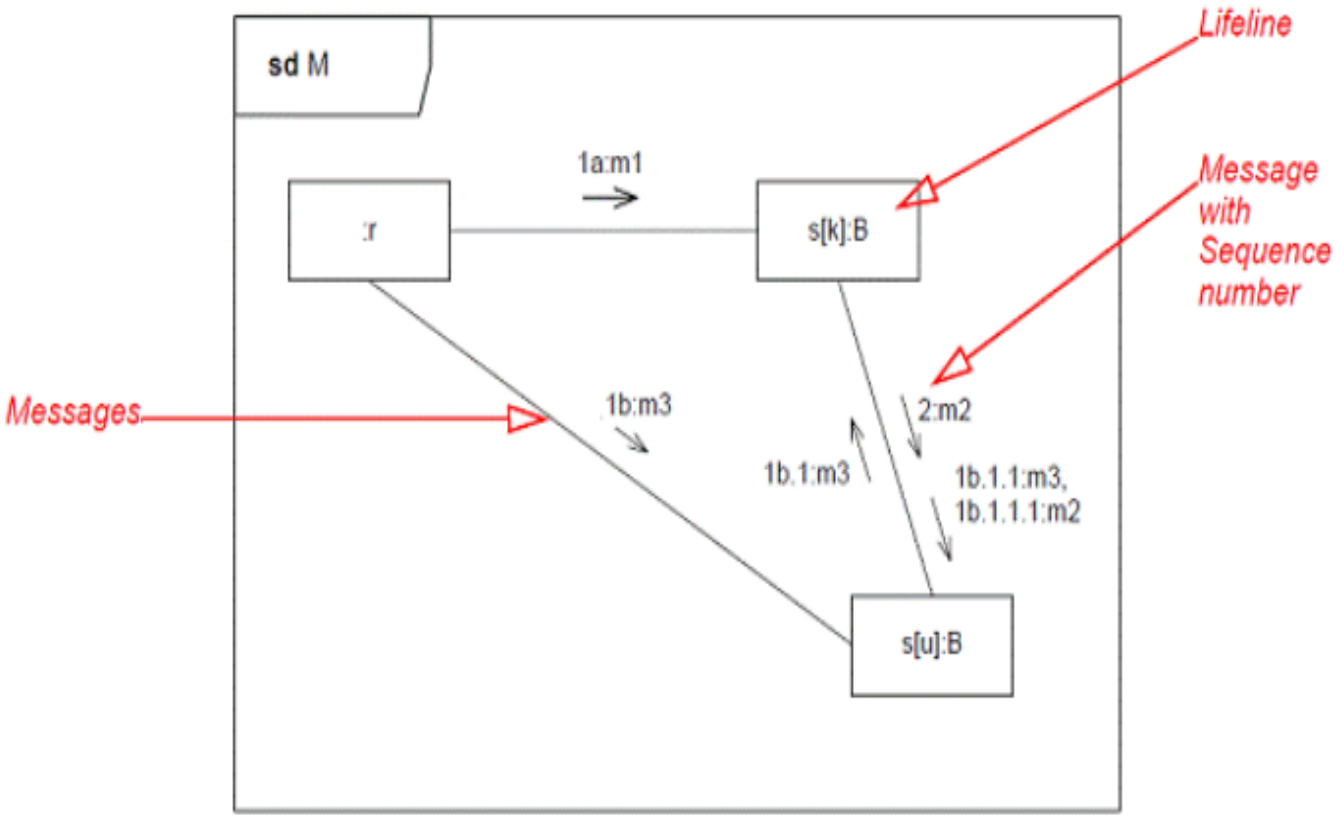


Figure 17.26 Communication diagram

The Interaction described by a Communication Diagram in Figure 17.26 shows messages m1 and m3 being sent concurrently from :r towards two instances of the part s. The sequence numbers show how the other messages are sequenced. 1b.1 follows after 1b and 1b.1.1 thereafter etc. 2 follows after 1a and 1b.

循环和分支控制

◆1．循环表示

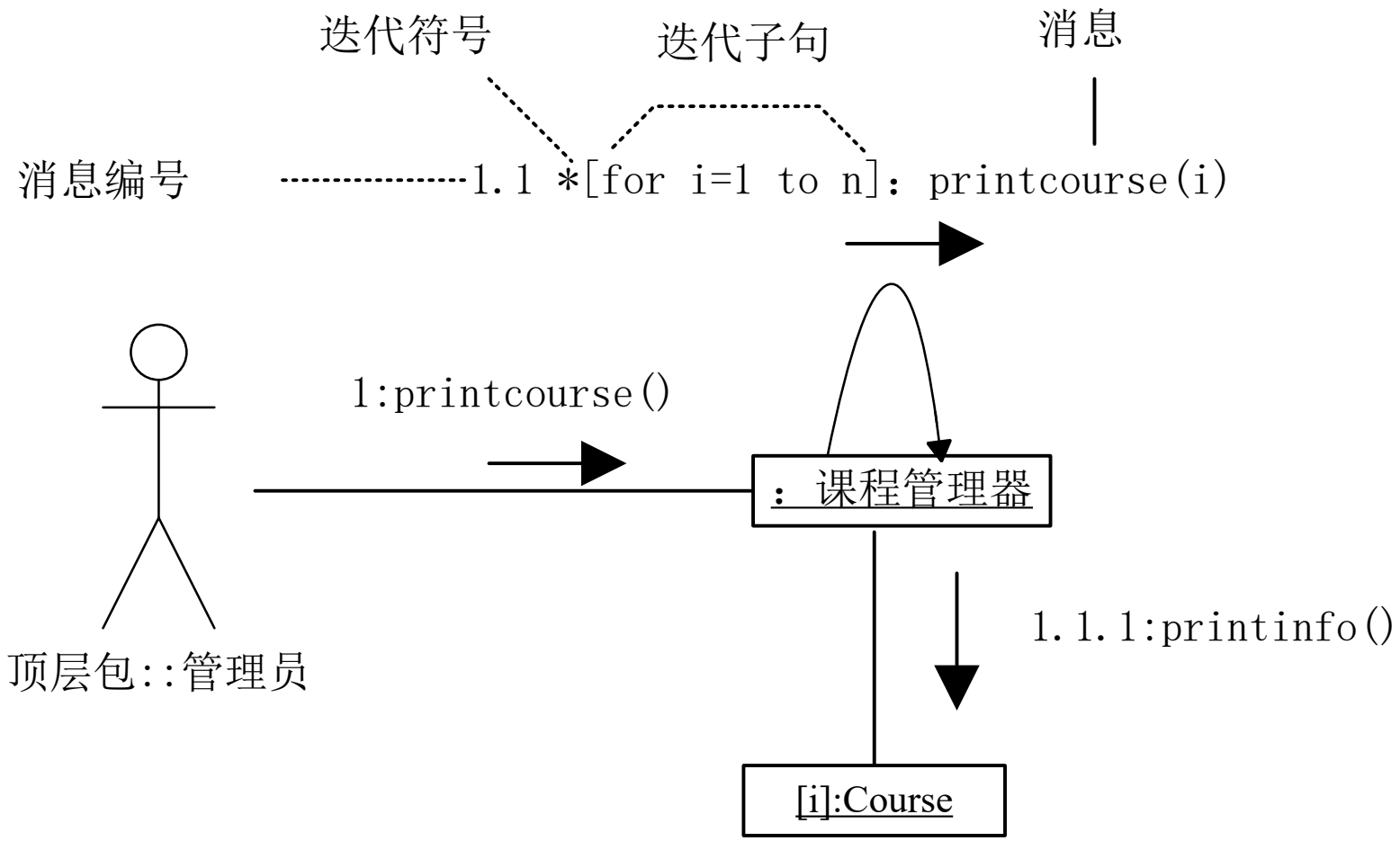
◆在协作图中，用一个迭代符“*”和迭代子句（可选）来表示循环。可以使用任何有意义的句子来表示迭代子句。常用的迭代表子句如表所示。

常用迭代表达式

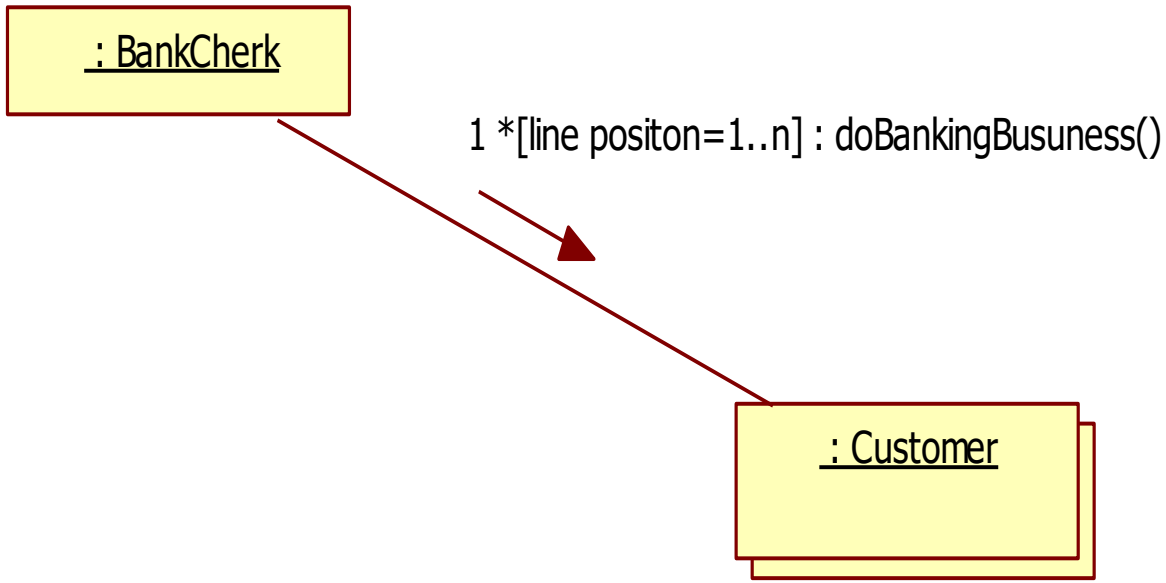
迭代子句	语　义
[i:=1...n]	迭代 n 次
[I=1...10]	I 迭代 10 次
[while(表达式)]	表达式为 true 时才进行迭代
[until(表达式)]	迭代到表达式为 true 时，才停止迭代
[for each(对象集合)]	在对象集合上迭代

循环和分支控制

◆管理员通过课程管理器打印所有的课程信息的协作图。



◆有时，按顺序发送消息是很重要的。例如，银行出纳员（bank clerk）要按照顾客排队的次序为排名顾客（customer ）服务。可以用“while”条件表达出消息的顺序（例如“line position=l...n”）



2．分支的表示

◆在协作图中，用监护条件来表示分支。监护条件的格式是：

[条件表达式]

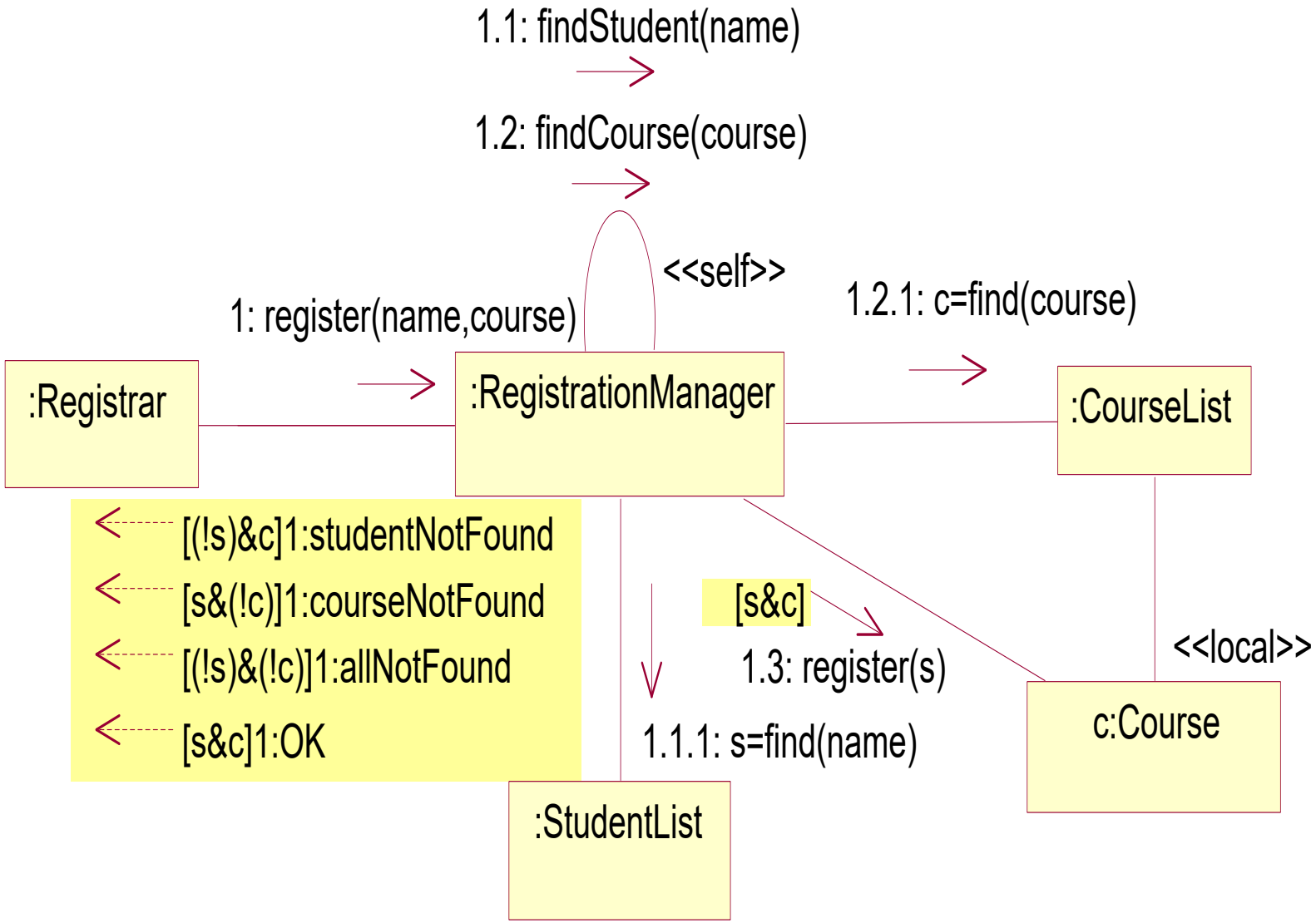
当监护条件为真时，才执行消息。

◆在通信图中使用监护条件一定要有所限制，通常应只列出主要的监护条件，否则会影响其阅读。如果需要，尽可能还是通过顺序图来表示

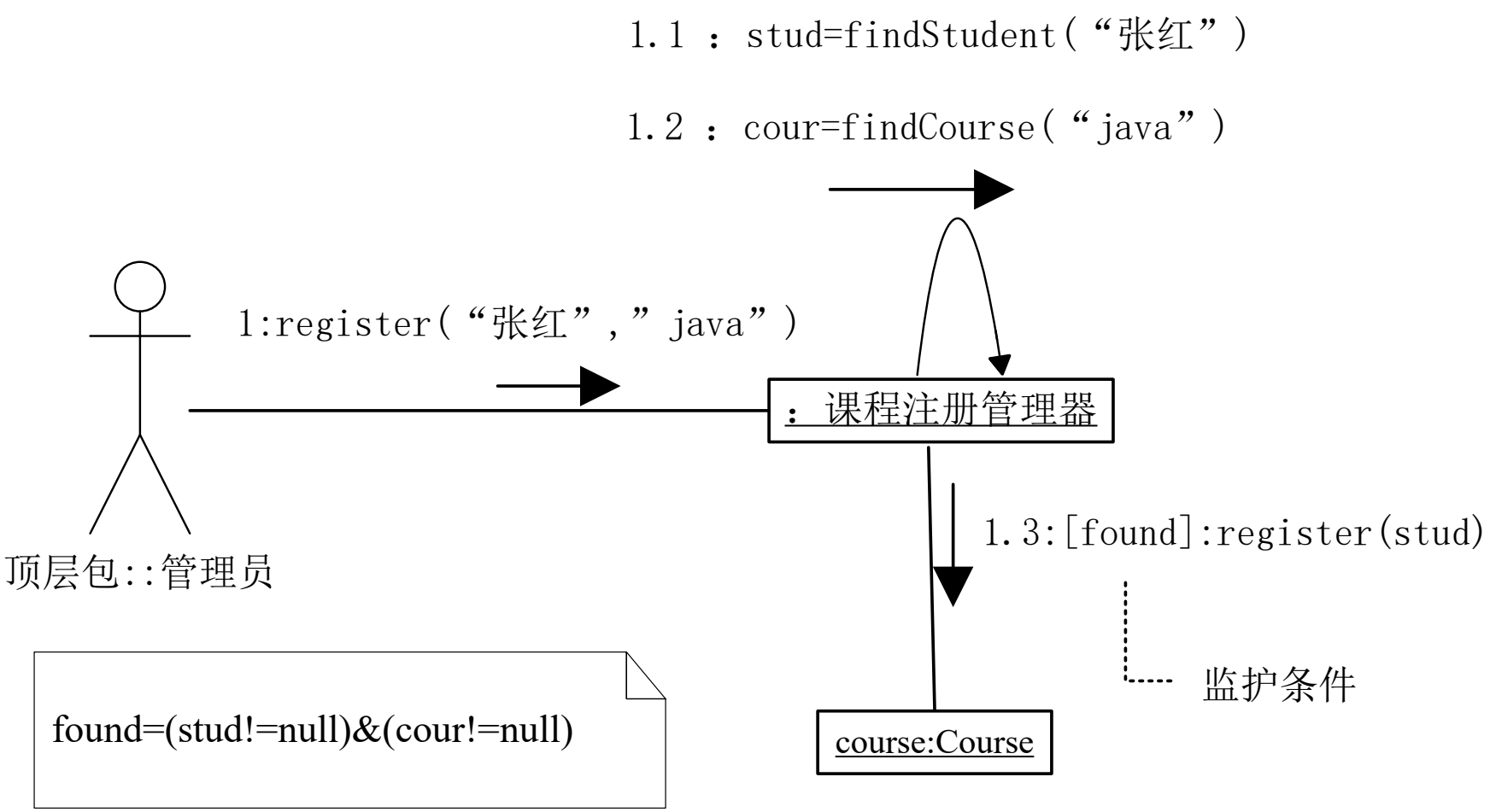
◆课程注册系统的协作图。为学生注册课程包括三个步骤：

- ■第一步：在系统中找出某个学生的信息；
- ■第二步：在系统中找出正确的课程；
- ■第三步：在学生和课程都存在的情况下，为该学生注册。

2．分支的表示


1.1: findStudent(name)
1.2: findCourse(course)
<<self>>
1: register(name,course)
1.2.1: c=find(course)
:Registrar
:RegistrationManager
:CourseList
[(!s)&c]1:studentNotFound
[s&(!c)]1:courseNotFound
[(!s)&(!c)]1:allNotFound
[s&c]1:OK
[s&c]
1.3: register(s)
<<local>>
1.1.1: s=find(name)
c:Course
:StudentList

2．分支的表示(简化画法)


1.1 ： stud=findStudent(“张红”)
1.2 ： cour=findCourse(“java”)
1:register(“张红”,”java”)
：课程注册管理器
1.3:[found]:register(stud)
顶层包::管理员
监护条件
found=(stud!=null)&(cour!=null)
course:Course

9.3.5 例题：画协作图

云计算的核心思想，是将大量用网络连接的计算资源统一管理和调度，构成一个计算资源池向用户按需服务，资源管理是其主要功能。在资源管理中，包含一个资源申请用例，其执行流程为：

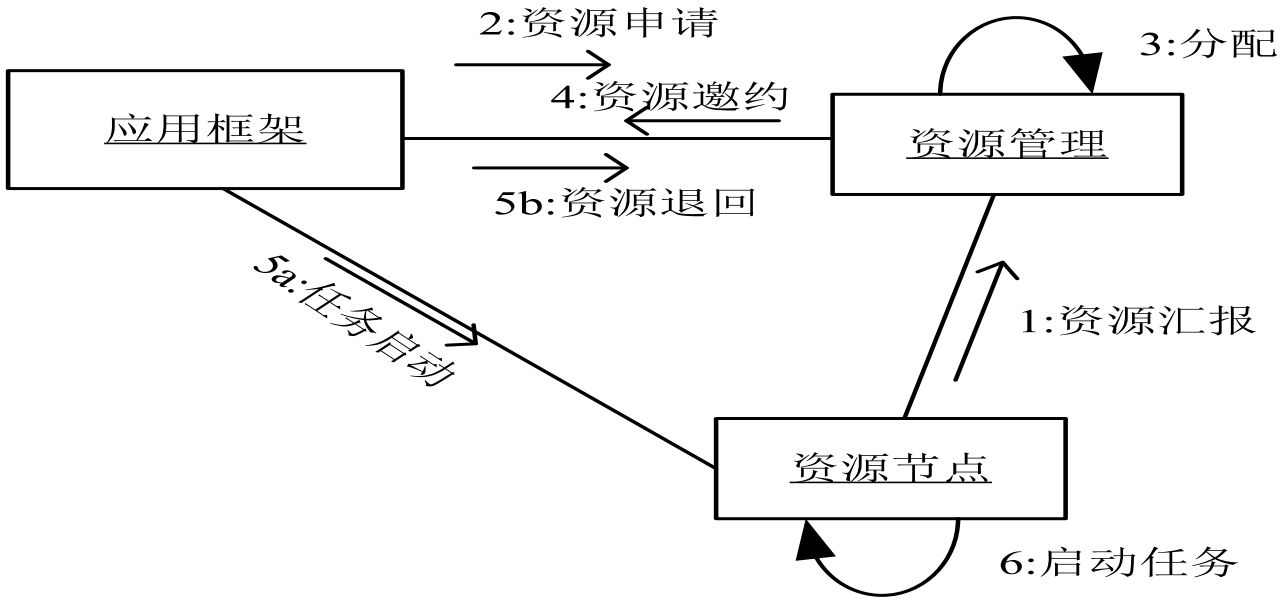
- 接收到用户的应用框架的资源请求后，资源管理器在资源池中找到可用的资源，然后以资源邀约的形式提交给应用框架，应用框架接收到资源邀约后，选择自己合适的资源，并将剩余的资源邀约退回资源管理器。
- 应用框架向资源节点发送启动任务的请求，资源节点收到任务启动请求后，在该节点上启动任务运行的容器。
- 相对于资源管理器，各个资源节点首先向资源管理器汇报自己的可用资源，该资源是资源管理器分配资源的依据。

根据以上描述，画出资源申请的协作图。

题解：

协作图体现的是资源申请用例中的对象之间的交互。参与交互的对象有：应用框架、资源管理器、资源节点。

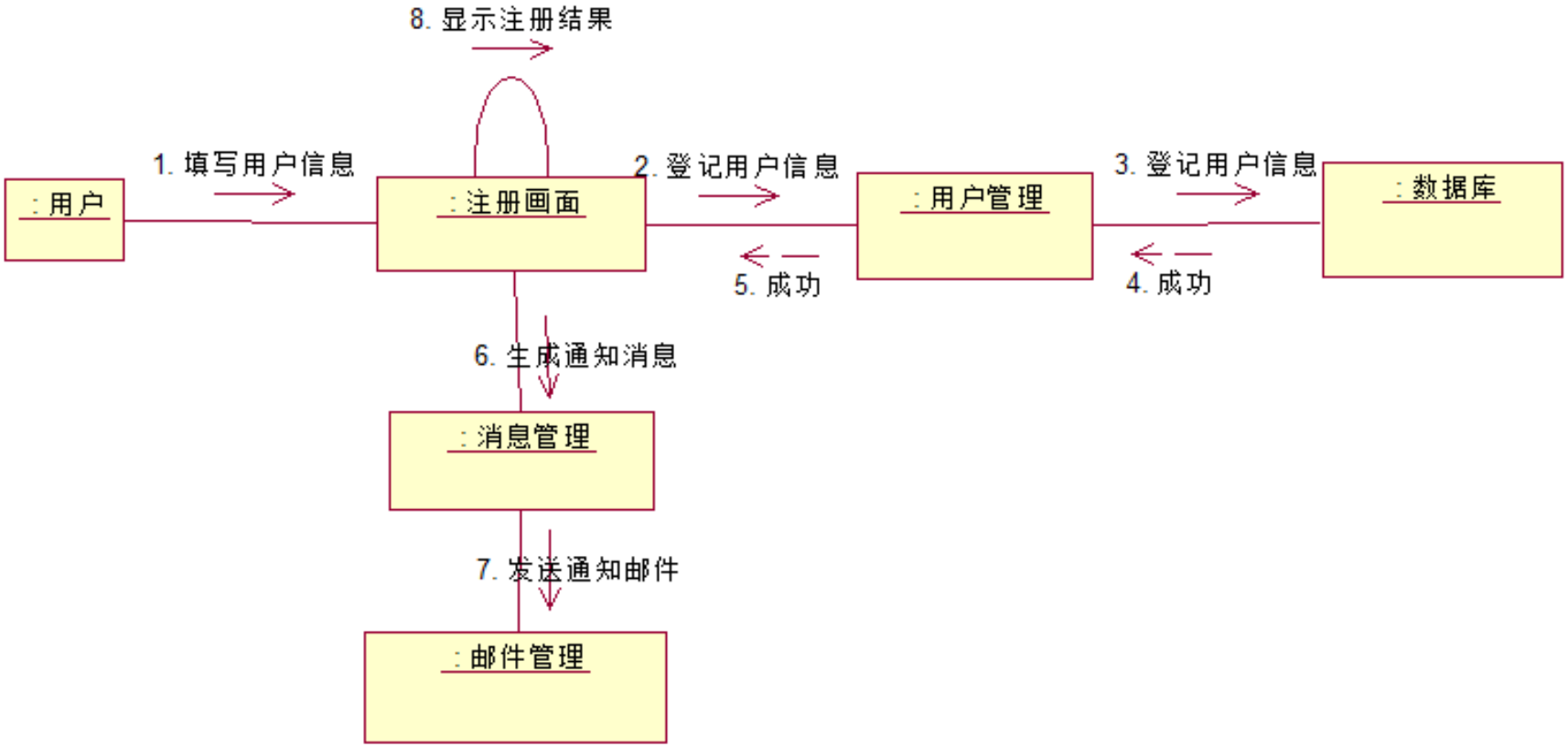
从资源申请用例的描述看，资源管理器与应用框架有联系，也与资源节点有联系；资源节点与应用框架也有联系。故，协作图如图



用户注册

1. 用户在注册画面上填写用户信息。
2. 确定后把用户信息登记在数据库中。
3. 通过电子邮件通知用户注册结果等消息。
4. 在注册画面上显示注册结果。

◆解答


8. 显示注册结果
1. 填写用户信息
2. 登记用户信息
3. 登记用户信息
: 用户
: 注册画面
: 用户管理
: 数据库
5. 成功
4. 成功
6. 生成通知消息
: 消息管理
7. 发送通知邮件
: 邮件管理

9.4 顺序图和协作图的比较

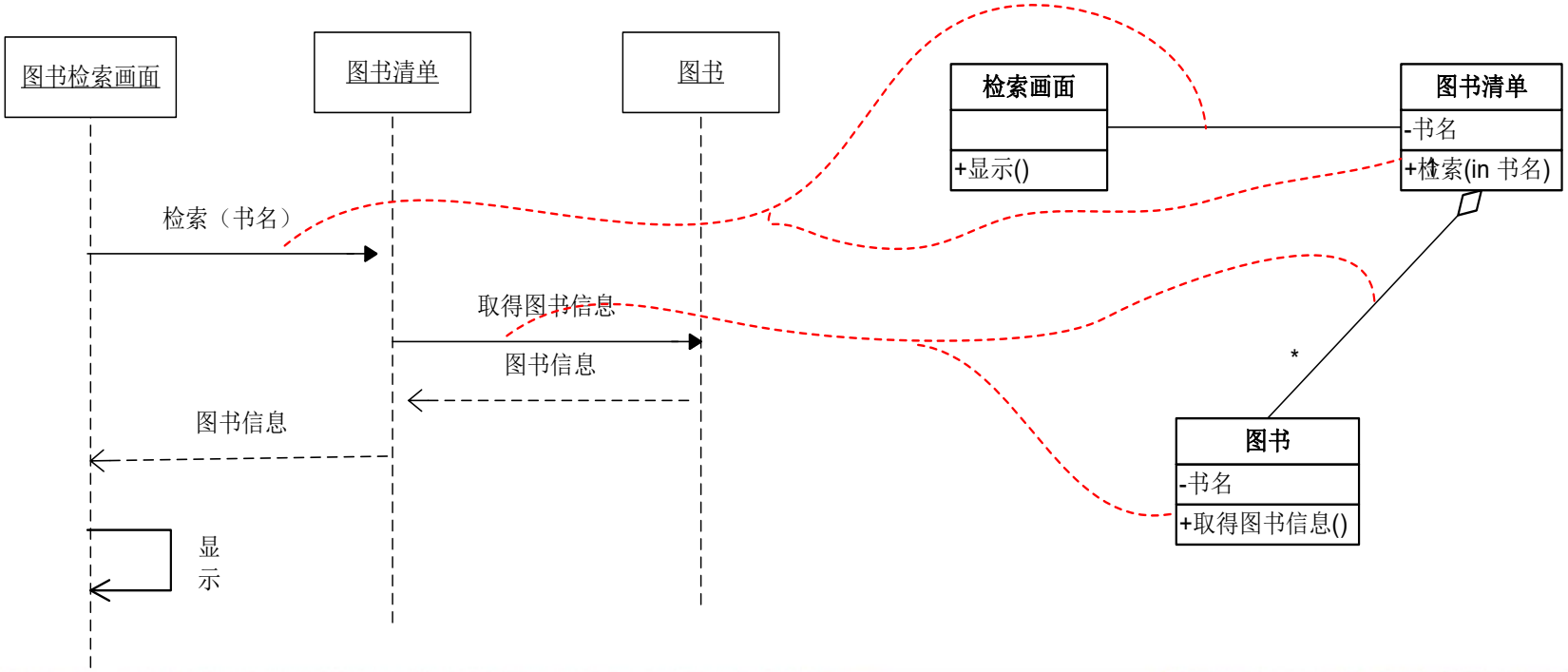
- 顺序图强调消息的时间顺序，协作图强调参加交互的对象的组织，两者可以相互转换。
- 顺序图不同于协作图的两个特征：
  - 顺序图有对象生命线
  - 顺序图有控制焦点
- 协作图不同于顺序图的两个特征：
  - 协作图有路径
  - 协作图必须有消息顺序号

9.4 顺序图和协作图的比较

- 顺序图是按照时间顺序从上到下排列消息来描述系统中生命线之间的消息交互情况。协作图是着眼于生命线之间的链接来描述生命线之间的消息交互情况。
- 顺序图中的消息可以省略消息编号。协作图中的消息必须明确消息编号来表示消息发送的先后顺序。
- 顺序图中，用组合片段“par”来描述消息的并行处理。协作图中，用“数字+字母”的消息编号定义方式来描述消息的并行处理。

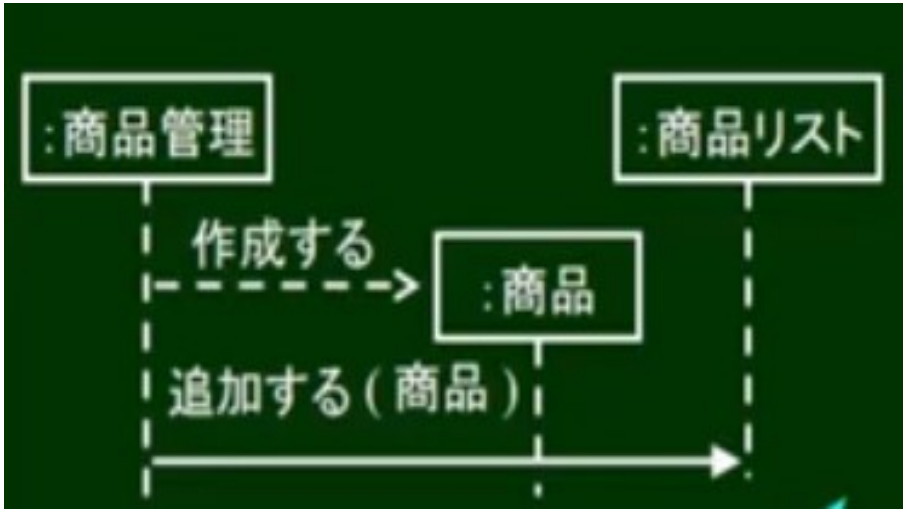
9.4 类图和交互图的一致性

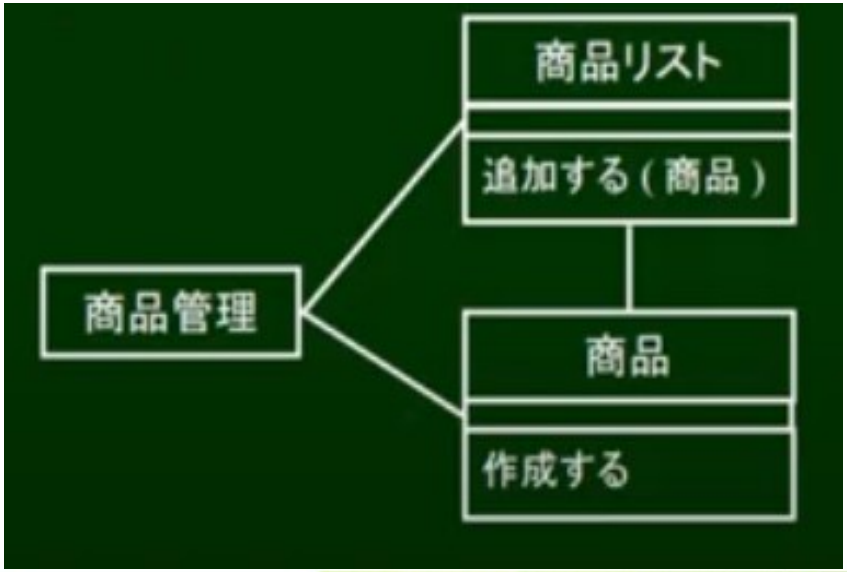
- 类图表示的是系统的静态结构，交互图表示的是系统的动态行为。两者之间要保持一致性。
- 保持类图和交互图之间的一致性观点有：
  - 交互图中的生命线对应于类图中适当的类。
  - 交互图中有消息交互的两个生命线，在类图中对应的两个类之间应该有一定的关系。
  - 交互图中的消息，在类图中接受该消息的生命线所对应的类中应该有某个操作与之相对应。



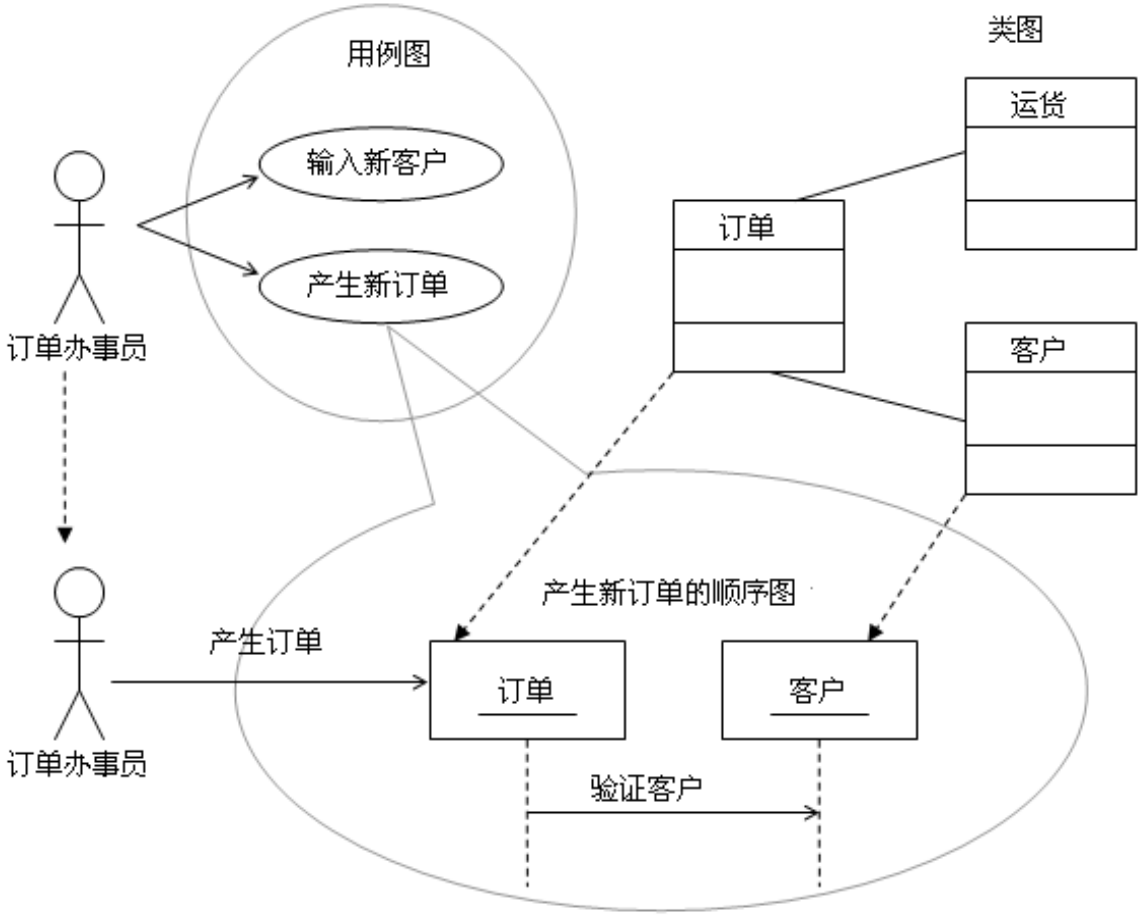
9.4 类图和交互图的一致性

- 生命线(对象)←→类
  - 生命线上的对象具有对应的类。这里是商品List，商品，商品管理三个类。
- 消息的交互←→类之间的关系
  - 商品管理和商品，商品List之间有消息交互，所以它们之间有关联关系
- 消息接收者←→类的操作
  - 类的操作属于消息接收者. 这里是商品的操作“创建(作成)"和商品List的操作"追加（商品)"。
- 操作需要的信息(返回值，参数) ←→类的属性或者关系
  - 因为”追加“的参数是”商品“，所以商品和商品List之间有关联关系。





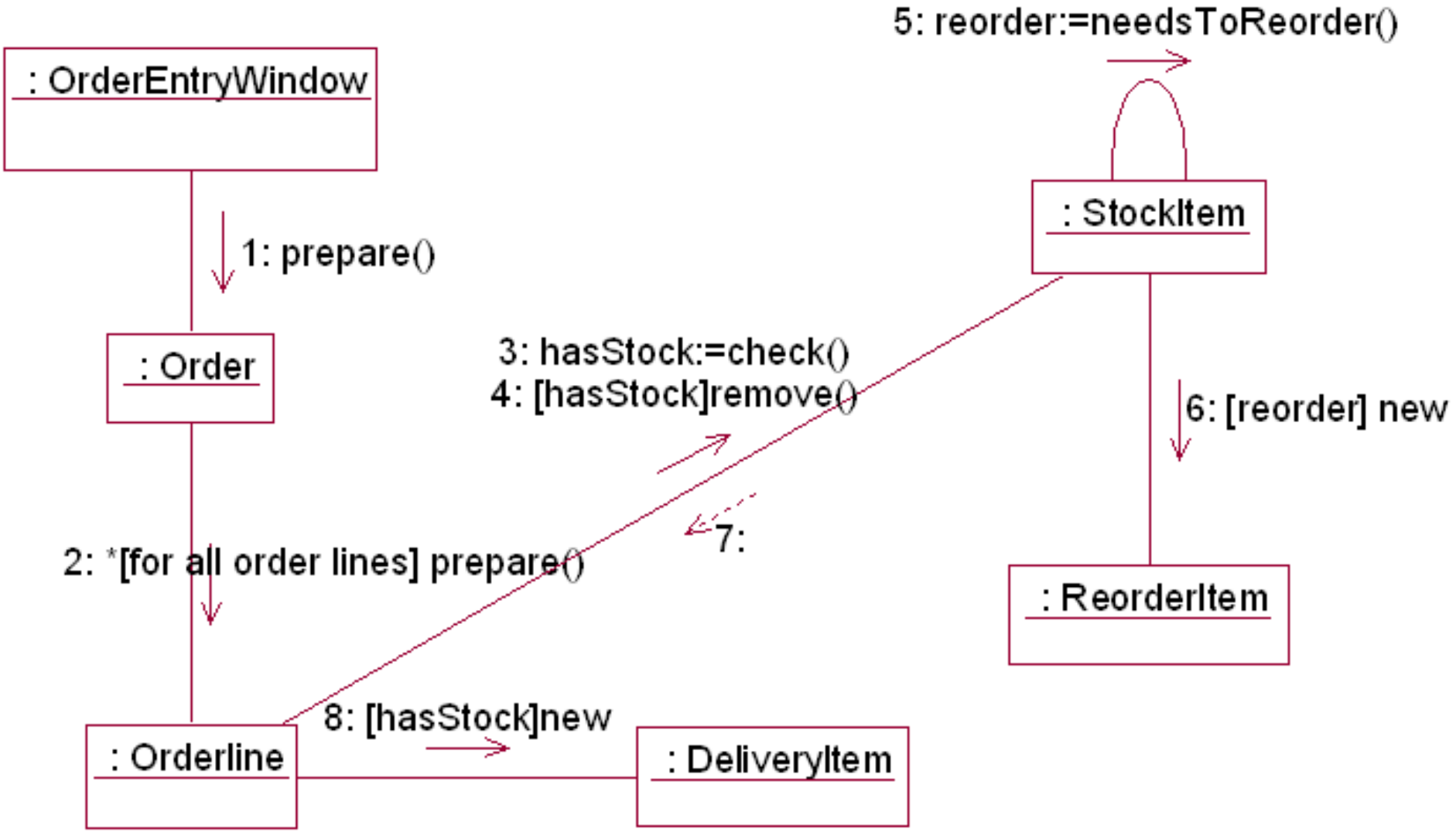
顺序图与用例图和类图之间关系图



建立collaboration图的步骤

1. 确定交互过程的上下文(context)；
2. 识别参与交互过程的对象；
3. 如果需要，为每个对象设置初始特性；
4. 确定对象之间的链(link)，以及沿着链的消息；
5. 从引发这个交互过程的初始消息开始，将随后的每个消息附到相应的链上；
6. 如果需要表示消息的嵌套，则用Dewey十进制表示法；
7. 如果需要说明时间约束，则在消息旁边加上约束说明；
8. 如果需要，可以为每个消息附上前置条件和后置条件。

Collaboration图的例子：由顺序图转换来的协作图



协作图建模风格

类似于顺序图：

◆把注意力集中于关键的交互。

◆对于参数，优先考虑使用参数名而不是参数类型。

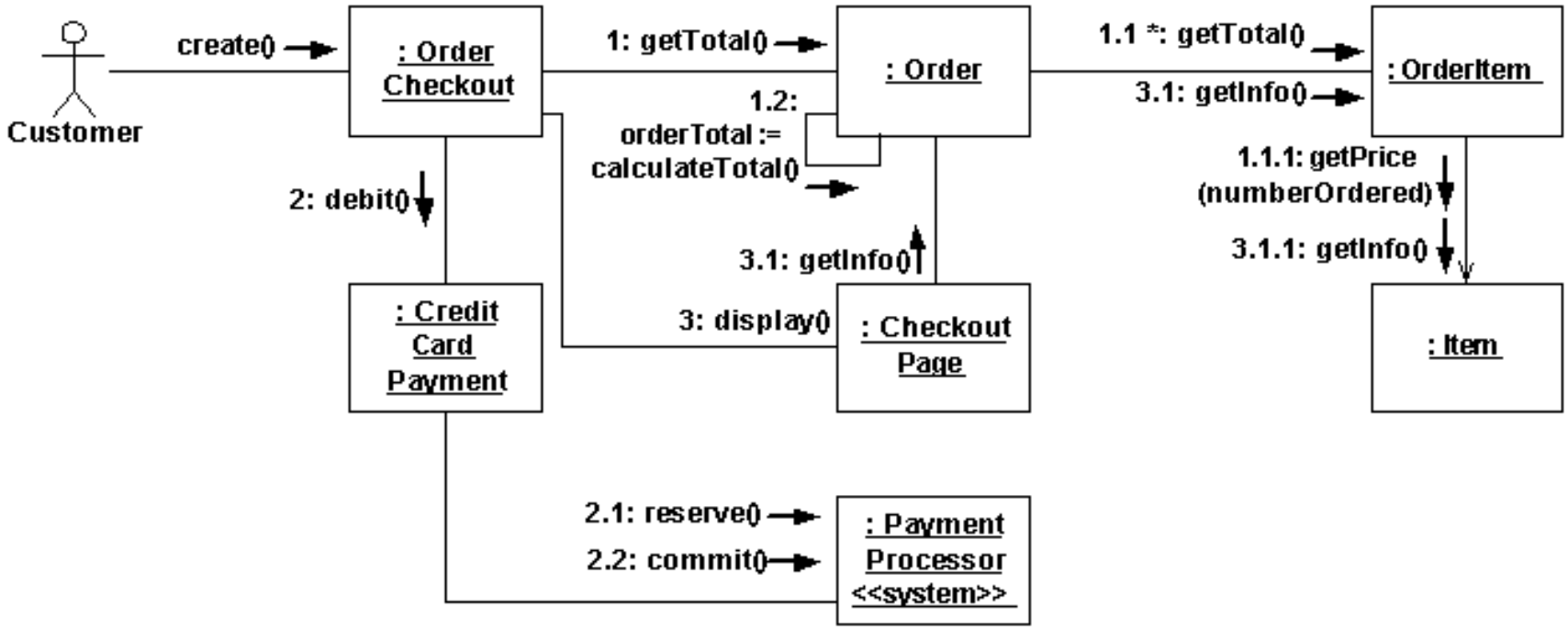
◆不要对明显的返回值建模。

◆可以把返回值建模为方法调用的一部分。

◆建模风格5：为每个消息画出箭头。

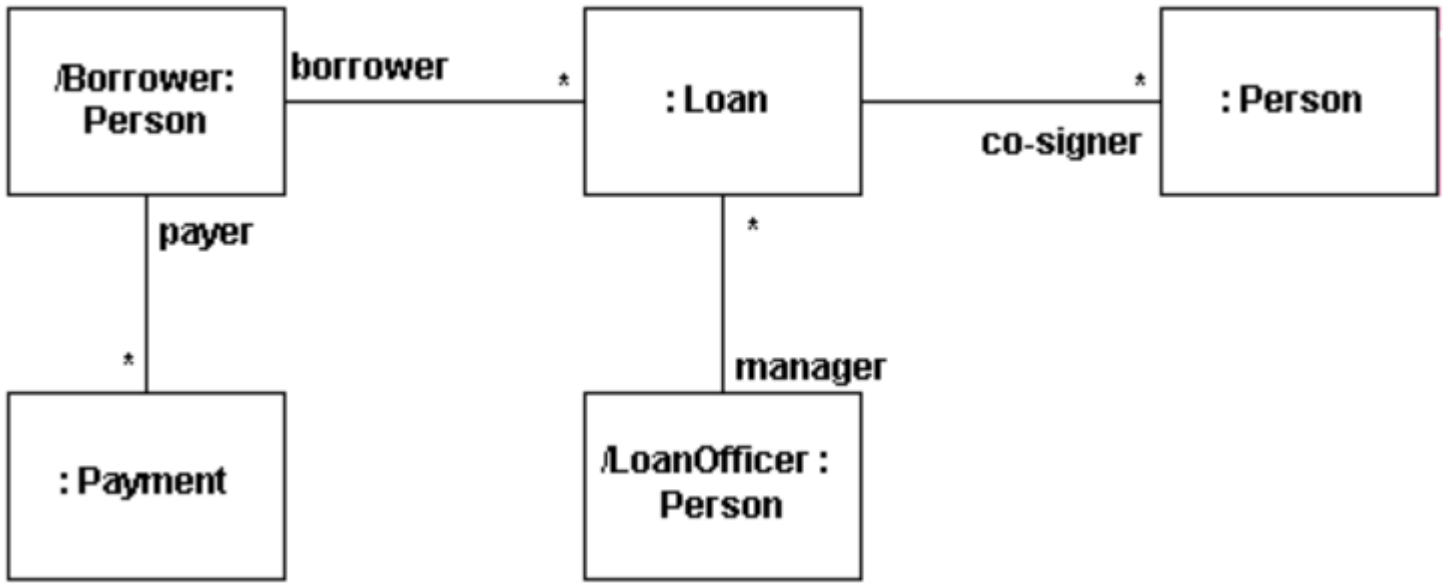
■便于可视化地确定流向一个给定对象的消息数量，以此判断该对象所涉及的潜在耦合情况，这通常是在对设计进行重构时要考虑的一个重要因素。

■例子：



◆建模风格6：注明导航性(Navigability)时要慎重。

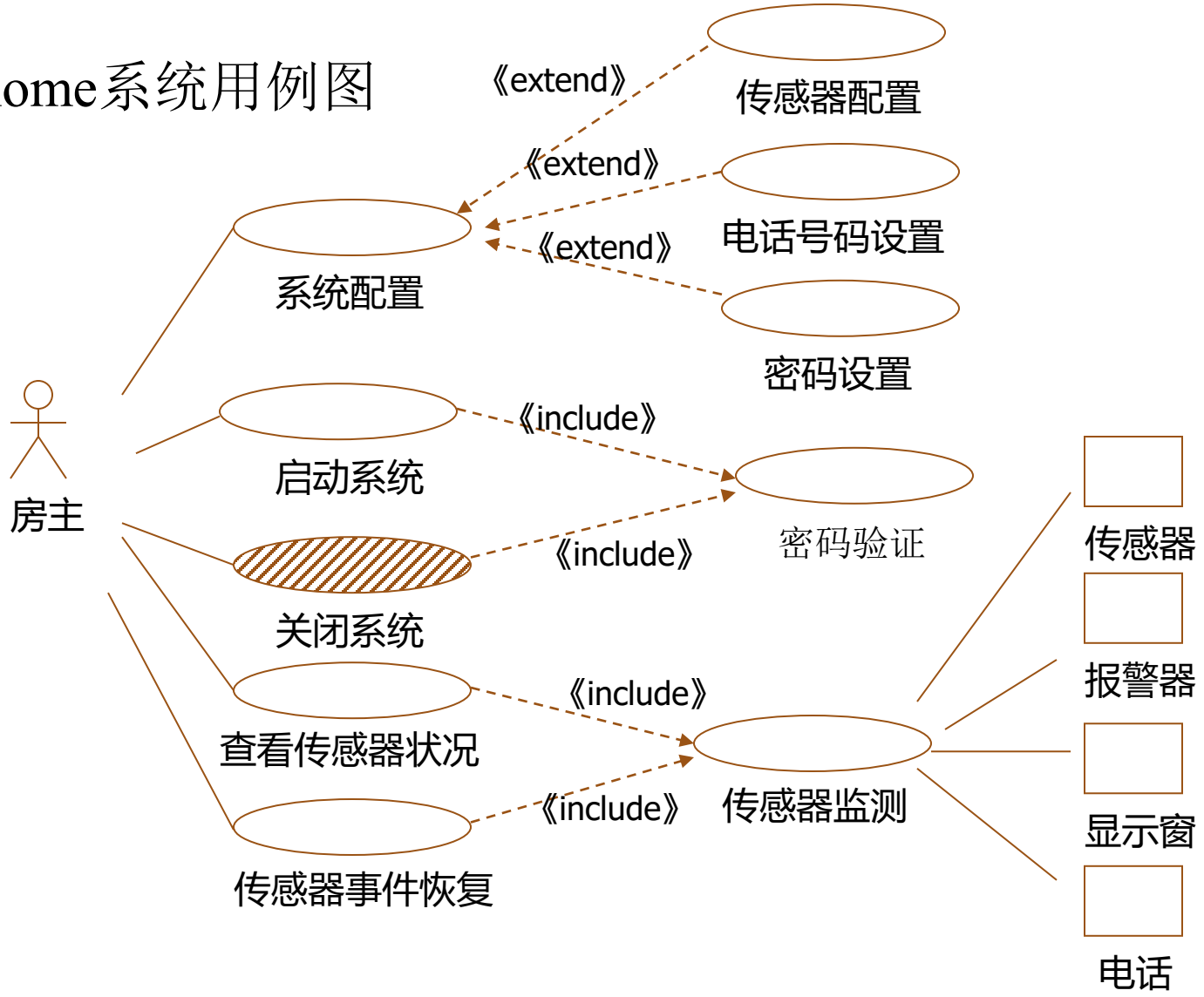
- ■可以在协作图中对导航性建模，但这不是很常见，因为这太容易和消息流混淆。
- ■用UML类图来描绘导航性更好。
- ■链是关联的实例，包括关联、聚合、组合、依赖。
- ■例子：



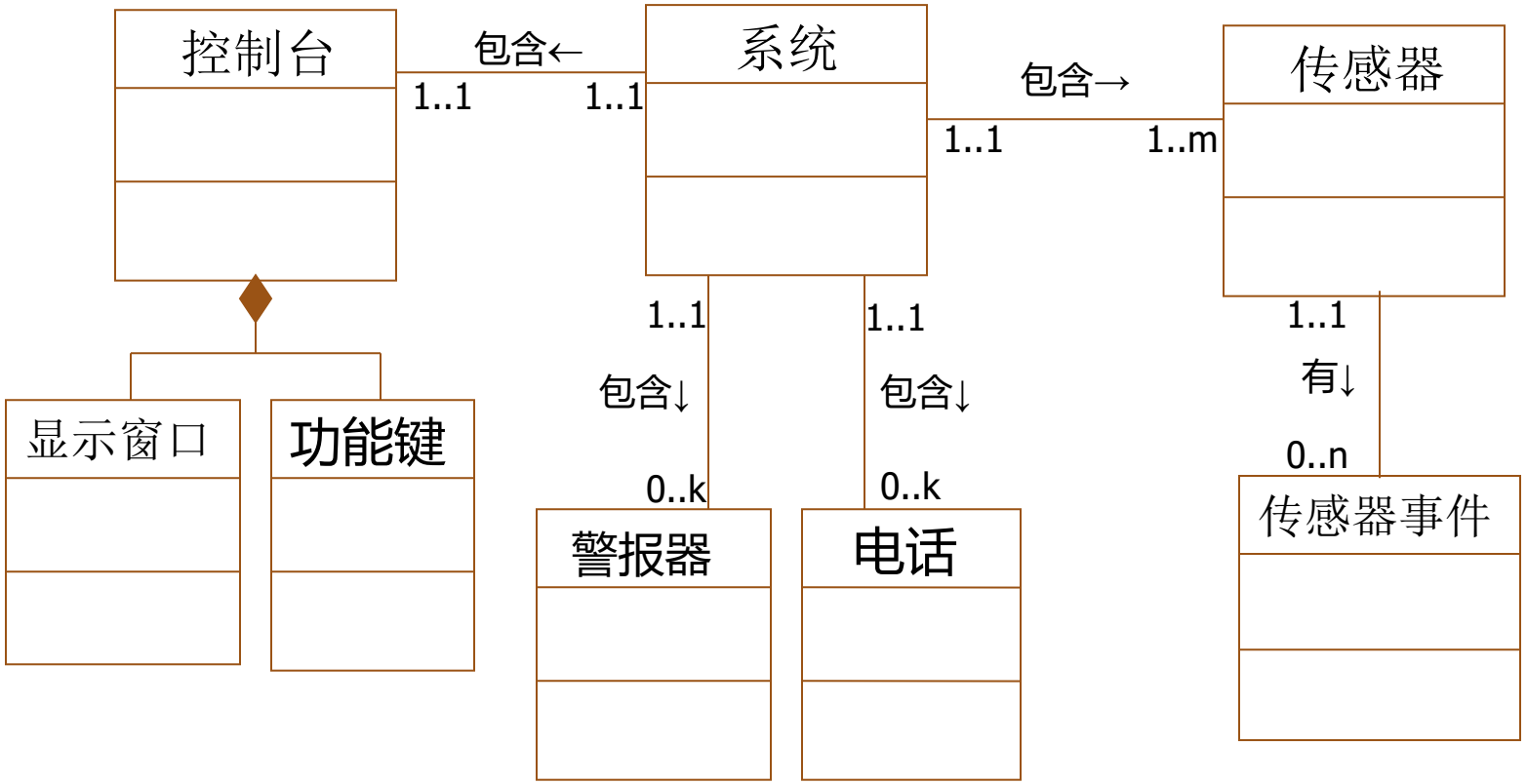
总结

- 顺序图和协作图之间的比较：
  - ■顺序图是按照时间顺序从上到下排列消息来描述系统中生命线之间的消息交互情况。
    协作图是着眼于生命线之间的链接来描述生命线之间的消息交互情况。
  - ■ 顺序图中的消息可以省略消息编号。
    协作图中的消息必须明确消息编号来表示消息发送的先后顺序。
  - ■ 顺序图中，用组合片段“par”来描述消息的并行处理。
    协作图中，用“数字+字母”的消息编号定义方式来描述消息的并行处理。
- 描述系统静态结构的类图和描述系统动态行为的交互图之间要保持一致性：
  - ■交互图中的生命线对应于类图中的类。
  - ■交互图中有消息交互的生命线，对应在类图中的两个类之间应该有一定的关联。
  - ■交互图中的消息对应于类图中接受该消息生命线所对应的类的某个操作。

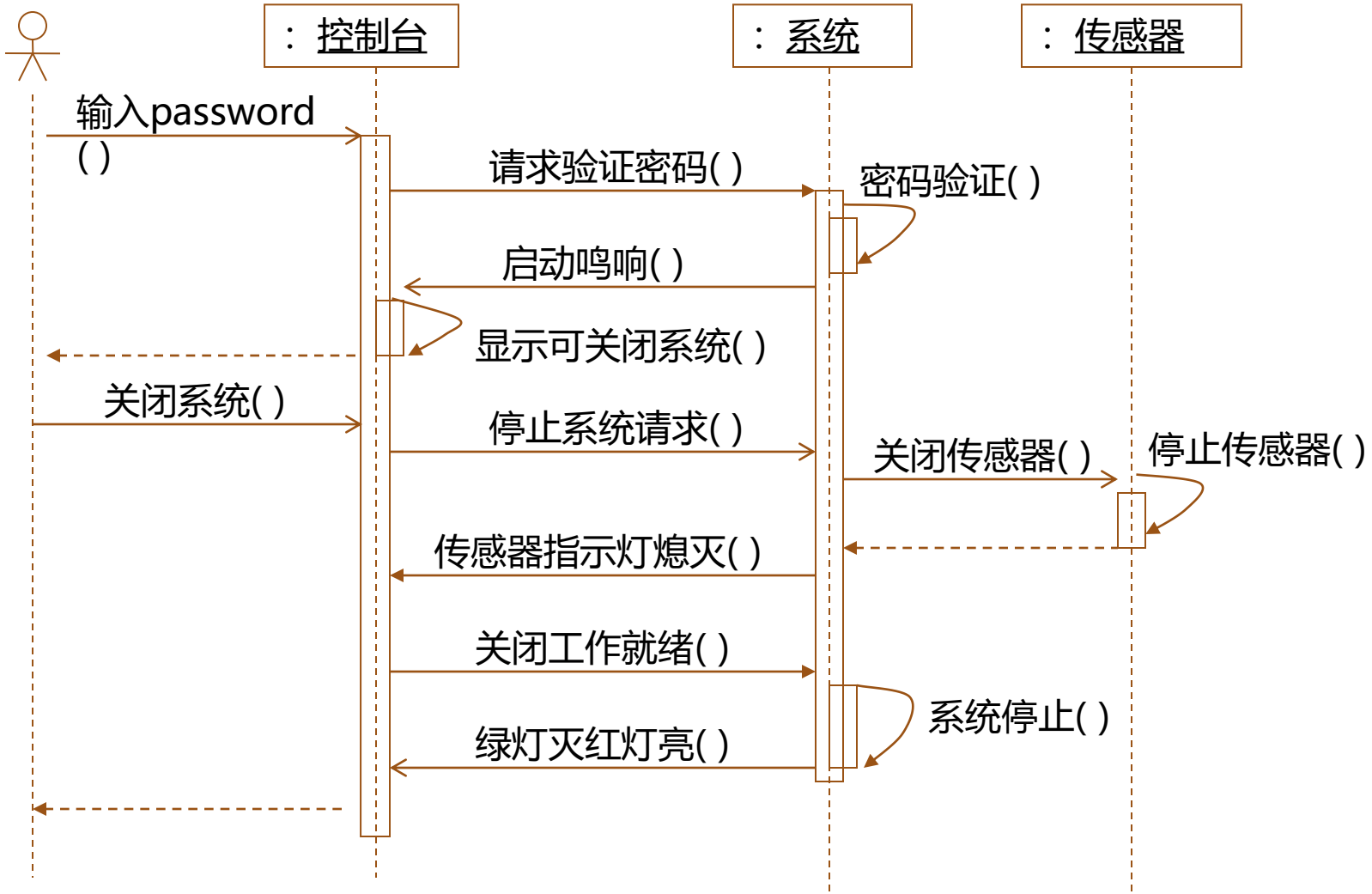
Safehome系统用例图



Safehome系统类图



SafeHome关闭系统用例的顺序图



9.5 练习题2

以下给出了一个购物支付用例的脚本描述，请根据描述画出顺序图。

S1.客户在支付界面上按下“支付”按钮

S2.系统显示购物车的内容，并询问送货地址

S3.客户输入发货地址

S4.系统询问信用卡信息

S5.客户输入信用卡信息

S6.系统向银行系统发送付款信息

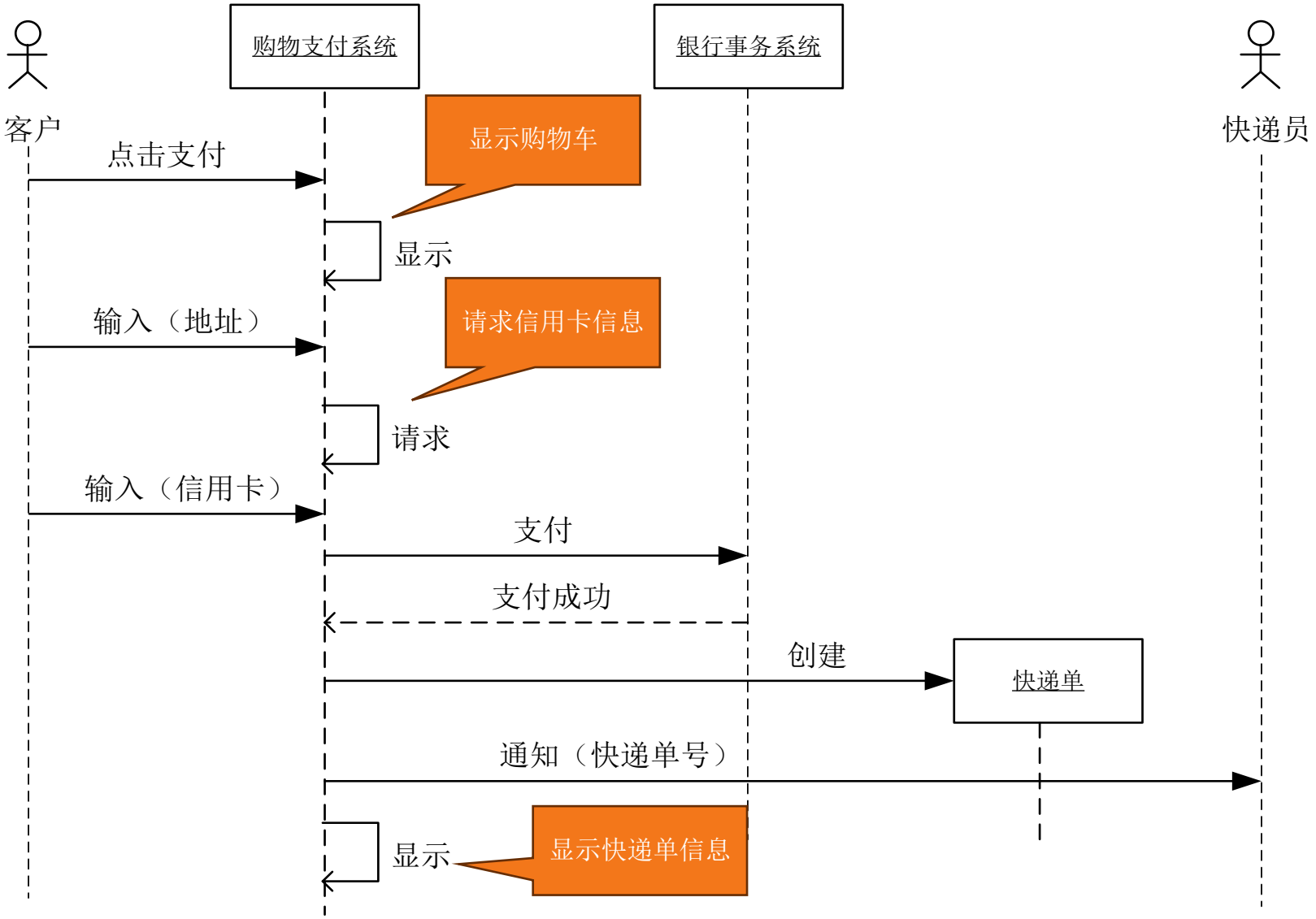
S7.银行系统返回交易正常

S8.系统创建快递单

S9.系统向配送员发送通知

S10.系统显示快递单信息页面

9.5 练习题2-解答


购物支付系统
银行事务系统
客户
快递员
显示购物车
点击支付
显示
请求信用卡信息
输入（地址）
请求
输入（信用卡）
支付
支付成功
创建
快递单
通知（快递单号）
显示
显示快递单信息

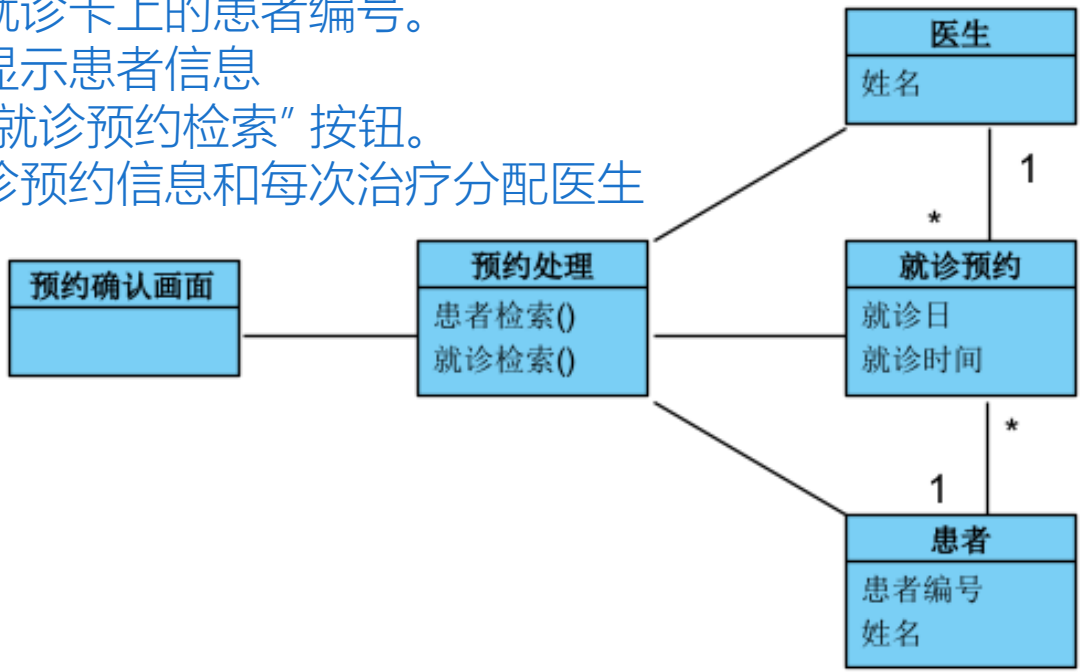
练习题: UMTP L2 sample

- 某家医院，正在讨论通过将传统的手写预约管理台账电子化，以减轻前台接待负担，提高工作效率。
- 要求系统能够完成：就诊预约(相当于门诊挂号)的登记和确认。为此患者的登记、更新和删除，医生的登记、更新和删除等功能也必不可少。
- 关于"医疗预约", 基于BCE模式整理的结构和交互如下(Boundary-Control-Entity，即边界-控制-实体模式。)

某患者就诊预约的确认处理流程如下：

1. 患者将就诊卡交给前台
2. 前台在预约确认画面上输入就诊卡上的患者编号。
3. 系统根据患者编号在画面上显示患者信息
4. 如果没有问题，前台就点击“就诊预约检索” 按钮。
5. 系统检索与该患者相关的就诊预约信息和每次治疗分配医生信息，显示在画面上

下面是整理出的类图。



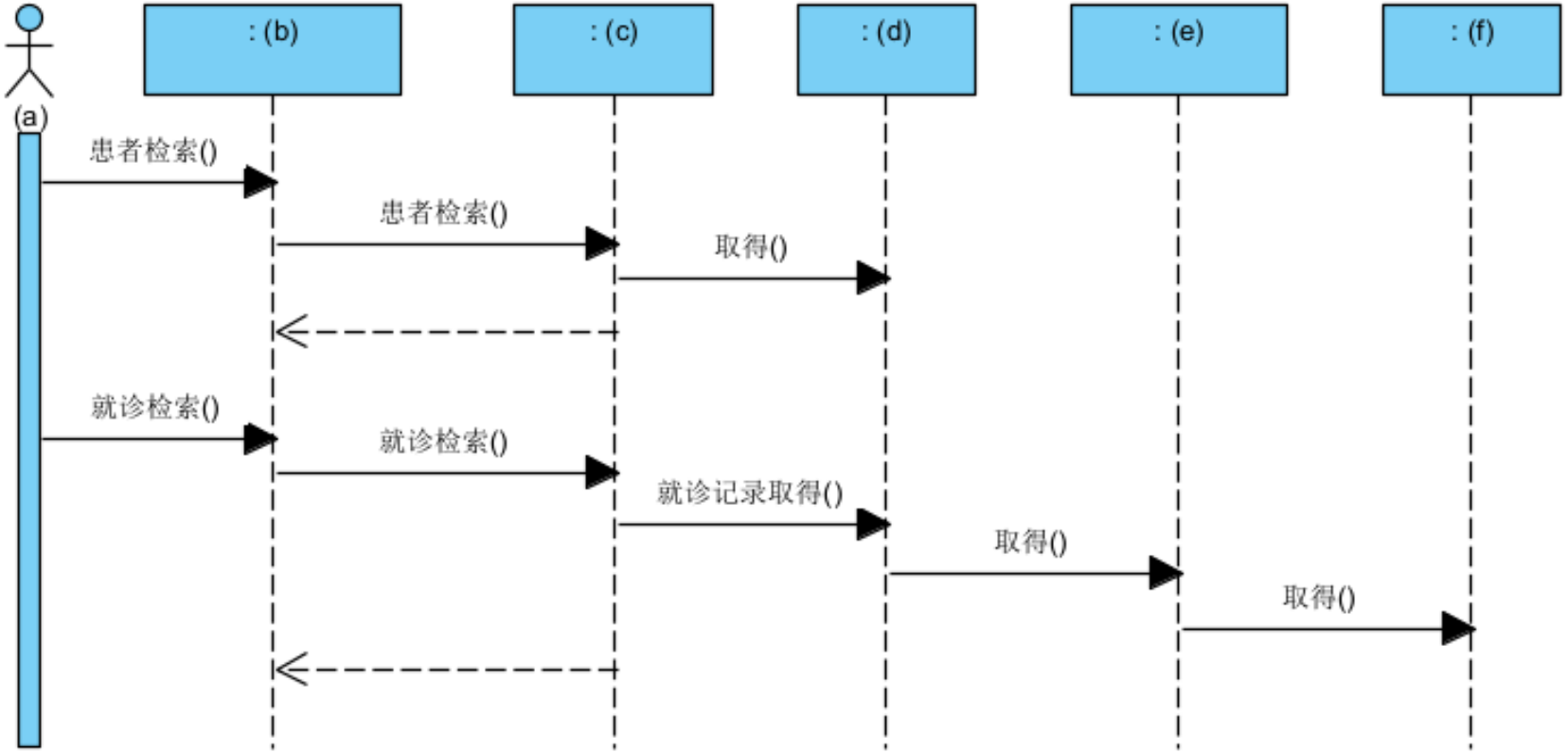
练习题: UMTP L2 sample(续)

◆选择适合顺序图的空白 (a) 至 (f) 的语句。

（1）医院 （2）医生（3）前台 （4）患者 （5）预约管理台账 （6）就诊预约

（7）就诊日 (8)就诊时间 (9)"就诊预约检索”按钮 (10)检索 (11) Actor

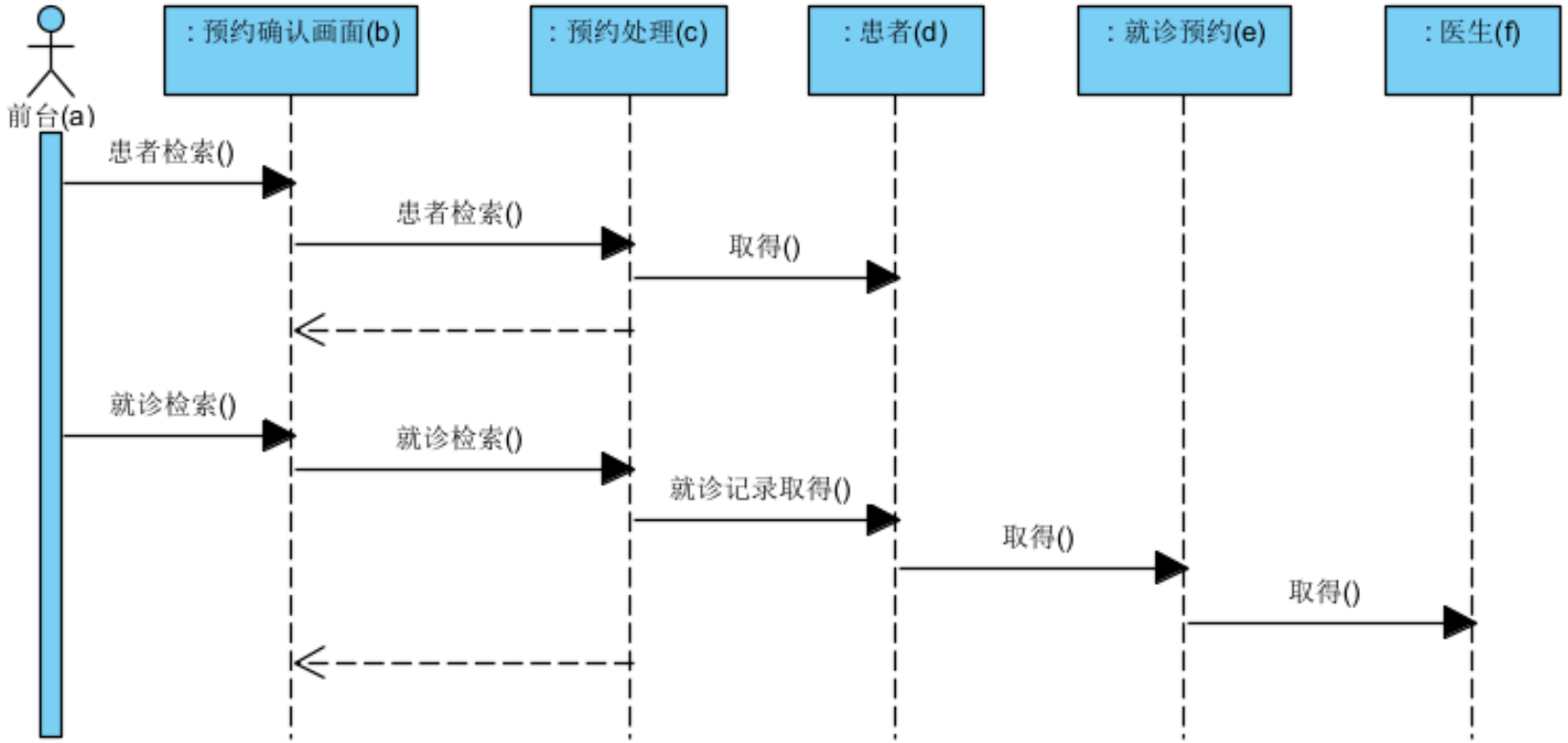
(12) 登录画面 (13) 预约确认画面 (14)预约处理



◆解答 = (a)-(3) (b)-(13) (c)-(14) (d)-(4) (e)-(6) (f)-(2)

◆选择适合顺序图的空白 (a) 至 (f) 的语句。

（1）医院 （2）医生（3）前台 （4）患者 （5）预约管理台账 （6）就诊预约 （7）就诊日 (8)就诊时间 (9)"就诊预约检索”按钮 (10)检索 (11) Actor (12) 登录画面 (13) 预约确认画面 (14)预约处理



总结

- 顺序图和协作图之间的比较：
  - 顺序图是按照时间顺序从上到下排列消息来描述系统中生命线之间的消息交互情况。
    协作图是着眼于生命线之间的链接来描述生命线之间的消息交互情况。
  - 顺序图中的消息可以省略消息编号。
    协作图中的消息必须明确消息编号来表示消息发送的先后顺序。
  - 顺序图中，用组合片段“par”来描述消息的并行处理。
    协作图中，用“数字+字母”的消息编号定义方式来描述消息的并行处理。
- 描述系统静态结构的类图和描述系统动态行为的交互图之间要保持一致性：
  - 交互图中的生命线对应于类图中的类。
  - 交互图中有消息交互的生命线，对应在类图中的两个类之间应该有一定的关联。
  - 交互图中的消息对应于类图中接受该消息生命线所对应的类的某个操作。